

Sevak Documentation

Description	User guide and reference for Sevak, a keyboard-first desktop launcher for Windows, macOS and Linux.
Repository	https://github.com/ninad-k/Sevak
Copyright	Sevak is licensed under the Apache License 2.0.

Table of Contents

[Home](#)

[Getting started](#)

[Install](#)

[Quick start](#)

[User guide](#)

[Searching and launching](#)

[Keyboard shortcuts](#)

[Settings window](#)

[Themes](#)

[Features](#)

[Features overview](#)

[Applications](#)

[Calculator and conversions](#)

[Web search](#)

[Files and folders](#)

[Browser bookmarks](#)

[Clipboard history](#)

[Snippets](#)

[Emoji picker](#)

[System commands](#)

[Automation tasks](#)

[Media controls](#)

[Shell commands](#)

[Selection actions](#)

[Contacts](#)

[1Password](#)

[Dictionary and spelling](#)

[Script plugins](#)

[Workflows](#)

[Reference](#)

[Configuration file](#)

[Command line](#)

[Files and data](#)

[Help](#)

[Troubleshooting](#)

[FAQ](#)

[Privacy](#)

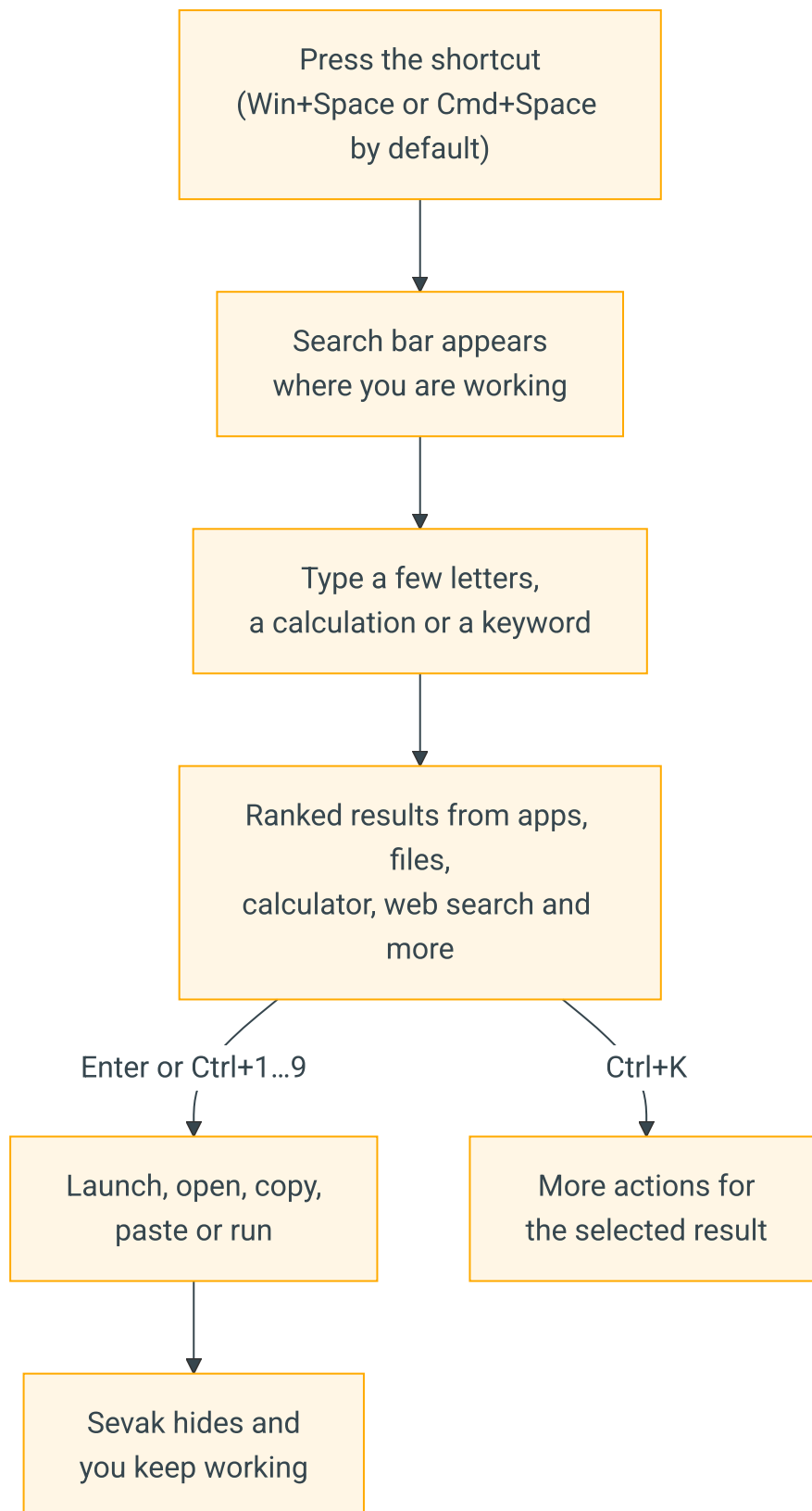
[Developers](#)

Sevak documentation

Your desktop. At your service. Sevak is a keyboard-first launcher for Windows, macOS and Linux. Press a shortcut, type what you need, and press `Enter ↵` to open an app, find a file, calculate, convert units, search the web, paste a snippet or run a command, all without leaving the keyboard.



How it works in one picture



Where to go next

New to Sevak

Everyday use

Install it, then follow the [five-minute quick start](#).

[Searching and launching](#), the [keyboard shortcuts](#) and every [feature](#) with examples.

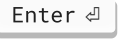
Make it yours

The [Settings window](#), [themes](#) and the complete [configuration file reference](#).

When something is off

[Troubleshooting](#), the [FAQ](#) and what Sevak does with your data in [Privacy](#).

Everything at a glance

You type	What happens	Read more
<code>fire</code>	Finds Firefox;  launches it	Applications
<code>12*7, 5 km in mi</code>	Shows the answer;  copies it	Calculator and conversions
<code>g rust traits</code>	Searches Google in your browser	Web search
<code>f report</code>	Finds files in your chosen folders	Files and folders
<code>ff budget, in invoice 2026</code>	Finds files anywhere, or by the words inside them, through your OS index	Files and folders
<code>~/Downloads/, C:\Users\</code>	Browses folders as you type	Files and folders
<code>b recipes</code>	Opens a browser bookmark	Browser bookmarks
<code>cb</code>	Shows recent clipboard entries: text, images and files (once clipboard history is turned on)	Clipboard history
<code>s sig</code>	Pastes a saved snippet	Snippets
<code>:heart</code>	Picks an emoji from a grid and pastes it	Emoji picker

You type	What happens	Read more
<code>lock , sleep</code>	Runs a system command	System commands
<code>dark mode , vol 30 , kill chrome</code>	Runs an automation task	Automation tasks
<code>pause , next , play</code>	Presses a media button; shows what is playing	Media controls
<code>> git status</code>	Runs a shell command in a terminal	Shell commands
<code>define word , spell word</code>	Looks up a word or fixes its spelling, offline	Dictionary and spelling
<code>c ada , 1p github</code>	Finds a contact or a 1Password login (both opt-in)	Contacts, 1Password
<code>⬆ Shift</code> (tap)	Previews the selected result	Preview, Text View and Grid View
Your own keyword	Runs a workflow built in Settings → Workflows	Workflows

The exact keywords above are the defaults; most can be changed. The [features overview](#) lists every keyword and how to change it.

Prefer a printed manual?

The whole user guide is also available as a single [PDF](#), generated from these pages.

Getting started

Installing Sevak

Download Sevak for your operating system from the [Releases](#) page.



Unsigned builds

Release builds are currently unsigned. Your OS may show a security warning on first run. This is normal; see the platform-specific instructions below.

Package managers (coming soon)

Packages for winget (`NinadKulkarni.Sevak`), Scoop, Homebrew and the AUR (`sevak-bin`) are prepared but not published yet; progress is tracked in [#12](#). Until then, use the installers below.

Once they are available, Scoop and AUR installs will leave updates to the package manager (Sevak's own update check is off there, and "Check for updates" in the tray names the command to run), while winget and Homebrew installs keep Sevak's own updater, which those package managers recognise.

Windows 10 / 11

1. Download the installer from [Releases](#):
2. `Sevak_<version>_x64-setup.exe` (recommended): Per-user install, no admin rights needed.
3. `Sevak_<version>_x64_en-US.msi` : Per-machine install for enterprise deployments. Needs administrator rights; MSI updates are automatic.
4. Run the installer and follow the prompts.
5. After installation, Sevak appears in your Start Menu and runs in the background at startup.



SmartScreen warning

Windows SmartScreen may show "Windows protected your PC" because the installer is unsigned. Click "More info", then "Run anyway" to proceed.



WebView2 runtime

Sevak uses Microsoft WebView2. On Windows 11 and recent Windows 10 versions it is already installed. If it is missing, the installer automatically downloads it from Microsoft (internet required).

Uninstall: Open Settings → Apps → Installed apps, find Sevak, and click Uninstall. Your config and data in `%APPDATA%\sevak\` are preserved; see [Files and data](#) to delete them.

macOS 11+

1. Download `Sevak_<version>_universal.dmg` from [Releases](#).
2. The .dmg runs natively on Apple silicon and Intel Macs.
3. Open the .dmg and drag Sevak into Applications.
4. Open Applications and click Sevak to launch it.

Gatekeeper warning

macOS blocks the first launch with "Apple could not verify Sevak" because the app is not notarized.

Click **Open Anyway** in System Settings → Privacy & Security (scroll down to find it).

Alternatively, run in Terminal:

```
xattr -dr com.apple.quarantine /Applications/Sevak.app
```

Usage

- **Menu bar:** Sevak lives in your menu bar (top-right), not the Dock.
- **Hotkey:** The default is `⌘ Cmd` + `Space`, which Spotlight uses. If Spotlight's shortcut is on, Sevak asks once whether it may turn that shortcut off; say No and Sevak uses `⌥ Option` + `Space` instead. Change it any time in Settings or the config file, and restore Spotlight's shortcut from Settings. See [Cmd+Space and Spotlight](#).
- **File search permissions:** Sevak asks for permission to index Desktop, Documents and Downloads. Decline if you don't use file search; remove those folders in Settings.
- **Pasting:** Clipboard history and snippets require Accessibility permission to paste with `Cmd+V`. Without it, they copy to the clipboard instead.
- Go to System Settings → Privacy & Security → Accessibility and add Sevak.
- If pasting stops after updating, remove Sevak and re-add it (ad-hoc signed builds are treated as new apps).
- **Contacts:** The optional contacts plugin (`[contacts] enabled = true`) asks for access to your contacts the first time you press Enter on its "Allow" row; nothing is asked at startup. Change the answer under System Settings → Privacy & Security → Contacts.
- **Snippet expansion:** Expanding snippets as you type (opt-in, `[snippets] auto_expand`) also needs **Input Monitoring** to see the keys; macOS asks the first time, and Sevak must be restarted after you allow it.

Uninstall: Delete `/Applications/Sevak.app`. Config and data remain at `~/Library/Application Support/sevak/`; see [Files and data](#) to delete them.

Ubuntu 22.04+ / Debian

Download `Sevak_<version>_amd64.deb` from [Releases](#), then install:

```
sudo apt install ./Sevak_<version>_amd64.deb
```

Start Sevak from your application menu, or run:

```
sevak
```

Uninstall:

```
sudo apt remove sevak
```

Fedora 39+

Download `Sevak-<version>-1.x86_64.rpm` from [Releases](#), then install:

```
sudo dnf install ./Sevak-<version>-1.x86_64.rpm
```

Uninstall:

```
sudo dnf remove Sevak
```

AppImage (other distributions)

Download `Sevak_<version>_amd64.AppImage` from [Releases](#), then run:

```
chmod +x Sevak_<version>_amd64.AppImage  
./Sevak_<version>_amd64.AppImage
```

Requirements

AppImages need FUSE 2 (install `libfuse2` on Ubuntu/Debian, `fuse-libs` on Fedora).

If you cannot install FUSE, extract and run instead:

```
./Sevak_<version>_amd64.AppImage --appimage-extract-and-run
```

The AppImage is built on Ubuntu 22.04 and requires glibc 2.35 or newer.

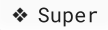


Permanent installation

Install the Applmage somewhere permanent (not `/tmp` or a temporary folder) before using `sevak --setup-hotkey`, since desktop shortcuts store the full path to the executable.

Setting up the hotkey on Linux

X11 sessions

Sevak registers hotkeys directly. Your configured hotkey works out of the box, except the default  `Super` + `Space` on GNOME, which switches input sources: Sevak then asks once whether it may move that GNOME shortcut (see below).

Wayland sessions

Applications cannot grab global keys on Wayland. Sevak provides `sevak --setup-hotkey` to ask the desktop to handle keys:

On GNOME:

```
sevak --setup-hotkey          # use hotkey from config.toml
sevak --setup-hotkey "Ctrl+Space" # or choose a different key
```



This creates GNOME custom keyboard shortcuts for:

- Main hotkey (`[general] hotkey`) → `sevak --toggle`
- Universal Actions hotkey (`[general] actions_hotkey`) → `sevak --actions`
- Each `[[hotkey]]` entry in `config.toml` → `sevak --query '<text>'` or `sevak --run <id>`

After editing `config.toml`, run `--setup-hotkey` again to update shortcuts. Delete old shortcuts in GNOME Settings → Keyboard Shortcuts if you remove config entries.

On other desktops (KDE, Sway, etc.):

Bind a key to `sevak --toggle` in your desktop's keyboard settings. The `--setup-hotkey` command only works on GNOME.



GNOME and Super+Space

GNOME binds `Super + Space` to switching input sources, which blocks Sevak's default hotkey. `sevak --setup-hotkey` (and the **Set up GNOME shortcut** button in Settings) offers to move that shortcut to `Ctrl + Super + Space`, and only does it after you confirm. It saves the old value; `sevak --restore-hotkey` or the **Restore** button puts it back. On X11, Sevak asks the same question the first time it cannot register `Super + Space`.

Prefer to leave GNOME alone? Choose `Ctrl + Space` or `Ctrl + Alt + Space` in Settings.



GNOME Alt+Space conflict

GNOME binds `Alt + Space` to the window menu by default, which blocks Sevak's hotkey if you choose it. Pick a different key or free `Alt + Space`:

```
gsettings set org.gnome.desktop.wm.keybindings activate-window-menu "[ ]"
```

Wayland rendering

By default, Sevak uses XWayland (X11 compatibility layer) on Wayland to position itself and take focus reliably. You can use the native Wayland backend instead by setting `[linux] wayland_use_xwayland = false` in your config, though the window may appear off-centre or not accept input.

Tray icon on GNOME (Ubuntu and Fedora)

Sevak displays a tray icon through the AppIndicator protocol.

Ubuntu: The AppIndicator extension is enabled by default. You should see the tray icon.

Fedora: Install the AppIndicator extension:

```
sudo dnf install gnome-shell-extension-appindicator
```

Then enable it in GNOME Extensions or at extensions.gnome.org, or restart GNOME Shell:

```
killall -HUP gnome-shell
```

Without a tray: Sevak still runs. Use the hotkey to show it, and run `sevak --quit` to exit. Open Settings with `sevak --settings`.

Red Hat Enterprise Linux 9 and clones (Rocky, AlmaLinux)



Not supported on RHEL 9 / Rocky 9 / AlmaLinux 9

Prebuilt packages do not work on EL9. Sevak uses Tauri v2, which requires WebKitGTK 4.1 (`webkit2gtk4.1`). EL9 distributions only ship WebKitGTK 4.0 (`webkit2gtk3`), which is not compatible and is not available in any EL9 repository (BaseOS, AppStream, CRB, or EPEL 9).

The AppImage also fails to run on EL9 (compiled against glibc 2.35, EL9 has 2.34).

Workarounds on EL9:

- Run Sevak in a **Fedora or Ubuntu container** (e.g. distrobox) with display access.
- **Build from source** on a Fedora or newer distribution.
- Use **Fedora or Ubuntu** directly.
- Wait for **EL10 with EPEL**: AlmaLinux 10 reportedly ships `webkit2gtk4.1` in EPEL 10, so the `.rpm` is expected to work on EL10 clones.

Your first five minutes with Sevak

A quick walkthrough to get you searching and launching.

1. Install and open

Download from [GitHub Releases](#) and follow the [platform-specific instructions](#).

On **Linux Wayland**, configure your desktop hotkey first: [Setting up the hotkey on Linux](#).

Open Sevak once. You'll see a tray or menu-bar icon that keeps it always accessible.

2. Bring up the search bar

Press  **Win** +  **Space** ( **Cmd** +  **Space** on macOS,  **Super** +  **Space** on Linux) to open the launcher.

That key already means something to your system (Windows switches the input language, macOS opens Spotlight), so Sevak takes it over. On Windows nothing needs to be asked. On macOS and GNOME, Sevak asks once whether it may turn the system shortcut off; say No and it uses  **Option** +  **Space** instead. See [the Win+Space, Cmd+Space and Super+Space notes](#).

A dark blue rounded rectangular search bar with a flame icon on the left and the text "Type to search..." in a light blue font.

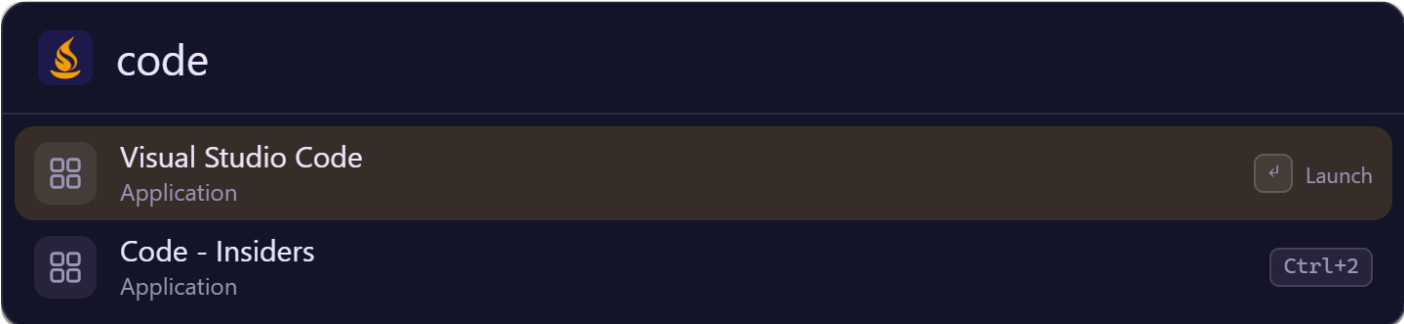
Type immediately—the cursor is already in the search box. Press  **Esc** to hide it.

If the shortcut does not work or is already used by another app, open **Settings** from the tray and pick another one (the **Presets** list has  **Alt** +  **Space**).

3. Launch an app

Type the name of an installed app: `code`, `firefox`, `vs code`.

Use  **Up** /  **Down** to pick the right result, then press  **Enter** to launch.

A dark blue rounded rectangular search results window. At the top is the Sevak flame icon and the text "code". Below is a list of two items: "Visual Studio Code" and "Code - Insiders". Each item has a small icon on the left, the name in bold, the word "Application" below it, and a button on the right. The "Visual Studio Code" button says "Launch" and the "Code - Insiders" button says "Ctrl+2".

 **code**



Visual Studio Code
Application



Launch



Code - Insiders
Application

Ctrl+2

Sevak learns what you use most and ranks results by frequency and recency. Recent searches reappear first.

4. Do a quick calculation

Type `12*7` and press `Enter ↵` to copy the result (`84`).

Paste it anywhere. Try `sqrt(16)` , `2^10` , `(125+75)/4` , or `10 km in mi` .

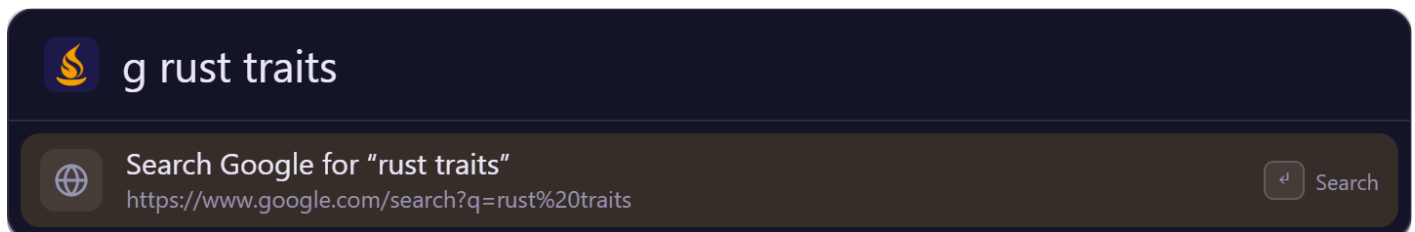


Sevak handles arithmetic, unit conversion, and more. See [Calculator](#).

5. Find a file or search the web

Query	Result
<code>f project</code>	Finds files/folders matching "project" in your indexed folders (default: Desktop, Documents, Downloads)
<code>g rust traits</code>	Opens Google with that search in your browser
<code>yt svelte</code>	YouTube search
<code>gh tauri</code>	GitHub search

Important: put a space after the keyword (`f` , `g` , `yt` , `gh`).



Make it yours

Five quick customizations in **Settings**:

1. **General:** Change the hotkey and set whether Sevak starts at login
2. **Appearance:** Choose Light, Dark, or System theme; adjust font size and window width
3. **Search:** Set the max number of results and fallback web engine
4. **Files:** Add more folders to search
5. **Plugins:** Turn features on or off (Calculator, Bookmarks, Clipboard history, etc.)

Click **Save** when done.

More options: [Configuration guide](#).

Watch it in action

[12-second workflow demo with captions](#) (real interface, sample results).

What's next?

- **Search like a pro:** [Searching and launching](#) — everything you can search and how results are ranked
- **All keyboard shortcuts:** [Keyboard reference](#)
- **Universal Actions:** [Selection-based actions](#) — search, transform, calculate or run actions on text, links or files from any app
- **Deep customization:** [Configuration guide](#) — hotkeys, themes, custom search engines and more
- **Stuck?** [Troubleshooting](#) or [FAQ](#)

Enjoy! Sevak is here to save you time.

User guide

Searching and launching

Learn how Sevak works: opening it, typing queries, selecting results and running actions.

Opening and closing

Show the launcher

Press your global shortcut (default:  Win +  Space on Windows,  Cmd +  Space on macOS,  Super +  Space on Linux). The search bar appears centered on your screen. Sevak takes this key over from the system; see [Win+Space, Cmd+Space and Super+Space](#), including how to go back to  Alt +  Space.

On **Linux Wayland**, configure the shortcut first: [Setting up the hotkey](#).

Hide the launcher

- Press  Esc while the launcher is open (it first closes whatever is open on top: Large Type, the Text View, the action panel, the preview pane, a folder picker or a workflow's text output)
- Click outside the search bar (if "Hide when focus is lost" is on in Settings)
- Run a result and "Hide when focus is lost" is enabled

Hiding does not quit Sevak. It stays in the background until you press the hotkey again.

Tray menu and system integration

Click the tray icon (Windows, Linux) or menu-bar icon (macOS) to access:

Menu item	Action
Show	Open the launcher
Settings	Open the Settings window
Reload index	Re-read config, rebuild app/file/bookmark indexes, load new script plugins
Check for updates	Look for a new version on GitHub
Quit	Exit Sevak

Searching

Typing a query

The cursor is in the search box as soon as Sevak opens. Start typing immediately—no need to click.

Sevak searches as you type. Results update in real time.

How results are ranked

Sevak combines three factors:

1. **Query match:** how well your typing matches a result (fuzzy matching allows typos and abbreviations)
2. **Frequency:** how often you have run that result recently
3. **Recency:** how long ago you last ran it

Results using frequently and recently rank higher. This means your most-used apps and files appear first, even with a short query.

Examples: - Type `cod` → sees `code` (VS Code), `codeserver`, and `mycode.txt`; if you use VS Code most, it ranks first - Type `pro` → apps like `Prometheus` mix with files like `project.md`, `proposal.docx`; your recent choices decide the order

New installations: on first run, all results are equally ranked. As you use Sevak, personalization kicks in.

Query history

On an **empty search bar**, press `↑ Up` and `↓ Down` to step through your last 50 searches, newest first.

Typing leaves history. Toggle this on or off in **Settings** → **Search** → **Remember searches** or with `[search] query_history`.

Viewing results

The launcher shows a list of matching results. Each row displays:

- **Icon:** the app, file type, or plugin icon
- **Title:** the name of the app, file, or result
- **Subtitle:** additional info (folder path, app description, calculation result, etc.)
- **Action hint:** what pressing `Enter ↵` will do (Launch, Open, Copy, etc.)

Use `↑ Up` / `↓ Down` or `^ Ctrl` + `P` / `^ Ctrl` + `N` to select a result.

Press **++pageup++** or **++pagedown++** to jump by a page (7 results).

Running a result

Press  to run the selected result:

- **Apps:** launch the application
- **Files:** open with the default application (or folder in file manager)
- **Web search:** open the URL in your browser
- **Calculations:** copy the result to your clipboard
- **Clipboard history:** paste the text into the app that had focus before Sevak opened
- **Snippets:** paste the template (with placeholders filled) into the previous app
- **Custom results:** defined by each plugin

Running from keyboard numbers

Press  +  through  +  to run the corresponding result in the list without arrow keys. The numbers match the rows 1–9.

Result actions

Most results can be used in more than one way. The row at the bottom of the list shows what the modifier keys do:

Shortcut	Effect
	Run the primary action (Launch, Open, etc.)
 + 	Alternative action bound to Ctrl, if available
 + 	Alternative action bound to Shift, if available
 + 	Alternative action bound to Alt, if available

Action panel

Press  (with the cursor at the end of the text) or  +  to open the action panel, which lists **every available action** for the selected result.

In the action panel:

-  /  or  +  /  +  : select an action
-  : run the selected action

- **^ Ctrl1 + 1** through **^ Ctrl1 + 9** (Universal Actions only): run the numbered action
- **⌘ Esc** or **← Left** or **^ Ctrl1 + K**: close the panel

Copying and Large Type

Copy a result: press **^ Ctrl1 + C** (when nothing is selected in the text input) to copy the result's value, path, or URL to your clipboard.

Large Type: press **^ Ctrl1 + L** to show the selected result's text huge across the screen. Press any key to dismiss it. Useful for reading: - Long file paths - URLs - Calculation results - Large text from the selection (Universal Actions)

Preview, Text View and Grid View

Preview pane

Tap **⇧ Shift** (press and release it alone) or press **^ Ctrl1 + Y** (**⌘ Cmd + Y** on macOS) to open a pane under the list that shows what the selected result is. It follows the selection as you move; tap **⇧ Shift** or press **^ Ctrl1 + Y** again, or **⌘ Esc**, to close it (the first **⌘ Esc** closes the pane, the next hides Sevak). It shows:

Result	The pane shows
Text and code files	The first 64 KB in a monospace font, with size and modified date
Images (PNG, JPEG, GIF, WebP, SVG, BMP, ICO; up to 4 MB)	The picture and its pixel size
PDFs and other binary files	Kind, size, path and modified date (pages are not rendered)
Folders	The first 100 entries, folders first, and the item count
Applications	Kind, path or launch command, and the version when it is cheap to read (macOS apps)
Web results and bookmarks	The address, title and site. Nothing is fetched from the network
Snippets	The text as Enter ↵ would paste it, with {date} and the other placeholders filled in

Result	The pane shows
Clipboard history entries	The copied text, the full image, or the copied file (several files: their paths)
Calculator and conversions	The result and the calculation
Emoji	The emoji large, with its name, keywords and code points

The pane reads only the file or folder the selected result refers to, only when it is open, and never reads network locations (`\\server\share`). The window grows to make room and shrinks back when you close the pane; near the bottom of a small screen it moves up so the pane stays visible.

Text View

A result that carries a long text (a long or multi-line clipboard entry, a snippet, a script plugin's output) can be opened in a scrollable, taller view with `^ Ctrl + T`. `↑ Up` `↓ Down` `++pageup++` `++pagedown++` `↑ Home` `↓ End` scroll; `^ Ctrl + C` copies the text, `Enter ↵` runs the result as usual, and `⌫ Esc` or `← Left` goes back to the list. Rows that exist only to show text (marked by the script that produced them) open the Text View on `Enter ↵`.

Grid View

Plugins that offer pictures show their results as a grid of tiles instead of a list, when every result of a search is a tile. Move with `← Left` `→ Right` `↑ Up` `↓ Down` (`++pageup++` / `++pagedown++` jump three rows), `Enter ↵` runs the selected tile, `^ Ctrl + K` opens its actions, and the preview pane (`↑ Shift` or `^ Ctrl + Y`) works as for list rows. The grid shows up to 60 tiles. Built in:

- The [emoji picker](#): `:heart` or `emoji heart`.
- Copied images in [clipboard history](#): `cb image`.
- Script plugins can request tiles too: see [Views: text and grid](#).

Tab completion

Press `Tab →` to complete the input to the selected result's name or value. The text in the search box is replaced.

Examples: - Searching apps: **Tab** fills in the selected app's full name - File browsing: **Tab** completes to the selected file or folder name (folders get a trailing slash so you can keep drilling) - Web search: **Tab** completes to `keyword` (the keyword with a space, ready for your search terms)

When browsing a folder path, `↑ Shift + Tab →` goes back up one level.

Plugins and features

Sevak includes several built-in result sources. Each can be turned on or off in **Settings** → **Plugins**:

Applications

Type part of an installed application's name. Fuzzy matching lets a short query find a longer name.

Sources differ by platform: - **Windows**: Start Menu shortcuts and installed packages - **macOS**: application bundles - **Linux**: desktop entries

After installing a new app, choose **Reload index** from the tray menu if it doesn't appear immediately.

More: [Applications](#) plugin.

Calculator

Enter an expression directly; no keyword is required. Results update as you type.

Enter  copies the number to your clipboard.

Examples: - Arithmetic: `12*7`, `(125+75)/4` - Powers: `2^10` - Functions: `sqrt(16)`, `sin(1.5)`, `ln(100)` - Factorial: `5!` - Remainder: `17%5`

Unit conversion

Type `<amount> <unit> (in|to|as|=) <unit>`. The amount can be any expression: `(2+3) km in m`.

Supported: length, mass, temperature, volume, area, speed, data, time, pressure, energy, angle.

Examples: - `10 km in mi`, `5'11" to cm`, `3 ft 4 in to cm` - `100°F in C`, `0 c to k` - `1 kg in lb`, `8 oz to g` - `5 GB in MiB`, `100 Mbit to MB` - `90 min to h`

For data: `MB` is megabytes (1,000,000 bytes); `MiB` is mebibytes (1,048,576 bytes). `KB` / `MB` / `GB` are powers of 1000; `KiB` / `MiB` / `GiB` are powers of 1024.

Currency conversion

Optional and off by default (requires network). Enable in **Settings** → **Plugins** → **Currency conversion**.

When on, `100 usd in eur` works. Use ISO codes, signs or words: `100 usd in eur`, `50 € to $`, `$100 in eur`.

The subtitle shows the exchange rate and the European Central Bank's publication date.

Rates are downloaded at most once per day and cached locally.

More: [Calculator](#) plugin.

Files and folders

Type **f** (keyword + space) followed by a filename or folder name to search indexed directories.

f searches **names** in the folders you chose. Default roots: Desktop, Documents, Downloads.

Whole disk and inside files: `ff report` asks your computer's own file index (Windows Search, Spotlight, `locate`) for names anywhere, and `in invoice 2026` for words inside documents. Queries stay on your computer. See [Whole-disk and content search](#).

By default, dot-files and cache folders (node_modules, .git, etc.) are excluded.

Browsing a path: type a full path (e.g., `~/Documents/`, `/etc/`, `C:\Users\`) to list a folder directly. **Tab** completes entries; `↑ Shift` + `Tab →` goes up; `Enter ↵` opens.

File buffer: press `⌘ Alt` + `↓ Down` on file results to collect them, then `⌘ Alt` + `→ Right` to open, move, copy, zip or trash them all at once. See [File buffer](#).

Configure in **Settings** → **Files** or with `[files]` in config.toml.

More: [Files](#) plugin.

Bookmarks

Type **b** and part of a bookmark's title or URL to search bookmarks from Chrome, Edge, Firefox, Safari and other browsers.

Sevak reads your bookmarks across all profiles, read-only.

`Enter ↵` opens the URL in your default browser.

More: [Bookmarks](#) plugin.

Clipboard history

Optional and off by default. Enable in **Settings** → **Plugins** or with `[clipboard] enabled = true`.

Type **cb** to search what you copied recently: text, images and files. `Enter ↵` pastes the entry into your previous app. `cb image` shows copied images as a grid of thumbnails.

Sevak never records content password managers mark as secret, copies made in apps listed in `ignore_apps`, very long text or very large images. Images are kept as PNG files in the data folder, unencrypted; turn them off with `[clipboard] images = false`.

More: [Clipboard history](#) plugin.

Snippets

Type `s` and a snippet name to search your saved text templates. `Enter ↵` pastes the template (with placeholders filled in) into your previous app.

Placeholders: - `{date}`, `{time}`, `{datetime}`: today, current time, both - `{date:FORMAT}`: custom [strftime](#) format, e.g. `{date:%d %B %Y}` - `{clipboard}`: the current clipboard text - `{uuid}`: a new random UUID - `{{ , }}`: literal `{` and `}`

Configure in `[paste]` and with `[[snippet]]` entries in `config.toml`.

Expand as you type (off by default): with `[snippets] auto_expand = true` (or **Settings** → **Plugins**), typing a snippet's `keyword` in any app replaces it with the snippet. While on, Sevak watches your keystrokes, keeping only the last 64 characters in memory. See [Expand snippets as you type](#).

More: [Snippets](#) plugin.

System commands

Type a command's name: `lock`, `sleep`, `restart`, `shutdown`, `logout`, `empty trash`, or settings pages (`bluetooth`, `display`, `wifi`, `sound`, etc.).

Dangerous actions (restart, shut down, logout, empty trash) ask for confirmation first. You can disable the ask or hide commands in **Settings** → **Plugins** or with `[system]` in `config.toml`.

More: [System commands](#) plugin.

Automation tasks

Type a task's name, or `t` to list them all: `dark mode`, `vol 30`, `screenshot`, `quit`, `kill chrome`, `eject`, `awake 45`, `wifi`, `flush dns` ... Only the tasks that work on your machine are offered. Force quit, kill and restarting Explorer or Finder ask first.

More: [Automation tasks](#).

Media controls

Type `play`, `pause`, `next`, `previous` or `stop` to press the media button of whatever is playing. `play` (with a space) or `music` also shows the current track.

More: [Media controls](#).

Shell commands

Type `>` (or `>`) followed by a shell command. `Enter ↵` opens your terminal and runs it.

`>` (space only) shows your recent commands and an "Open terminal" row.

Nothing runs until you press Enter—the launcher only prepares the command.

Terminal auto-detection by platform; customize in **Settings** or with `[shell]` in config.toml.

More: [Shell](#) plugin.

Web search

Type a **keyword and your terms**: `g rust traits`, `yt svelte tutorial`, `gh tauri`.

Built-in keywords: - `g` : Google - `yt` : YouTube - `gh` : GitHub

`Enter ↵` opens the constructed URL in your browser.

When a plain query has no results, a fallback web search is offered (default: Google). Configure or add custom engines in **Settings** → **Web search** or with `[[web_search]]` in config.toml.

More: [Web search](#) plugin and [adding custom engines](#).

Script plugins

Add your own keywords without building Sevak. Put a folder with `plugin.toml` and a script in the `plugins` folder next to `config.toml`, then **Reload index**.

Scripts run with your permissions (not sandboxed), so install only plugins you trust.

Three examples are in `examples/plugins/`. The full guide is [Plugins](#).

Workflows

Chain a trigger (a keyword, a script filter, a hotkey, a Universal Actions entry or `sevak --trigger`) to actions and outputs, built as boxes and connectors in **Settings** → **Workflows**: type `issue 14` to open issue 14, or select text in any app and have a workflow tidy it and paste it back. Start from one of the templates in **New from template....**

A workflow that runs scripts or commands asks for your permission first, and again if what it runs changes. **Settings** → **Gallery** installs ready-made workflows and script plugins, but only after you press **Load gallery** and **Install**.

More: [Workflows](#).

Emoji picker

Type `:` and a name (`:heart`) or `emoji` and a name. Matches are shown as a grid; `Enter ↵` pastes the emoji,

`⇧ Shift` + `Enter ↵` copies it.

More: [Emoji picker](#).

Contacts and 1Password

Both are **off by default**. `c` `ada` or `@ada` searches your address book (vCard files and the system address book); `Enter` copies the e-mail address. `1p` `github` lists your 1Password logins through the official `op` tool; `Enter` opens the website. Sevak never reads passwords.

More: [Contacts](#) and [1Password](#).

Dictionary and spelling

`define` `serendipity` shows definitions; `spell` `recieve` lists corrections and `Enter` pastes the right spelling. Everything is offline.

More: [Dictionary and spelling](#).

Universal Actions

Select something in any app—text, a link, files—press `^ Ctrl` + `⌘ Alt` + `Space` (or your configured hotkey) and Sevak shows what you can do with it.

Reading the selection

Sevak reads your selection by:

1. Saving your clipboard
2. Pressing `^ Ctrl` + `C` (`⌘ Cmd` + `C` on macOS) in the app you were using
3. Waiting for the copy (up to ~0.3 seconds)
4. Reading the text or files
5. Putting your clipboard back

Nothing is stored. The selection is never logged or recorded. Your OS clipboard history (Windows Win+V, etc.) or a clipboard manager *might* see it because the app itself makes the copy.

On **terminal windows** (Windows Terminal, xterm, etc.), Sevak skips the copy to avoid interrupting a running program; use the mouse and the clipboard fallback instead.

On **Wayland**, app-to-app clipboard reading is not possible. Enable **Use the clipboard if the selection can't be read** in Settings to act on the clipboard instead.

What you can do

Selection	Actions
Text	Search with each web engine, show as Large Type, copy, paste as plain text, calculate (if it's a math expression or unit conversion), transform: Uppercase,

Selection	Actions
	Lowercase, Title Case, Trim, URL-encode, URL-decode, Base64 encode, Base64 decode, Pretty-print JSON, Minify JSON
URLs (<code>http://</code> , <code>https://</code> , <code>mailto:</code> , <code>www.</code>)	Open, copy, Large Type
Files and folders	Open, show in file manager, copy path(s), open in terminal (folders), run as admin (Windows programs), send to Sevak (fills search box with the path)
Path as text	File actions above, then text actions

Running actions

Actions are shown in the panel. Use:

- `↑ Up` / `↓ Down` : select an action
- `Enter ↵` : run the selected action
- `^ Ctrl` + `1` through `^ Ctrl` + `9` : run the numbered action directly
- `^ Ctrl` + `Enter ↵` : run the action but copy instead of pasting/replacing
- `⇧ Shift` + `Enter ↵` (web search): copy the URL instead of opening it
- `⌫ Esc` : close the panel and return to the launcher

Where pasting works (Windows, macOS outside Wayland, Linux X11): - Transformations and calculator results **replace the selected text** in the app - `^ Ctrl` + `Enter ↵` copies instead of pasting

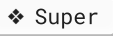
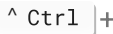
Where pasting doesn't work (macOS without permission, Wayland): - Results are **copied** only

More: [Universal Actions](#).

Keyboard shortcuts

Complete reference: [Keyboard shortcuts](#).

Quick summary:

Context	Shortcut	Action
Launcher	 Win +  Space /  Cmd +  Space /  Super +  Space	Show or hide (or your configured hotkey)
Navigation	 ↑ Up /  ↓ Down or  ^ Ctrl +  P /  ^ Ctrl +  N	Previous / next result
Running	 Enter ↵	Run selected result
Actions panel	 → Right or  ^ Ctrl +  K	Open actions panel
Panel	 ↑ Up /  ↓ Down	Select action
Completion	 Tab →	Complete to selected result
History	 ↑ Up /  ↓ Down (empty bar)	Recall searches
Copying	 ^ Ctrl +  C	Copy result value/path/URL
Large Type	 ^ Ctrl +  L	Show as Large Type
Preview	 ↑ Shift (tap) or  ^ Ctrl +  Y	Show or hide the preview pane
Text View	 ^ Ctrl +  T	Read the result's long text in full
File buffer	 ⌘ Alt +  ↑ Up /  ⌘ Alt +  ↓ Down	Add the file to the buffer and move
Close	 ⌫ Esc	Close the innermost panel or view, then hide the launcher

Command line

These assume `sevak` is on your path; otherwise use the full path to the executable.

```
sevak                                # Open the launcher
sevak --toggle                       # Toggle the launcher (open or hide)
sevak --query "> "                   # Open the launcher with text pre-filled
sevak --run system:lock              # Run a result by ID without showing the launcher
sevak --actions                      # Universal Actions for the current selection
sevak --trigger my-flow/go some text # Start a workflow's external trigger
sevak --background                   # Start without showing the launcher
sevak --settings                     # Open Settings
sevak --quit                         # Quit Sevak
sevak --setup-hotkey                 # Configure GNOME shortcuts on Linux
sevak --restore-hotkey               # Put back system shortcuts Sevak changed (GNOME, macOS Spotlight)
sevak --setup-hotkey Ctrl+Space      # Set up a specific key on Linux
sevak --config ~/Dropbox/sevak       # Use another config folder
sevak --help
sevak --version
```

Only one instance runs at a time; subsequent invocations forward their action.

See also

- [Keyboard reference](#) — every key Sevak handles
- [Universal Actions in depth](#)
- [All plugins and features](#)
- [Workflows](#)
- [Configuration guide](#)

Keyboard shortcuts

Complete reference to every key Sevak handles.

Global hotkeys

These work anywhere on your system:

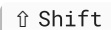
Shortcut	Action	Configure
 Win +  Space ( Cmd +  Space on macOS,  Super +  Space on Linux)	Show or hide the launcher	<code>[general] hotkey</code>
 ^ Ctrl +  Alt +  Space	Universal Actions for your current selection	<code>[general] actions_hotkey</code> (or empty to disable)
Custom <code>[[hotkey]]</code> entries	Type a query or run a result without showing the launcher	<code>[[hotkey]]</code> in <code>config.toml</code> or Settings → Hotkeys
A workflow's hotkey trigger	Start that workflow	The trigger node in Settings → Workflows
A snippet's keyword, typed in any app	Replace it with the snippet (only with expansion turned on)	<code>[snippets] auto_expand</code>

On **macOS**,  Alt is  Option, and  ^ Ctrl is  Cmd. The default launcher key Super+Space is  Cmd +  Space there. The system uses that key too (Windows: input language, macOS: Spotlight, GNOME: input sources), so Sevak takes it over: [how](#), [and how to undo it](#).

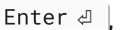
On **Linux Wayland**, apps cannot register global keys; run `sevak --setup-hotkey` to bind the launcher shortcut in GNOME. For the effect of a custom `[[hotkey]]` there, bind a desktop shortcut to `sevak --query "<text>"` or `sevak --run <result-id>` yourself (see [Command line](#)).

In the launcher window

Navigation and selection

Key	Action
 or  + 	Previous result
 or  + 	Next result
	Previous page (7 results)
	Next page (7 results)
	Complete the input to the selected result's name/value
 + 	Go up one folder while browsing a path

Running results and actions

Key	Action
	Run the selected result
 +   +   + 	Run the selected result's alternative action bound to that modifier, if it has one. The row shows which ones it offers (for example  + ), and each feature page lists them
 +  to  + 	Run the 1st to 9th result in the list directly

Actions panel

Press  (with cursor at the end of text) or  +  to open; shows every action of the selected result or your selection.

Key	Action
or +	Previous action
or +	Next action
	Run the selected action
+ to +	Run the corresponding action (Universal Actions only)
or + or	Close the panel (in Universal Actions, hides Sevak, as there is nothing behind the panel)
Any other key	Closes the panel and acts as usual (typing goes to the search bar)

When using Universal Actions, each action's row shows its modifiers (, ,) for alternative behavior.

Display and copying

Key	Action
+	Show the selected result as Large Type across the screen (dismiss with any key); for a contact, its phone number
+	Copy the selected result's path, URL, or value (when nothing is selected in the text input)
(tap) or +	Show or hide the preview pane for the selected result
+	Open the selected result's long text in the Text View

Text View

Key	Action
↑ Up ↓ Down ++pageup++ ++pagedown++ ↑ Home ↓ End	Scroll
^ Ctrl + C	Copy the text
Enter ↵	Run the result as usual
⌫ Esc or ← Left	Back to the list

Grid View

When every result is a tile (the [emoji picker](#), `cb image`, some script plugins):

Key	Action
← Left → Right ↑ Up ↓ Down	Move between tiles
++pageup++ / ++pagedown++	Jump three rows
Enter ↵	Run the selected tile
^ Ctrl + K	The tile's actions

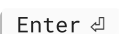
File buffer

On a file or folder result (see [File buffer](#)); `⌘ Alt` is `⌘ Option` on macOS:

Key	Action
⌘ Alt + ↑ Up / ⌘ Alt + ↓ Down	Add the selected result to the buffer and move up / down
⌘ Alt + ← Left	Remove the last item (only while the buffer holds something)

Key	Action
 + 	Open the buffer's actions (only while the buffer holds something)
 +  /  + 	Empty the buffer (only while the buffer holds something)

In the folder picker of **Move to...** / **Copy to...**:

Key	Action
 → /  Shift + 	Open the highlighted folder / go up
	Use the highlighted folder
 + 	Use the path exactly as typed
	Cancel

Search history

On an **empty search bar**:

Key	Action
	Step to the next earlier search (most recent first)
	Step to the next later search

Typing leaves history. Toggle history on or off with `[search] query_history`.

Closing

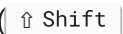
Key	Action
 Esc	Close the innermost thing first (Large Type, the Text View, the action panel, the preview pane, the folder picker of Move to... / Copy to..., a workflow's text output), then hide the launcher

Settings window

General navigation

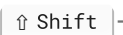
Key	Action
 Ctrl +  S	Save the current settings (in the workflow builder: save the workflow)
 Down or  Up	Move between tabs (when a tab button has focus)
 Home or  End	Jump to the first or last tab

Workflow builder

Key	Action
Arrow keys on a focused node	Move it ( Shift moves further)
 Del	Delete the selected node or connection
 Ctrl +  S	Save the workflow

See [The builder](#).

Form controls

-  Tab → /  Shift +  Tab → move between controls
-  Space toggles a switch or checkbox
- Arrow keys change a slider or the selected option in a group

Blocked keys

The following are blocked in production to prevent accidental page reloads:

Key	Context
F5	Launcher and Settings
^ Ctrl + R	Launcher and Settings
^ Ctrl + F	Launcher and Settings
^ Ctrl + P	Settings (print); in the launcher ^ Ctrl + P means "previous result"

Platform differences

Function	Windows/Linux	macOS
Ctrl prefix	^ Ctrl	⌘ Cmd
Alt prefix	⌘ Alt	⌥ Option
Tray access	Click the icon; no key	Click the menu-bar icon; no key

Custom shortcuts

Add global hotkeys in your config file or Settings → Hotkeys:

```
[[hotkey]]
key = "Ctrl+Alt+T"
query = "> " # open Sevak with this text

[[hotkey]]
key = "Ctrl+Alt+L"
run = "system:lock" # run this without showing the launcher
```

See [Custom hotkeys](#) for details.

Settings window

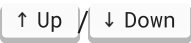
Configure Sevak through the Settings window.

Open Settings

- Click the tray or menu-bar icon and select **Settings**
- Run `sevak --settings` from the command line

Using Settings

Each tab covers a different aspect of Sevak. Changes are **not saved** until you click **Save**. Click **Close** or **Cancel** to discard them.

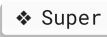
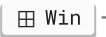
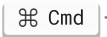
Control	Result
	Save settings
 on a tab button	Move between tabs
 on a tab button	Jump to first or last tab

Validation errors appear under each field and prevent saving until they are fixed.

General

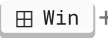
Shortcut

The global hotkey to show and hide Sevak. Tap it to toggle the launcher from anywhere.

The default is  (shown as  on Windows and  on macOS).

Examples: , , .

Three ways to set it:

- **Record** the keys. On Windows the recorder also sees , which Windows normally keeps from apps.
- Pick one from the **Presets** list: Super+Space, Alt+Space, Ctrl+Space or Ctrl+Alt+Space.

- **Type it:**  Win,  Win,  Meta,  Cmd and  Super all mean the Windows/Command key, and Sevak saves it as Super.

Under the field Sevak says how the key is delivered: registered the normal way, taken over with the Windows keyboard hook, Spotlight's shortcut turned off with your permission, GNOME's input-source shortcut moved, or Option+Space used because you kept Spotlight's. When Sevak changed a system shortcut with your permission, a **Restore** button puts it back; when the system shortcut is in the way, **Let Sevak use ...** asks. See [Win+Space](#), [Cmd+Space](#) and [Super+Space](#).

Another app's or your desktop's shortcut can stand in the way of a key; the status under the field says so.

On **Linux Wayland**, the desktop manages global hotkeys. If the field won't register, click **Set up GNOME shortcut** to create a desktop binding, or run `sevak --setup-hotkey "Your+Combo"`.

Links to: `[general] hotkey` in configuration.

Universal Actions shortcut

Global hotkey to capture and act on what you have selected in another app: text, a link, or files.

Leave **empty** to turn off Universal Actions.

On **Wayland**, use the same **Set up GNOME shortcut** button.

Links to: `[general] actions_hotkey` in configuration.

Use the clipboard if the selection can't be read

Wayland, terminal windows and some apps prevent Sevak from reading the selection. Turn this on to act on the clipboard instead when the direct method fails.

Links to: `[actions] use_clipboard_fallback` in configuration.

Hide when focus is lost

Automatically close the launcher when you click outside it or switch to another window.

Links to: `[general] hide_on_blur` in configuration.

Launch at login

Start Sevak in the background when you sign in.

Links to: `[general] launch_at_login` in configuration.

Check for updates

Look for a new version at startup and once per day. If an update is available, you are asked before it is installed.

Links to: `[general] check_for_updates` in configuration.

Hotkeys

Define custom global hotkeys that open Sevak with text already typed, or run a result without showing the launcher.

Adding a hotkey

Click **Add hotkey** and fill in:

- **Key:** the shortcut (e.g., `^ Ctrl` + `⌘ Alt` + `T`)
- **Query or Run:** either type text to pre-fill the search bar, or give a result ID to run immediately

Examples

Hotkey	Action	Result
<code>^ Ctrl</code> + <code>⌘ Alt</code> + <code>T</code>	Query <code>></code>	Opens Sevak with <code>></code> typed (ready for a shell command)
<code>^ Ctrl</code> + <code>⌘ Alt</code> + <code>L</code>	Run <code>system:lock</code>	Locks the screen without showing the launcher

Finding result IDs

Hover over any result for a moment to see its ID. Examples: - `apps:firefox.desktop` (Linux app) - `apps:Mozilla Firefox` (Windows app) - `system:lock` , `system:settings:bluetooth` - `files:/Users/me/Documents` - `shell:git status`

Calculator, web search and clipboard history results have no stable ID and cannot be bound.

Wayland: custom hotkeys are registered as GNOME shortcuts. Delete them in GNOME's keyboard settings when you remove an entry.

Links to: `[[hotkey]]` in [configuration](#).

Appearance

Theme

- **System:** follow your operating system's light/dark setting
- **Light:** always light
- **Dark:** always dark

Links to: `[appearance] theme` in configuration.

Window width

The width of the search bar in pixels. Range: 400–1600 px.

Links to: `[window] width` in configuration.

Accent color

Highlight color for buttons, toggles and the caret. Format: `#rgb`, `#rrggbb`, or `rgb(r, g, b)`. Leave empty to use the theme's default.

Examples: `#7c3aed`, `#fa0`, `rgb(124, 58, 237)`.

Links to: `[appearance] accent` in configuration.

Font size

Size of the result titles in pixels. Range: 12–22 px. The rest of the search bar scales with it.

Links to: `[appearance] font_size` in configuration.

Font family

Font for the launcher. Leave empty to use your system's default. Examples: `Fira Sans`, `sans-serif`, `Monospace`.

Links to: `[appearance] font_family` in configuration.

Opacity

Background opacity of the search bar as a percentage. Range: 30–100%. Text stays fully opaque.

Links to: `[appearance] opacity` in configuration.

Corner radius

Rounding of the search bar's corners in pixels. Range: 0–32 px (0 = sharp corners).

Links to: `[appearance] radius` in configuration.

Theme editor and gallery

Theme editor creates and edits theme files with a live preview: eight built-in themes (Sevak Light, Sevak Dark, Nord, Dracula, Solarized Light, Solarized Dark, Gruvbox and High Contrast), color pickers, sliders for sizes, a WCAG contrast check, undo and redo, **Save as...**, **Apply**, **Import...** and **Export...**. **Open themes folder** shows the `themes` folder next to `config.toml`.

Browse online themes loads the community theme gallery. Nothing is requested until you click it, and **Install** saves a theme only if its checksum matches.

Links to: `[appearance] theme_file` in configuration and [Theme files and the editor](#).

Custom stylesheet

A CSS file in your config folder to override theme variables. Example: `theme.css`. Leave empty to use none.

Sevak's CSS variables are listed in [Themes and appearance](#). Your stylesheet is loaded after the built-in theme, so it overrides everything.

Links to: `[appearance] custom_css` in configuration and [Themes guide](#).

Search

Maximum results

How many rows the result list shows. Range: 1–20.

Links to: `[search] max_results` in configuration.

Fallback web search

Offered when a plain query (not a web keyword) produces no results. Choose from your defined web search engines, or **None** to turn it off.

Examples: If set to `g`, a query like `rust traits` that has no app or file results shows a Google search row.

`config.toml` can list several fallbacks; the dropdown shows them. If you pick a different one here, it overwrites the list.

Links to: `[search] fallback_web_search` in configuration.

Remember searches

On an empty search bar, press `↑ Up` and `↓ Down` to step through your last 50 searches, newest first.

Disabling this also deletes the stored search history.

Links to: `[search] query_history` in configuration.

Plugins

Enable or disable built-in result sources:

- **Applications** (Windows Start Menu shortcuts and packages, macOS app bundles, Linux desktop entries)
- **Calculator** (arithmetic, unit conversion, currency)
- **Files** (indexed folders; also see Files tab)
- **Bookmarks** (browser bookmarks; also see Bookmarks section in configuration)
- **Clipboard history** (if enabled; search recent copied text, images and files)
- **Snippets** (saved text templates)
- **Emoji picker** (`:heart` , `emoji heart`)
- **System commands** (lock, sleep, settings, etc.)
- **Automation tasks** (dark mode, volume, quit, kill, eject, keep awake...)
- **Media controls** (play, pause, next, now playing)
- **Shell** (type `> command` to run in a terminal)
- **Selection** (Universal Actions)
- **Contacts** and **1Password** (also need `enabled = true` in `config.toml`)
- **Dictionary** (`define` , `spell`)
- **Script plugins** and **workflows** you installed
- **Web search engines** (see Web search tab)

Contacts, 1Password, the dictionary, automation tasks and media controls have no fields of their own here; edit their sections in `config.toml` ([configuration](#)).

Currency conversion

Optional: download the European Central Bank's daily exchange rates to convert currencies.

When enabled, `100 usd in eur` works in the calculator. Rates are cached locally for 24 hours. Disable to keep Sevak offline.

Links to: `[calculator] currency` in configuration and [Privacy](#).

Expand snippets as you type

Off by default. When on, typing a snippet's keyword in any app replaces it with the snippet's text. While it is on, Sevak watches your keystrokes for the keyword (the last few characters, in memory only). When it is on, two more fields appear:

- **Keyword prefix:** typed before every keyword, such as `;`, so that `sig` expands and a plain `sig` does not.
- **Expand:** as soon as the keyword is typed, or after a space or punctuation mark.

A warning under the switch says what is missing (macOS Input Monitoring permission, Wayland). The other options (`case_sensitive`, `ignore_apps`, `expand_in_terminals`) are in `config.toml`.

Links to: `[snippets]` in [configuration](#) and [Expand snippets as you type](#).

Workflows

Lists every [workflow](#) with a switch to turn it on or off, **Edit**, **Delete**, and **Review...** for one that waits for your permission. **New workflow** starts a blank one in the visual builder; **New from template...** offers three starting points. See [The builder](#).

Workflows are saved as files in the `workflows` folder next to `config.toml`, not in `config.toml`, so they are saved by the builder's own **Save**, not the Settings **Save** button.

Gallery

Lists ready-made workflows and script plugins. Opening the page requests nothing: **Load gallery** downloads the list, **Install** downloads one package and checks its checksum, and the installed folder asks for permission before anything in it runs. See [The gallery](#).

Web search

Define keywords and URL templates for web search.

Adding an engine

1. Click **Add engine**
2. Fill in:
3. **Keyword:** what you type (e.g., `g`, `yt`, `gh`)
4. **Name:** displayed name (e.g., `Google`, `YouTube`)
5. **URL:** search URL with `{query}` placeholder (e.g., `https://www.google.com/search?q={query}`)

Type the keyword and your search terms to use it: `g rust traits` opens Google with those terms.

Example engines

Keyword	Name	URL
g	Google	<code>https://www.google.com/search?q={query}</code>
yt	YouTube	<code>https://www.youtube.com/results?search_query={query}</code>
gh	GitHub	<code>https://github.com/search?q={query}</code>

Default engines

Sevak starts with Google, YouTube and GitHub. You can remove them and add your own.

Links to: `[[web_search]]` in [configuration](#).

Files

Folders to search

Directories and their subdirectories that are indexed and searchable.

Click **Add folder...** to browse and select. Click the ✕ to remove.

The index is capped at 100,000 entries. Keep roots focused on your active projects.

Links to: `[files] directories` in configuration.

Folder depth

How many levels below each folder are indexed. Range: 0 (just the folder itself) to unlimited.

Default: 4.

Links to: `[files] max_depth` in configuration.

Include hidden files

Index dot-files and dot-folders (e.g., `.git`, `.env`).

Default: off.

Links to: `[files] include_hidden` in configuration.

Keyword

Type "keyword filename" to search only files. Leave empty to disable.

Default: `f`.

Links to: `[files] keyword` in configuration and [configuration guide](#).

Show in global results

Also include matching files in plain searches, ranked lower than apps and bookmarks.

Default: on.

Links to: `[files] global` in configuration.

Search the whole disk

Turns `ff` (file names anywhere) and `in` (words inside files) on or off. Both ask your computer's own file index (Windows Search, Spotlight, `locate`, Tracker or Baloo); nothing leaves your computer. When on, two fields set the keywords:

- **Whole-disk keyword:** default `ff`. Empty turns off name search.
- **Contents keyword:** default `in`. Empty turns off content search.

Every keyword must be one word and differ from all other keywords; Settings refuses to save a clash.

Links to: `[files] use_os_index`, `index_keyword` and `content_keyword` in configuration, and [Whole-disk and content search](#).

Linux

Wayland-specific options (appears only on Wayland sessions).

Use XWayland on Wayland

Native Wayland windows cannot position themselves or reliably receive focus, so Sevak draws through XWayland to stay centered and typeable.

Turn this off to use the native Wayland backend (experimental; you may need to position the window manually).

Restart Sevak to apply.

Links to: `[linux] wayland_use_xwayland` in configuration.

Footer buttons

Button	Action
Open config file	Opens <code>config.toml</code> in your default editor
Reveal logs folder	Shows the folder containing Sevak's debug logs
Save	Save and apply all changes (only enabled when valid)
Cancel / Close	Close Settings without saving (or discard changes if you made any)

Validation errors (shown in red) must be fixed before you can save.

Themes and appearance

Sevak follows your system's light or dark mode by default. Everything else is optional and lives in the `[appearance]` table of `config.toml` (the Settings window, Appearance page, edits the same values). **Settings** → **Appearance** → **Theme editor** builds complete themes visually, with eight built-in themes and an optional online gallery.

Settings

```
[appearance]
theme = "system"      # "system", "light" or "dark"
accent = ""           # "#rgb", "#rrggbb", "rgb(r, g, b)" or ""; "" uses the theme's accent
font_size = 15        # 12-22 pixels; the size of result titles
font_family = ""      # "Fira Sans, sans-serif"; "" uses the system font
opacity = 100         # 30-100, percent opacity of the search bar's background
radius = 14           # 0-32, corner radius in pixels
theme_file = ""       # "themes/Nord.toml", a theme file in the config folder; "" uses none
custom_css = ""       # a stylesheet in the config folder; "" loads none
```

Changes apply when you save in **Settings**, or after "Reload index" in the tray menu when you edit the file by hand.

There are four layers, each overriding the one before: the built-in light or dark look, a **theme file** (`theme_file`), the settings above, and your **custom stylesheet**.

Configuration through Settings

All these options are available in **Settings** → **Appearance**. The GUI guides you with hints for each setting.

Validation

Every value is checked when it is applied. A value that is not valid is ignored (that one setting falls back to its default), a warning is written to the log and shown under **Settings** → **Appearance**:

Setting	Valid range	Default
theme	"system", "light" or "dark"	"system"
accent	#rgb, #rrggbb, rgb(r, g, b) or empty	empty (the theme's accent)
font_size	whole number 12-22	15

Setting	Valid range	Default
<code>font_family</code>	names made of letters, digits, spaces and <code>- _ . ,</code> , separated by commas; up to 200 characters	empty (system font)
<code>opacity</code>	whole number 30–100	100
<code>radius</code>	whole number 0–32	14
<code>theme_file</code>	relative path inside the config folder to a valid theme file	empty
<code>custom_css</code>	relative path inside the config folder	empty

Accent color

Setting `accent` derives related CSS variables:

- `--accent-strong` : 18% darker (for hover states)
- `--selected` : accent at 17% opacity (background of selected result)
- `--on-accent` : black or white, whichever reads better on the accent

Opacity and styling

`opacity` fades only the search bar's background; text stays fully opaque. It requires a window that supports transparency, which Sevak uses for rounded corners and shadow anyway.

Sevak does not offer background blur (Acrylic, Mica, vibrancy) because it would blur the transparent margin around the bar, creating a square frame effect.

`font_size` scales the whole search bar (input, subtitles, row height), not only the result titles. It does not affect the Settings window.

`radius` sets corner rounding. `0` = sharp corners; `32` = very rounded.

Theme files and the editor

A theme is a small TOML file in the `themes` folder of your config folder (**Settings** → **Appearance** → **Open themes folder**). **Settings** → **Appearance** → **Theme editor** creates and edits them without touching the file:

- Eight themes ship with Sevak: Sevak Light, Sevak Dark, Nord, Dracula, Solarized Light, Solarized Dark, Gruvbox and High Contrast. Click one to preview it. Changing anything in a built-in makes a copy, because the built-in names stay as shipped.
- The preview is the launcher's own result row with sample rows, and updates as you edit. **Light / Dark** switches between a theme's two variants.
- Each color has a picker, a text field (`#rrggbb` , `#rrggbbaa` , `rgb()` or `rgba()`) and an opacity slider. Sliders set the font size, corner radius, background opacity, row height, search field size, icon size and window width; **Default** takes a size back out of the theme. The font is a free-text list; the **Installed fonts** button fills the suggestions from the system when the window allows it.
- **Contrast** shows the WCAG contrast ratio of the text on the background, of the selected row's text on the selection, of the subtext on the background and of the text on accent buttons. A pair under 4.5:1 (level AA) is flagged. It is a warning, not a block; every built-in theme passes all four.
- **Undo**, **Redo** and **Reset** step through your edits.
- **Save as...** writes `themes/<name>.toml` . **Apply** saves it if needed and sets `theme_file` ; press **Save** in Settings to keep that choice. **Import...** and **Export...** copy a theme file from and to anywhere on disk (an imported file is validated and rewritten in canonical form).

A theme file:

```

name = "Nord"
author = "Sevak"
description = "Arctic dark colors."

[font]
family = "Fira Sans, sans-serif" # optional; empty is the system font
size = 15 # 12-22

[layout]
radius = 14 # 0-32
opacity = 100 # 30-100
row_height = 48 # 32-96, px
search_size = 22 # 14-40, px, the text you type
icon_size = 32 # 16-64, px
window_width = 720 # 400-1600, px

[dark] # and/or [light]
background = "#2e3440"
text = "#eceff4"
subtext = "#bcc7d8"
border = "rgba(236, 239, 244, 0.12)"
accent = "#88c0d0"
accent_strong = "#709dab"
on_accent = "#2e3440"
selection = "rgba(136, 192, 208, 0.2)"
# selection_text = "#eceff4" # optional: the selected row's title; default: text
tile = "rgba(236, 239, 244, 0.08)"
kbd_background = "rgba(236, 239, 244, 0.08)"
kbd_border = "rgba(236, 239, 244, 0.16)"
shadow = "0 8px 28px rgba(0, 0, 0, 0.55), 0 1px 3px rgba(0, 0, 0, 0.4)"
surface = "rgba(236, 239, 244, 0.05)"
input_background = "rgba(236, 239, 244, 0.06)"
input_border = "rgba(236, 239, 244, 0.24)"
switch_off = "rgba(236, 239, 244, 0.28)"
warn = "#ebcb8b"
error = "#e5848d"
ok = "#a3be8c"

```

Key	CSS variable	Used for
background	--bg	Search bar and Settings background
text	--fg	Text
subtext	--muted	Subtitles, hints, placeholder
border	--border	Hairlines

Key	CSS variable	Used for
<code>accent</code> , <code>accent_strong</code> , <code>on_accent</code>	<code>--accent</code> , <code>--accent-strong</code> , <code>--on-accent</code>	Caret, buttons, hover shade, text on accent
<code>selection</code>	<code>--selected</code>	Background of the selected row
<code>selection_text</code>	<code>--selected-fg</code>	Title of the selected row (default: <code>text</code>)
<code>tile</code> , <code>kbd_background</code> , <code>kbd_border</code>	<code>--tile</code> , <code>--kbd-bg</code> , <code>--kbd-border</code>	Icon tiles and key caps
<code>shadow</code>	<code>--shadow</code>	The bar's drop shadow
<code>surface</code> , <code>input_background</code> , <code>input_border</code> , <code>switch_off</code>	<code>--surface</code> , <code>--input-bg</code> , <code>--input-border</code> , <code>--switch-off</code>	Settings window
<code>warn</code> , <code>error</code> , <code>ok</code>	<code>--warn</code> , <code>--error</code> , <code>--ok</code>	Notices

How a theme file is read:

- **Every value is checked.** A value that is not valid (a color that does not parse, a number out of range, a font name with odd characters, a shadow that is not a plain `box-shadow` list, an unknown key) is ignored with a warning shown under **Settings** → **Appearance**, and the rest of the file still applies. Only a file that is not TOML at all is refused. The file must be UTF-8 and at most 64 KiB, and its path follows the same rules as `custom_css`.
- **One palette or two.** A theme with only `[dark]` or only `[light]` uses it in every mode (the `theme` setting does not switch it). With both, the light one is used in light mode and the dark one in dark mode. Colors a palette leaves out keep the Sevak Light or Sevak Dark value of that mode.
- **Your settings still win.** `font_size`, `font_family`, `opacity` and `radius` from the `[appearance]` table override the theme's values when they differ from their default (15, empty, 100, 14), and a non-empty `accent` replaces the theme's accent and the colors derived from it. The theme's `window_width` is used while `[window] width` is the default 720.
- `custom_css` **comes last**, so it can still override anything.

Theme gallery

The gallery is a list of community themes kept in Sevak's repository (`gallery/themes.json` , with the files in `gallery/themes/`). It is opt-in:

- Nothing is requested until you click **Browse online themes** in the theme editor. That one click makes a single `GET` of `https://raw.githubusercontent.com/ninad-k/Sevak/main/gallery/themes.json` (no cookies, no query, no identifying headers beyond a `Sevak/<version> (gallery)` user agent; 20-second timeout; 256 KiB limit), through the same download code as the [workflow gallery](#). The list is only listed, not installed.
- **Install** downloads that theme's file (https only, 64 KiB limit) and saves it to your themes folder only if its SHA-256 matches the `sha256` in the list. A mismatch, a file that is not a valid theme, or a download that is too large is refused and nothing is written. The saved file is the validated theme in canonical form.
- Nothing is sent about you, and nothing is downloaded or updated in the background. Without a network you can still use the built-in themes and **Import...**

To share a theme, put the file in `gallery/themes/` , add an entry to `gallery/themes.json` and open a pull request; see [gallery/README.md](#).

Custom stylesheet

`custom_css` names a file inside the config folder, for example `theme.css` next to `config.toml` (or `themes/dark-purple.css`). It is a plain CSS stylesheet loaded after Sevak's built-in theme and appearance settings, so it overrides all of them.

Rules for custom stylesheets

- The path must be relative and stay inside the config folder: no absolute paths, no `..` , and a symlink pointing outside is refused
- The file must be UTF-8 and at most 64 KiB; otherwise it is ignored with a warning
- It is read again whenever the config is reloaded (tray menu) or saved (Settings)
- It cannot load anything from the network: Sevak's content security policy blocks remote stylesheets, fonts and images. Use fonts installed on your system and `data:` URLs
- If you keep the config folder in a synced folder, your theme travels with it

Example theme

```
/* theme.css: purple dark theme */
:root {
  --bg: #1b1030;
  --fg: #f1e9ff;
  --muted: #a99bc8;
  --accent: #a78bfa;
  --accent-strong: #8b5cf6;
  --on-accent: #1b1030;
  --selected: rgba(167, 139, 250, 0.2);
}
```

Scoping to light or dark mode

The built-in light and dark values are applied with the same specificity as a plain `:root` rule, and your file comes later, so `:root { ... }` overrides everything.

To change a variable only in one mode:

```
:root[data-theme="dark"] { --bg: #0a0a0a; } /* when theme = "dark" */

:root[data-theme="light"] { --bg: #fafafa; } /* when theme = "light" */

@media (prefers-color-scheme: dark) { /* when theme = "system" and OS is dark */
  :root:not([data-theme]) { --bg: #0a0a0a; }
}

@media (prefers-color-scheme: light) { /* when theme = "system" and OS is light */
  :root:not([data-theme]) { --bg: #fafafa; }
}
```

CSS variables

Set by the light and dark themes (`ui/src/app.css`):

Appearance

Variable	Used for
<code>--bg</code>	Search bar and Settings window background
<code>--fg</code>	Main text color
<code>--muted</code>	Secondary text: subtitles, hints, placeholder
<code>--border</code>	Hairlines between sections and form fields
<code>--accent</code>	Caret, buttons, switches, slider thumbs, focus rings
<code>--accent-strong</code>	Hover and focus states of accented elements
<code>--on-accent</code>	Text and icons on an accent background
<code>--selected</code>	Background of the selected result row

Variable	Used for
<code>--tile</code>	Background behind result icons
<code>--shadow</code>	The search bar's drop shadow (a <code>box-shadow</code> value)

UI elements

Variable	Used for
<code>--kbd-bg</code> , <code>--kbd-border</code>	Key-cap hints (<code>^ Ctrl</code> + <code>K</code> , <code>Enter ↵</code> labels)
<code>--warn</code> , <code>--error</code> , <code>--ok</code>	Notice, error and success message colors
<code>--surface</code>	Panels and sections in the Settings window
<code>--input-bg</code> , <code>--input-border</code>	Text fields and select boxes in Settings
<code>--switch-off</code>	Toggle track when off; slider track
<code>--selected-fg</code>	Title of the selected result; when unset it follows <code>--fg</code>

Computed from settings

These are set automatically from appearance settings (override them in your stylesheet if you prefer):

Variable	Source	Default
<code>--font-family</code>	<code>font_family</code> setting	system font
<code>--font-size</code>	<code>font_size</code> setting	15px
<code>--font-scale</code>	<code>font_size</code> / 15	multiplies other text sizes and row height

Variable	Source	Default
<code>--radius</code>	<code>radius</code> setting	<code>14px</code>
<code>--card-opacity</code>	<code>opacity</code> / 100	<code>1</code>
<code>--row-h</code>	a theme's <code>row_height</code>	<code>max(48px, 48px * scale)</code> : height of a result row
<code>--search-size</code>	a theme's <code>search_size</code>	<code>22px * scale</code> : size of the text you type; the bar's height follows it
<code>--icon-size</code>	a theme's <code>icon_size</code>	<code>32px</code> : size of the result icons

Theme examples

Share your theme through the [theme gallery](#)! Examples could include:

- High-contrast themes for accessibility
- Seasonal themes (holiday colors, etc.)
- Themed to match your wallpaper
- Minimal or dark themes

The class names inside Sevak's UI may change between versions, so override only CSS variables and avoid selectors like `.result` or `.row`. A [theme file](#) is the more durable way to share colors and sizes.

More customization

- [Custom hotkeys and keyboard shortcuts](#)
- [Complete settings reference](#)
- [Configuration guide](#)

See also

- [Privacy: custom stylesheets cannot load from the network](#)
- [Troubleshooting: appearance warnings and validation errors](#)

Features

Features overview

Sevak includes built-in result sources (plugins) for applications, calculations, files, bookmarks and more. Each plugin answers queries, and the search engine routes and ranks results to show the most relevant ones first.

All built-in features

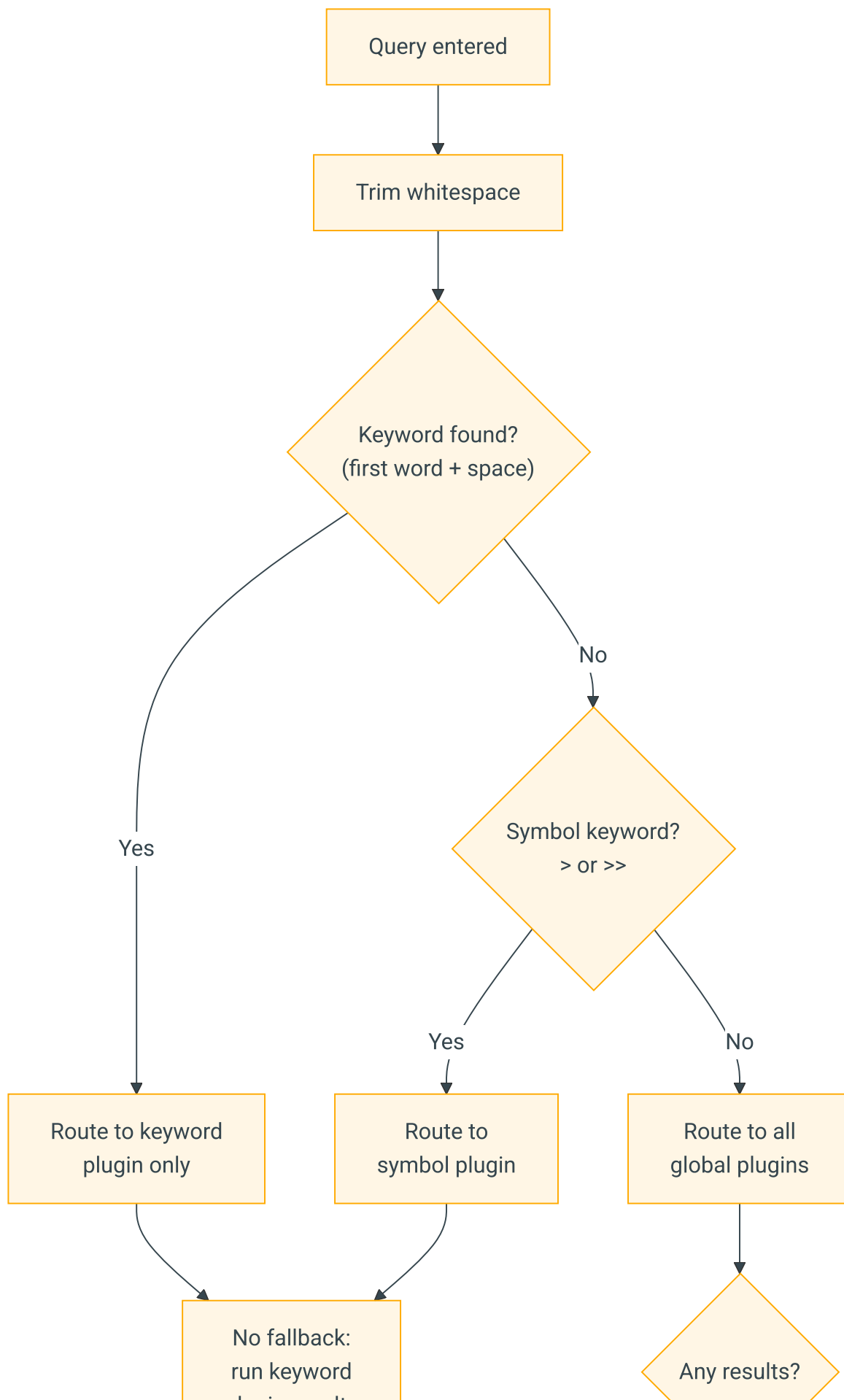
Plugin	Trigger	Keyword	What Enter does	Learn more
Applications	Type any app name	(global)	Launch the application	Applications →
Calculator	Type a math expression	(global)	Copy the result	Calculator →
Web search	<code><keyword></code> <code><search></code>	g, yt, gh	Open search results	Web search →
Files	<code>f <name></code> or path-browse	f	Open the file or folder	Files →
Whole-disk and content search	<code>ff <name></code> , <code>in <words></code>	ff, in	Open the file	Files →
File buffer	<code>Alt</code> + <code>↓ Down</code> on a file	(key)	Act on all collected files	Files →
Bookmarks	<code>b <bookmark name></code>	b	Open the URL	Bookmarks →
System commands	Type a command name	(global)	Run the command	System →
Automation tasks	Type a task name, or <code>t</code>	t (also global)	Run the task	Tasks →

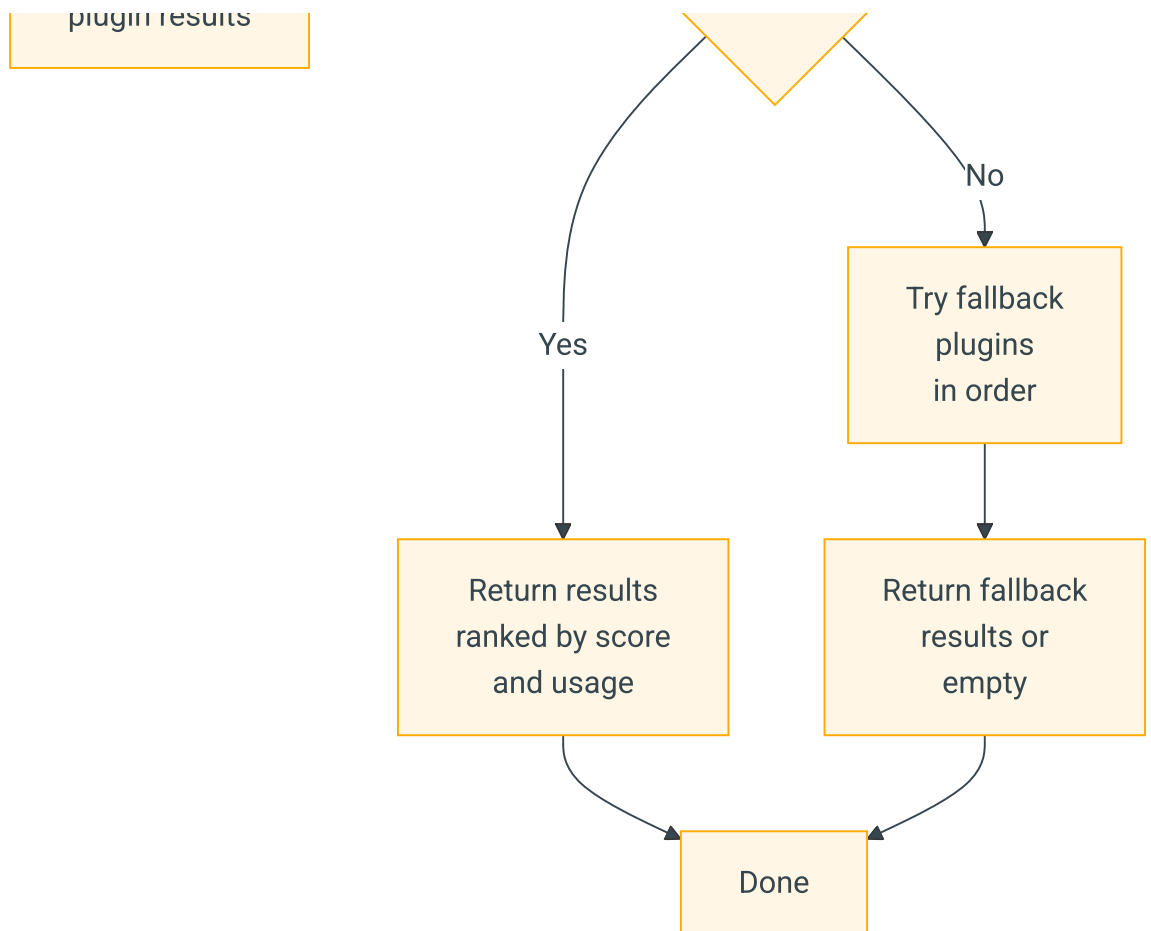
Plugin	Trigger	Keyword	What Enter does	Learn more
Media controls	<code>pause</code> , <code>next</code> , <code>play</code>	play (also global)	Press the media button	Media →
Terminal commands	<code>> <command></code>	<code>></code>	Run in a terminal	Shell →
Clipboard history	<code>cb <text></code>	cb	Paste the text, image or files	Clipboard →
Snippets	<code>s <snippet></code> , or the keyword in any app	s	Paste the saved text	Snippets →
Emoji picker	<code>:<name></code> or <code>emoji</code> <code><name></code>	.; emoji	Paste the emoji	Emoji →
Contacts (opt-in)	<code>c <name></code> or <code>@<name></code>	c, @	Copy the e-mail address	Contacts →
1Password (opt- in)	<code>1p <login></code>	1p	Open the login's website	1Password →
Dictionary and spelling	<code>define <word></code> , <code>spell <word></code>	define, spell	Copy the definition / paste the spelling	Dictionary →
Universal Actions	Press hotkey on selection	(hotkey)	Open action menu	Selection →
Script plugins	Your plugin's keyword	(yours)	What the script says	Script plugins →
Workflows	Your workflow's keyword, hotkey or trigger	(yours)	Run the workflow	Workflows →

Plugin	Trigger	Keyword	What Enter does	Learn more
UUID generator	<code>uuid</code>	uuid	Copy a generated UUID	—

How query routing works

The search engine routes each query to the right plugins using these rules:





Keyword routing

Type `<keyword><space><rest>` to query only plugins with that keyword. For example: - `f documents/report` searches only in Files - `g rust traits` searches only Google - `b github` searches only Bookmarks

The space is optional after symbol keywords: both `> ls` and `>ls` route to Terminal commands.

Global queries

Without a keyword, global plugins answer every query: - **Applications** (global, no keyword) - **Calculator** (global, no keyword) - **System commands** (global, no keyword) - **Automation tasks** and **Media controls** (global if enabled; `global = true` in `[tasks]` and `[media]`) - **Files** (global if enabled; `global = true` in config) - **Bookmarks** (global if enabled; `global = true` in config)

Plugins with a keyword but also answering globally (e.g., Files with `f` keyword) have their results multiplied by **0.5** to rank below their dedicated keyword searches.

Ranking

Results are scored by: 1. The plugin's own scoring (e.g., how closely a filename matches the query) 2. Your usage history (frequently picked results rank higher) 3. Query time (results from slower plugins lag behind faster ones)

Exact matches and keyword-triggered results keep the order their plugin gave them. They do not get boosted by usage history because they are already highly relevant.

Fallback

When no global plugin found any results, the search engine tries fallback plugins in order. This catches cases like: - Empty search → show suggestions from configured web engines (default: Google) - No file or app matches → try web search - New plugin answer → still show something useful

Fallback plugins run after all global plugins have answered, and results are shown in the fallback plugins' configured order (not re-ranked).

Keyword hints

Type just the keyword (`g` , `f` , `b`) and the search engine appends that plugin's keyword row, so Tab can complete to `g` , `f` , `b` ready for input.

Turning plugins on and off

In Settings

Open **Settings** → **Plugins**. Toggle any plugin or instance to enable or disable it.

In config

Add plugin ids or instance ids to `[plugins] disabled`:

```
[plugins]
disabled = ["clipboard", "web:yt"] # turn off clipboard history and YouTube
```

Family ids turn off all instances: `disabled = ["web"]` turns off every web engine. Instance ids turn off one instance: `disabled = ["web:g"]` turns off Google only.

The built-in ids are `apps`, `calculator`, `web` (instances `web:<keyword>`), `files` (its instances `files:names` and `files:content` are the `ff` and `in` searches), `bookmarks`, `system`, `tasks`, `media`, `shell`, `clipboard`, `snippets`, `emoji` (`emoji:word` and `emoji:colon` are its two keywords), `selection` (Universal Actions), `contacts`, `1password`, `dict` and `uuid`. Script plugins use `script:<name>` (or `script` for all), and workflows `workflow:<folder>` (or `workflow` for all). The full list is in the [configuration reference](#).

To find a plugin's id, hover over a result for a moment—the id appears at the bottom.

Customizing keywords and defaults

Read individual feature pages for options: - [Web search keywords and engines](#) - [Files keyword and folders](#) - [Bookmarks keyword and browsers](#) - [Calculator functions and units](#) - [Automation tasks](#) and [media controls](#) - [Contacts](#), [1Password](#) and [the dictionary](#)

All options are in `[section]` of `config.toml`. After editing, choose **Reload index** from the tray menu or restart Sevak.

Applications

Launch installed applications by typing their name. Sevak finds applications from your system (Start Menu and packaged apps on Windows, application bundles on macOS, desktop entries on Linux) and ranks matches by how closely they fit your query and how often you use them.

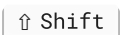
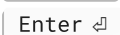
How to use it

Type any part of an application's name. Fuzzy matching lets you skip letters:

Input	What you see	What  does
chrome	Google Chrome	Launch Chrome
msg	Microsoft Outlook (messages)	Launch Outlook
calc	Calculator	Open Calculator
vs co	Visual Studio Code	Launch VS Code
fire	Firefox	Launch Firefox

Matching is case-insensitive. The launcher shows the best matches in order; use the arrow keys to choose a different result.

Actions

Key(s)	Action
	Launch the application
 + 	Show the application file in your file manager
 + 	Copy the application's full path to your clipboard

Key(s)	Action
 Alt + 	Run as administrator (Windows and Linux only; packaged apps cannot be elevated)

Use  ^ Ctrl +  K to open the actions panel and see all available actions.

Tips and troubleshooting

The app doesn't appear: Applications are indexed when Sevak starts. Install the app, then restart Sevak or choose **Reload index** from the tray menu.

Multiple versions of the same app: If you have several versions installed (e.g., both 32-bit and 64-bit), they all appear as separate results.

Packaged apps: On Windows, packaged apps from the Microsoft Store may have long, generated names (like 9P...) in the launcher even if they display nicely in the Start Menu. Pin your favorite packaged apps in the Start Menu to see them with their display names.

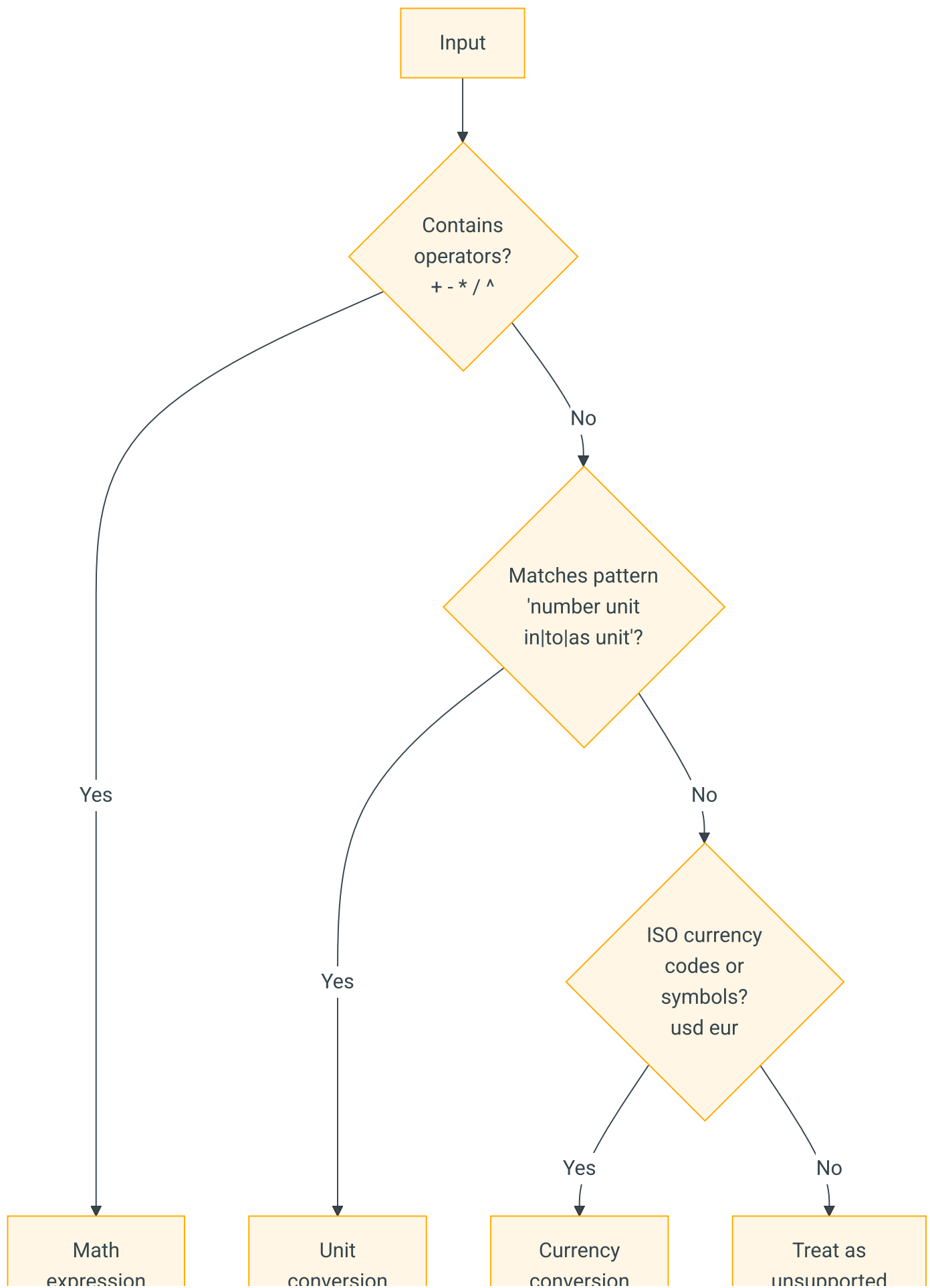
Custom keyboard shortcut: You can bind a key to launch a specific app using [hotkeys in config.toml](#). For example, `run = "apps:firefox.desktop"` on Linux or `run = "apps:Google Chrome"` on Windows.

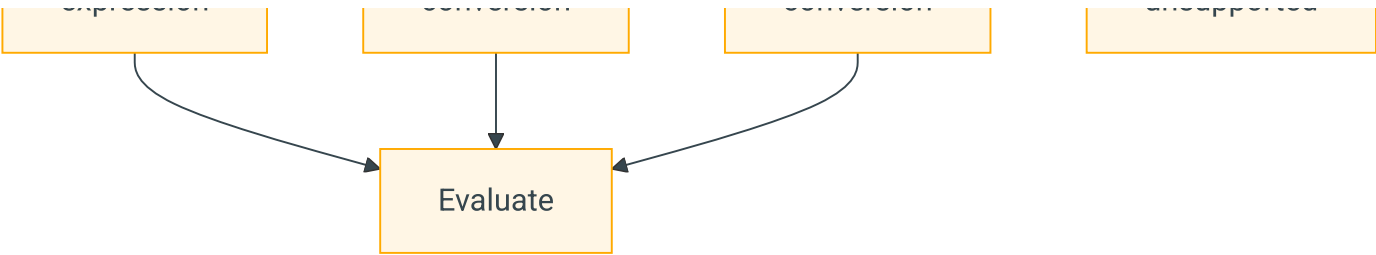
Calculator and conversions

Evaluate math expressions, convert units, and convert currencies without leaving the launcher. No keyword is required; just type.  copies the result.

How to use it

Type a math expression directly into Sevak. The calculator recognizes what you're asking and shows the result:





Math expressions

Type a math expression and `Enter ↵` copies the result. Results use up to 12 significant digits.

Expression	Result	Operation
<code>12*7</code>	<code>84</code>	Multiplication
<code>(125+75)/4</code>	<code>50</code>	Grouped arithmetic
<code>2^10</code>	<code>1024</code>	Powers (<code>^</code> or <code>**</code>)
<code>sqrt(16)</code>	<code>4</code>	Square root
<code>5!</code>	<code>120</code>	Factorial
<code>17%5</code>	<code>2</code>	Remainder (not percentage)
<code>sin(0)</code>	<code>0</code>	Trigonometry

Operators: `+` `-` `*` `/` `%` (remainder) `^` (power, right-associative) `!` (factorial) - `x` and `÷` work as aliases for `*` and `/` - `**` is an alias for `^` - Unary minus: `-2^2 = -4` (minus binds loose; negate then power)

Functions: - Roots: `sqrt(x)` `cbrt(x)` (cube root) - Trig: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh` - Logarithms: `ln` (natural log) `log` (base 10) `log2` `exp` (`e^x`) - Rounding: `floor` `ceil` `round` - Utility: `abs` `sign` `min(a,b)` `max(a,b)` `pow(a,b)` `hypot(a,b)` `atan2(y,x)`

Constants: `pi` (π) `e` `tau` (2π)

Unit conversion

Type `<amount> <unit> (in|to|as|=|→) <target_unit>`.

Input	Result
10 km in mi	6.213711922 mi
72°F to C	22.22222222°C
5'11" to cm	180.34 cm
2 h 30 min in min	150 min
1 acre in m²	4047 m²
60 mph in km/h	96.56064 km/h
5 GB in MiB	4768.371582 MiB

Units work offline. Amounts can be expressions: `(2+3) km in m = 5000 m`.

Supported categories: - **Length:** m, km, cm, mm, mi, yd, ft, in, nmi (nautical mile), ly (light-year), au (astronomical unit) - **Mass:** kg, g, lb, oz (mass ounce), t (metric ton), ton (short ton) - **Temperature:** °C, °F, K (no expression allowed; just `32 f to c`) - **Volume:** L, ml, gal (US gallon), cup, tbsp (tablespoon), tsp (teaspoon), fl oz (fluid ounce), pt (US pint), qt (quart) - Imperial: `imp gal`, `imp pt` for British units - **Area:** m², km², cm², ft², mi², acres, hectares - **Speed:** m/s, km/h, mph (miles per hour), knots, ft/min, any length/time combination (e.g., `mi/h`) - **Data:** B (byte), KB, MB, GB, TB, PB (powers of 1000); KiB, MiB, GiB, TiB, PiB (powers of 1024) - Case matters: `MB` = megabyte, `Mb` = megabit; `kb`, `mb`, `gb` = bytes - **Time:** s, min, h, d, mo (month = 1/12 year), yr (year = 365.25 days) - **Pressure:** Pa, kPa, MPa, bar, atm, psi, torr - **Energy:** J, kJ, MJ, cal, Cal (food Calorie = kcal), kWh - **Angle:** ° `deg` `rad` (radian) `turn` (full rotation)

Compound units: `10 km/h`, `5 sq ft`, `cm²`, `cubic m`, `m/s`, `miles per hour` all work.

Ambiguous cases: - `oz` = mass ounce; `fl oz` = fluid ounce - `m` = meters; `min` = minutes - ° or `deg` alone = angle; °C or `deg C` = temperature - `cal` = small calorie (4.184 J); `Cal` = food Calorie (kcal) - `t` = metric tonne; `ton` = US short ton

Results keep up to 10 significant digits.

Currency conversion

Off by default. Turn it on in **Settings** → **Plugins** → **Currency conversion**, or set `[calculator] currency = true` and reload.

Type `<amount> <currency> in <currency>`. Use ISO codes (`usd`, `eur`) or symbols (`$`, `€`):

Input	Result
100 usd in eur	92.15 EUR (example rate)
50 € to \$	54.50 USD (example rate)
\$100 in eur	92.15 EUR (example rate)

Rates come from the European Central Bank daily, updated at most once a day in the background (never while you type). Until the first download, the result says "Fetching exchange rates...". Amounts can be expressions.

Supported currencies: 30+ major fiat currencies with EUR as the base. The subtitle shows the exchange rate and the ECB's publication date. Cryptocurrencies are not supported.

Options

Setting	Default	What it does	Config section
Currency	false	Enable currency conversion (needs network)	[calculator] currency

Platform notes

Windows

The calculator works identically on all Windows versions. Keyboard layouts that use  as a dead key (European layouts) still parse  as the power operator inside Sevak.

macOS

The calculator works identically on all macOS versions. Type `option+0` for the degree symbol in unit conversions (e.g., 32°F to C).

Linux

The calculator works identically on all Linux desktop environments. Compose key sequences can type special characters outside Sevak; inside the launcher use ASCII equivalents like `deg` for .

Tips and troubleshooting

Percentage of a number: The `%` operator is **remainder**, not percentage. To find 15% of 200, type `200*15/100 = 30`.

Scientific notation: Write `1e3` for 1000, `2e-4` for 0.0002. Leading zeroes and underscores: `1_000_000 = 1000000`.

Rounding noise: `sin(pi)` shows something like `1.2e-16` instead of exactly `0` because of floating-point precision. Real results like `sin(pi/2) = 1` are exact.

Precision: Results are rounded to 12 significant digits to fit the launcher and remove noise from floating-point math. For most everyday use, this is more than enough.

Web search

Search the web with keyword triggers. Sevak comes with Google, YouTube and GitHub pre-configured. `Enter ↵` opens the search results in your browser.

How to use it

Type a keyword followed by a space and your search terms:

Input	Keyword	Search provider	What <code>Enter ↵</code> does
<code>g rust traits</code>	<code>g</code>	Google	Open <code>google.com/search?q=rust+traits</code>
<code>yt keyboard shortcuts</code>	<code>yt</code>	YouTube	Open <code>youtube.com</code> results
<code>gh sevak launcher</code>	<code>gh</code>	GitHub	Open <code>github.com/search</code> results

Press `Tab →` on a bare keyword (`g`, `yt`, `gh`) to complete it with a space, ready for search terms.

The default fallback behavior: when nothing else matches, Sevak searches with the configured fallback engine (by default, Google). You can disable this by setting `[search] fallback_web_search = ""`.

Actions

Key(s)	Action
<code>Enter ↵</code>	Open the search results
<code>↑ Shift</code> + <code>Enter ↵</code>	Copy the search URL to your clipboard

Use `^ Ctrl` + `K` to see all available actions.

Adding custom search engines

Edit `config.toml` to add, remove, or modify web search engines:

```
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"
```

To add a new engine: copy one block and change the `keyword`, `name`, and `url`. The `{query}` placeholder gets replaced with your search terms (percent-encoded).

To remove an engine: delete its entire `[[web_search]]` block.

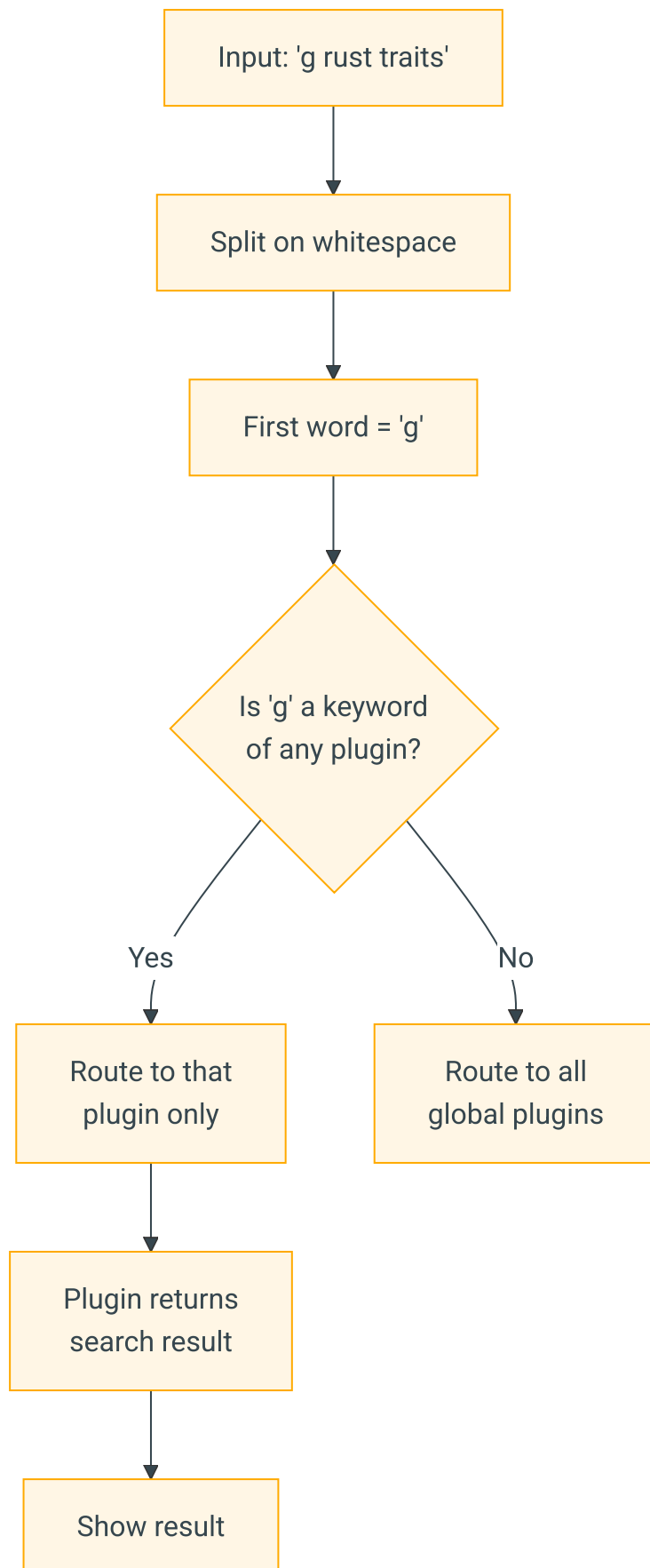
To change the default: define `[[web_search]]` entries in the order they should appear as fallbacks. The first one is used when nothing else matches.

Finding URL templates

Most search engines use `?q=` or `?query=`. Navigate to the search engine's homepage (e.g., [google.com](https://www.google.com)), search for something, and look at the resulting URL. Replace the search term with `{query}`. Examples:

- Google: `https://www.google.com/search?q={query}`
- DuckDuckGo: `https://duckduckgo.com/?q={query}`
- Wikipedia: `https://en.wikipedia.org/w/index.php?search={query}`
- Wiktionary: `https://en.wiktionary.org/w/index.php?search={query}`
- Stack Overflow: `https://stackoverflow.com/search?q={query}`
- npm packages: `https://www.npmjs.com/search?q={query}`
- Wikipedia (full-text): `https://en.wikipedia.org/w/index.php?fulltext=Search&search={query}`

How search routing works



Keyword-triggered web searches are always returned, never down-weighted. When you type `g` with no terms, the engine shows a "Search Google" hint.

Options

Setting	Default	What it does	Config section
Search engines	g, yt, gh	Keywords, names and URL templates for web search	<code>[[web_search]]</code>
Fallback	<code>g</code>	Web engine queried when nothing else matches (empty to disable)	<code>[search]</code> <code>fallback_web_search</code> <code>h</code>

Tips and troubleshooting

Unicode in search terms: Sevak automatically encodes search terms for URLs (spaces become %20, accented characters become UTF-8 bytes, etc.). Type naturally.

Special characters: Symbols like `&`, `#`, `@` are safe to type in search terms and will be encoded correctly.

Multiple fallback options: To offer several fallback engines in order, use a list:

```
[search]
fallback_web_search = ["g", "yt", "gh"]
```



If nothing matches, Sevak tries Google first, then YouTube, then GitHub, in order, returning the first one that gives results.

No search keyword: You can temporarily disable all web search by setting `disabled = ["web"]` in `[plugins]`, but you won't have a fallback for unmatched queries.

Files and folders

Find and open files by name from your configured directories, search the whole disk or inside documents through your computer's own file index, or browse folders by typing a path. `Enter ↵` opens the file or folder. The [file buffer](#) collects several files so you can act on them together.

How to use it

The Files plugin works two ways: **indexed search** (fast, pre-built) and **path browsing** (live, as you type).

Indexed search

Type `f` followed by at least two characters of a filename:

Input	What you see	What <code>Enter ↵</code> does
<code>f report</code>	All files/folders matching "report"	Open the selected file
<code>f config.toml</code>	Exact match for config.toml	Open config.toml
<code>f src/main</code>	Files in src/ matching "main"	Open the file
<code>report</code> (no keyword)	Files matching "report" + apps + other results	Open the file (if Files is global)

Matching is fuzzy: `fsrc` finds `File System (C:)`, `fmain.rs` finds `src/main.rs`.

Path browsing

Type a path to browse its contents live. `Tab →` and `↑ Shift + Tab →` navigate up and down folder levels:

Input	What you see	Action
<code>~/Doc</code>	Contents of ~/Documents (path-matched)	Press <code>Tab →</code> to complete folder name
<code>~/Documents/</code>	Contents of Documents folder	<code>Enter ↵</code> on a folder to open it

Input	What you see	Action
C:\Users\	Contents of C:\Users	Navigate folders with <code>Tab →</code>
/etc/	Contents of /etc (Linux)	Press <code>↑ Shift</code> + <code>Tab →</code> to go up one level

Folders end with `/` (or `\` on Windows), so `Tab →` drills deeper. Path browsing shows contents live without waiting for the full index.

Whole-disk and content search

`f` only knows the folders you chose. Two more keywords ask your computer's own file index, which covers the whole disk and can read inside documents:

Input	What you see	What <code>Enter ↵</code> does
ff report	Files and folders anywhere the index looks whose name matches	Open the selected file
in invoice 2026	Files whose contents contain the words (at least three characters)	Open the selected file

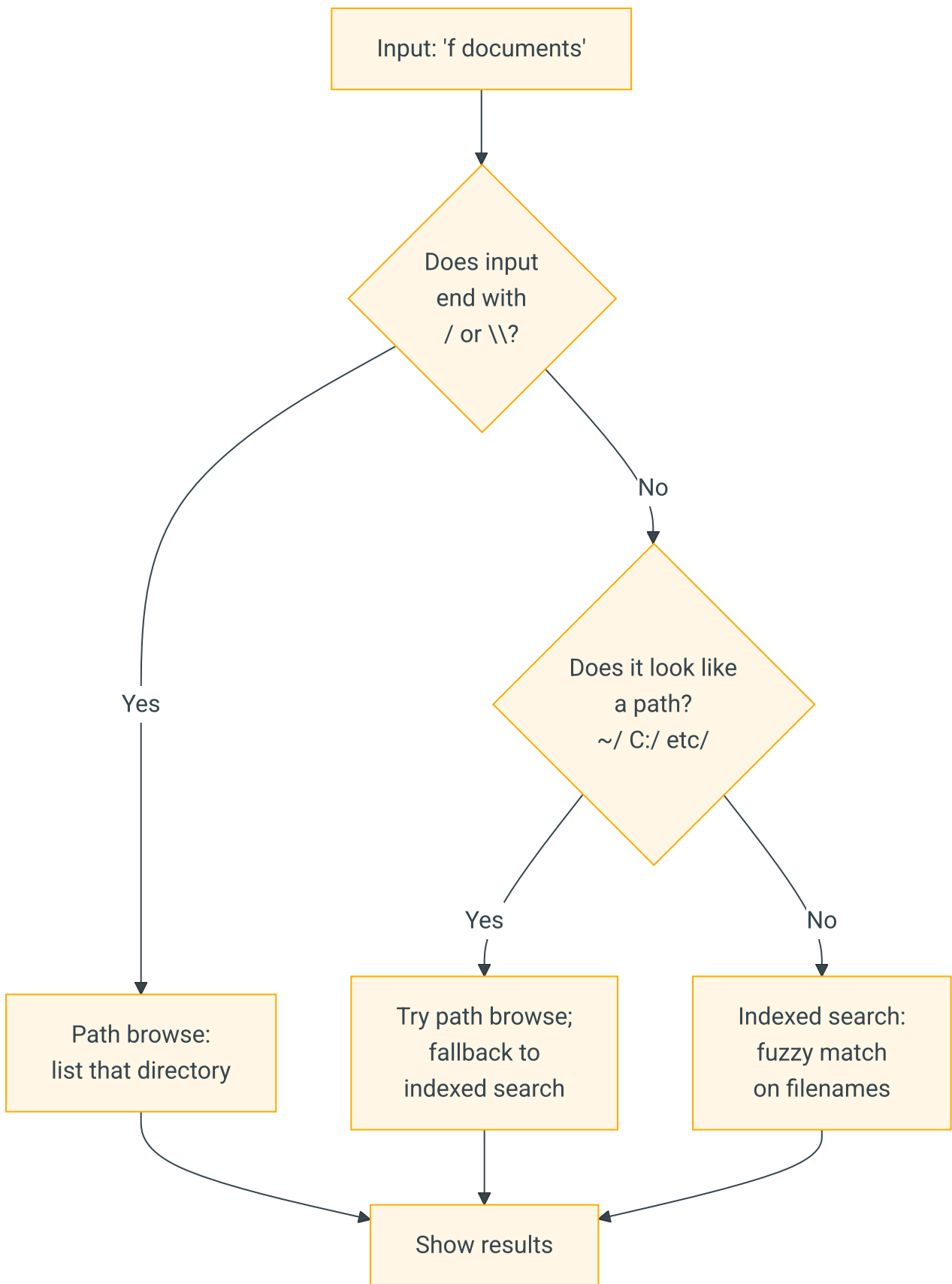
Several words must all match. Results are ordinary file results: `Enter ↵` opens, `^ Ctrl` + `Enter ↵` shows in folder, `↑ Shift` + `Enter ↵` copies the path, `Tab →` fills in the path. Names match from the start of each word (`rep` finds `annual_report.docx`). Hidden files, caches and generated folders are left out, as for `f`.

OS	Names	Inside files	Notes
Windows	Windows Search; "Everything" if <code>es.exe</code> is on <code>PATH</code> and Everything is running	Windows Search	Only indexed places are searched (by default your user folders and the Start menu). Add drives in <i>Indexing Options</i> . Reading PDFs and Office files depends on the installed search filters.

OS	Names	Inside files	Notes
macOS	Spotlight (<code>mdfind</code>)	Spotlight	Honors Spotlight's Privacy list; app bundles and system folders are left out.
Linux	<code>plocate</code> or <code>locate</code>	Tracker 3 (<code>tracker3</code>), else Baloo (<code>baloosearch</code>)	Names need a <code>locate</code> database (<code>updatedb</code> , usually a daily timer). Contents need Tracker or Baloo to be installed and indexing.

Asking the index takes a moment, so it never holds up typing: for `ff` , matches from the `f` folder index appear immediately, and the index's results join the list when they arrive. A search that takes more than two seconds is given up. If there is no index to ask (the Windows Search service is stopped, `locate` is not installed), a "File index unavailable" row says what to do, and `ff` still shows the folder matches. Queries go only to that local index, never over the network.

How indexing vs. path-browsing is chosen



Actions

Key(s)	Action
	Open the file or folder
+	Show the file in your file manager (reveal in folder)
+	Copy the full path to your clipboard
(path browsing)	Complete or drill into a folder
+ (path browsing)	Go up one folder level
+ / +	Add the file to the file buffer and move on
(tap) or +	Preview the file or folder (preview pane)

Use + to see all available actions.

File buffer

Collect several files and folders, then act on all of them. With a file or folder result selected (from file search or a browsed path; not bookmarks or apps):

Key	What it does
+ / +	Add the selected result to the buffer and move the selection up / down (on macOS)
+	Remove the last item
+ / +	Empty the buffer
+	Open the buffer's actions

The buffer is a strip of chips above the results (click the `x` on a chip to drop it). It stays while you search for other things, and is emptied when the launcher hides, unless you set `[file_buffer] keep_between_shows`. The same actions are in the action panel of a file result (**Add to file buffer**, and **File buffer actions** once something is collected). The `⌘ Alt + ← Left`, `⌘ Alt + → Right` and `⌘ Alt + ⌫ Backspace` keys only act while the buffer holds something, so on macOS `⌥ Option + ← Left` and `⌥ Option + ⌫ Backspace` keep their text-editing meaning when it is empty.

Buffer action	What it does
Open all	Opens each item with its default application (asks above 10 items)
Show in folder	Opens a file manager window for each folder the items are in (asks above 3)
Copy paths	Copies the paths, one per line
Copy files to clipboard	Puts the files on the clipboard as a file list, so pasting in Explorer, Finder or a file manager copies them
Move to... / Copy to...	Asks for a folder, then moves or copies the items there
Move to Trash	Sends the items to the Recycle Bin, Trash or freedesktop trash
Compress to .zip	Writes <code>Archive.zip</code> (one item: <code><name>.zip</code>) into the folder the items share
Open in terminal	Opens a terminal in each folder, or in the folder of each file (asks above 3)
More file actions...	The Universal Actions for files, over the collected items

Move to... and **Copy to...** turn the search bar into a folder picker, starting at `~/`. Type a path or browse it as usual: `Tab →` opens the highlighted folder, `↑ Shift + Tab →` goes up, `Enter ↵` uses the highlighted folder, `^ Ctrl + Enter ↵` uses the path exactly as typed, and `⌫ Esc` cancels. Only folders are listed; the path browsing needs `[files] global` or the `f` prefix.

What to expect:

- Moving and trashing ask first, naming the items. Nothing is overwritten: a name that is taken becomes `report (2).docx` (`photos (2)`, `a (2).tar.gz`).
- Move, copy, trash and zip run in the background with a progress line under the chips; you can keep typing. When done, a line says what happened. If some items failed it says how many worked and why the first one did not, and

the failed ones stay in the buffer. Moved and trashed items leave the buffer; copied and zipped ones stay for the next action.

- A folder is never moved or copied into itself. Items inside a collected folder are handled with the folder, not twice. Moving to another drive copies first and removes the original only if the copy worked.
- Zip archives skip symbolic links, and files that cannot be read (the line says how many).
- There is no undo, and no way to cancel a running operation. Trashed items can be restored from the Recycle Bin or Trash.
- On Linux, trashing needs `gio` (part of GLib), and "Copy files to clipboard" works with file managers that read `text/uri-list`; some (Nautilus) only accept their own format and may ignore it.

Options

Setting	Default	What it does	Config section
Directories	<code>~/Desktop</code> , <code>~/Documents</code> , <code>~/Downloads</code>	Folders whose contents are indexed	<code>[files]</code> <code>directories</code>
Max depth	<code>4</code>	How many folder levels below each directory are indexed	<code>[files]</code> <code>max_depth</code>
Include hidden	<code>false</code>	Whether to index dot-files and dot-folders (<code>.gitignore</code> , <code>.config/</code>)	<code>[files]</code> <code>include_hidden</code>
Keyword	<code>f</code>	Keyword to search only files	<code>[files]</code> <code>keyword</code>
Global	<code>true</code>	Also show file results in ordinary searches without the keyword	<code>[files]</code> <code>global</code>
Use the OS index	<code>true</code>	Turn whole-disk (<code>ff</code>) and content (<code>in</code>) search on or off	<code>[files]</code> <code>use_os_index</code>
Whole-disk keyword	<code>ff</code>	Keyword for names anywhere; <code>" "</code> turns that search off	<code>[files]</code> <code>index_keyword</code>

Setting	Default	What it does	Config section
Contents keyword	<code>in</code>	Keyword for words inside files; <code>" "</code> turns that search off	<code>[files]</code> <code>content_keyword</code>
Keep the buffer	<code>false</code>	Keep the file buffer when the launcher hides	<code>[file_buffer]</code> <code>keep_between_show</code> <code>s</code>

How indexing works

At startup and every few minutes, Sevak scans your configured directories and builds an index. Results from indexed search come back instantly.

Pruned directories (always skipped to save time): - `node_modules`, `.git`, `target`, `__pycache__`, `.cache`, `venv`, `.venv`, `appdata`

To search inside one of these, use path browsing or configure them as a separate directory with a small depth.

Limits: The index is capped at 100,000 entries to bound memory. If you exceed this, older or less-used files drop out.

Platform notes

Windows

Paths use backslashes: `C:\Users\Name\Documents\`. Type `~` to expand to your home directory. Hidden files have the hidden attribute set; they appear in path browsing only if you explicitly type `.`.

macOS

Paths use forward slashes: `/Users/Name/Documents/`. The index includes system directories but respects macOS privacy settings (it cannot read files in `~/Library` without permission).

Linux

Paths use forward slashes: `/home/name/documents/`. The `~` shortcut expands to your home. Root directories like `/etc/` and `/usr/` are not automatically indexed; add them to `[files] directories` if you need to search them.

Tips and troubleshooting

Slow indexing: The first index scan takes longer than refreshes. It runs in the background; you can keep using Sevak while it builds.

ff or in says "File index unavailable": Start the Windows Search service (`WSearch`), install `plocate` and run `updatedb` , or install Tracker or Baloo, as the row explains. The OS only finds what it has indexed, so a file created seconds ago may take a moment to appear.

Index doesn't include files I added: Refresh the index by choosing **Reload index** from the tray menu, or wait a few minutes for the automatic refresh.

Hidden files not showing: Path browsing respects `[files] include_hidden` ; if it's false, hidden files appear only when the filter itself starts with a dot (e.g., `.git`).

Path completion is slow: Path browsing reads the filesystem live, so large directories (like `node_modules`) can take a moment. The indexed search is faster for large collections.

Symlinks: Sevak follows symlinks when indexing, so the same file can appear under different paths. If this is unwanted, add the symlinked directory to the pruned list in the code and recompile, or use path browsing to navigate directly.

Browser bookmarks

Search bookmarks from all your browsers and profiles. Everything is read locally from disk; nothing is sent anywhere. `Enter ↵` opens the bookmarked URL in your browser.

How to use it

Type `b` followed by a bookmark name or part of a URL:

Input	What you see	What <code>Enter ↵</code> does
<code>b github</code>	Bookmarks with "github" in the title or URL	Open github.com
<code>b documentation</code>	Pages bookmarked as "Documentation"	Open the URL
<code>b stackoverflow</code>	Stack Overflow bookmarks	Open the URL
<code>documentation</code> (no keyword)	Bookmarks matching "documentation" + apps + files	Open the bookmark (if global is enabled)

Sevak searches bookmark titles and URLs. Bookmarks are deduplicated across browsers and profiles; if the same URL exists in Chrome and Firefox, they merge into one result showing both browser names.

Supported browsers

Chromium family (Chrome, Edge, Brave, Vivaldi, Chromium, Opera, Opera GX on Windows and macOS): - Reads the `Bookmarks` JSON file from each profile - All profiles of each browser are searched automatically

Firefox family (Firefox, LibreWolf, Zen): - Reads `places.sqlite` from each profile - Profiles listed in `profiles.ini` are searched automatically

Actions

Key(s)	Action
Enter ↵	Open the bookmarked URL
⇧ Shift + Enter ↵	Copy the URL to your clipboard

Use `⌘ + K` to see all available actions. If a bookmark is stored in multiple browsers, the browser names appear in the result's subtitle.

Options

Setting	Default	What it does	Config section
Browsers	<code>[]</code> (auto-detect)	Which browsers to search (<code>[]</code> = all found, or <code>["chrome", "firefox"]</code>)	[bookmarks] browsers
Keyword	<code>b</code>	Keyword to search only bookmarks	[bookmarks] keyword
Global	<code>true</code>	Also show bookmarks in ordinary searches without the keyword	[bookmarks] global

Choosing which browsers to search

By default, Sevak searches every installed browser and all profiles. To limit it:

```
[bookmarks]
browsers = ["chrome", "firefox"]    # search Chrome and Firefox only
browsers = []                      # search all browsers (default)
browsers = ["edge"]                # search Edge only
```

Valid browser ids: `chrome`, `edge`, `brave`, `vivaldi`, `chromium`, `opera`, `opera-gx`, `firefox`, `librewolf`, `zen`.

How indexing works

Bookmarks are indexed at startup and every few minutes in the background. Sevak reads each browser's bookmarks file only when it changes (checked by file modification time), so an idle refresh is cheap.

Limits: The index is capped at 50,000 bookmarks to bound memory. If you exceed this, older bookmarks may not appear.

Excluded: Bookmarks whose URL is not `http(s)` (e.g., `javascript:`, `chrome:`, `file:`, `place:`) are skipped.

Platform notes

Windows

Browsers store bookmarks in `%APPDATA%\<browser>\User Data\Default\Bookmarks` (Chrome family) or `%APPDATA%\<browser>\Profiles\<profile>\places.sqlite` (Firefox). Sevak reads from these locations automatically.

macOS

Bookmarks are stored in `~/Library/Application Support/<browser>/`. Sevak respects system privacy settings and may ask for permission to read the Library folder.

Linux

Bookmarks are stored in `~/.config/<browser>/` or `~/.mozilla/firefox/`. Sevak reads from these locations automatically.

Tips and troubleshooting

Bookmarks don't appear: Open **Settings** and check that bookmarks are enabled under **Plugins**. If the browser was just installed, choose **Reload index** from the tray menu.

Browser not detected: Sevak looks for browsers in their default locations. If you installed a browser somewhere unusual, it may not be found. In that case, bookmark a page in your default browser or use path browsing to navigate to the bookmark file manually.

Too many bookmarks: If you have 50,000+ bookmarks (rare!), the oldest or least-used ones may not be indexed. Reduce the number of bookmarks or add more of your favorites to the top level of your bookmark structure so they stay in the index.

Merging profiles: If you use multiple profiles in the same browser, Sevak searches all of them. Bookmarks with the same URL appear as one result showing both profile names in the subtitle.

Bookmarks not updating: The index refreshes every few minutes. If you just added a bookmark, wait a moment or choose **Reload index** from the tray menu to see it immediately.

Clipboard history

Store and search what you've copied recently: text, images and files. Type `cb <text>` to find a clipboard entry and paste it back into the app you were using. Entries are stored newest first, and duplicates are merged (copying the same text, picture or files again moves the existing entry to the top).

This feature is **opt-in**; nothing is watched or stored unless you turn it on.

Turn it on

1. Open **Settings** (tray or `sevak --settings`), go to **Plugins**, and enable **Clipboard history**.
2. Or manually edit `config.toml` and set `[clipboard] enabled = true`, then reload.

The first time you enable it, Sevak starts watching the clipboard. Everything copied *after* that moment is remembered; what was on the clipboard before is not.

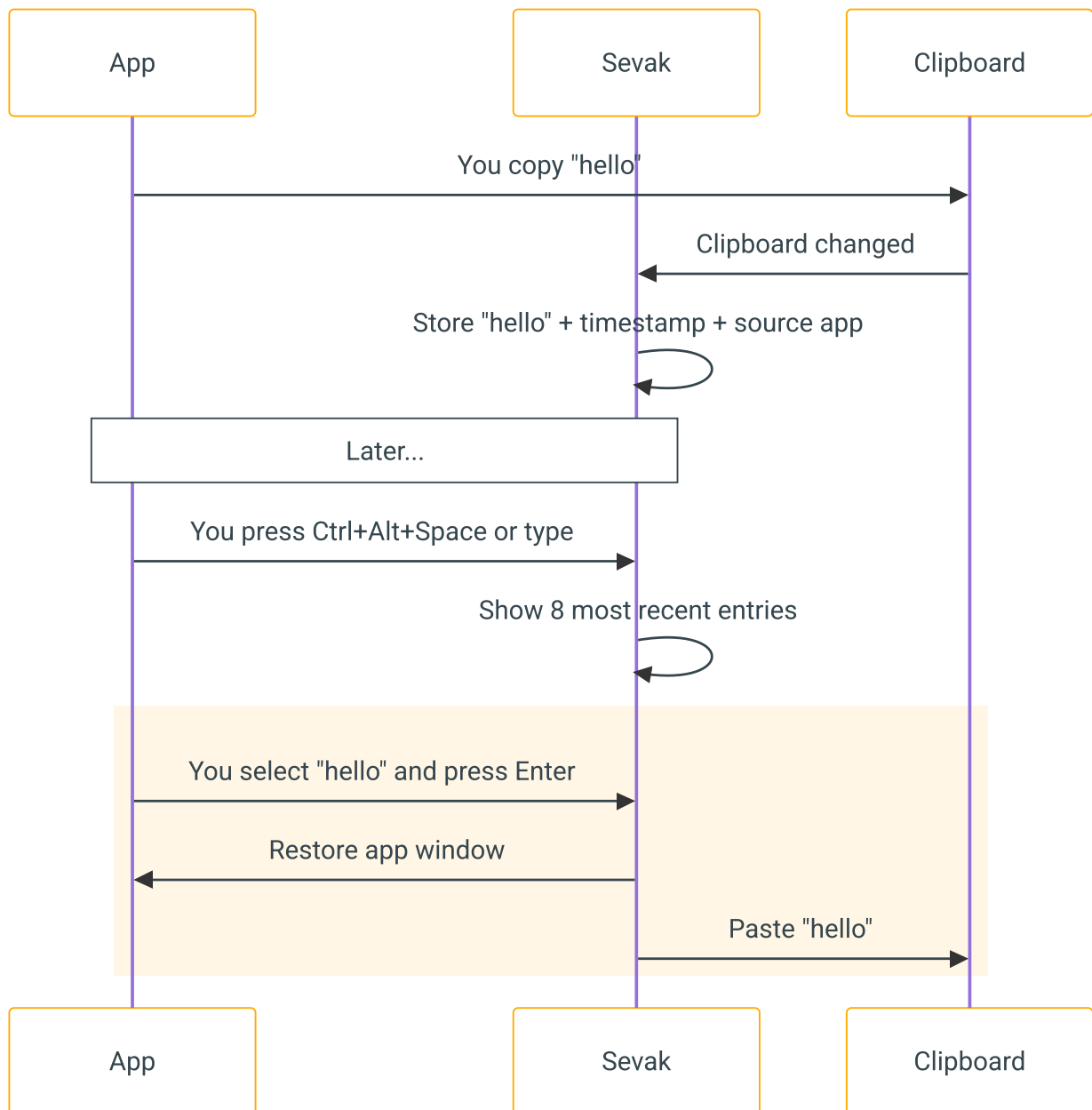
How to use it

Type `cb` followed by at least one character of the text you want to find:

Type	Shown	Press 
<code>cb test</code>	Clipboard entries containing "test" (or copied files named so), newest first	Pastes the selected entry into the app you were in
<code>cb image</code>	Copied images, as a grid of thumbnails when only images match	Pastes the selected image
<code>cb clear</code>	A "Clear clipboard history" row	Permanently deletes all clipboard history

When you press , Sevak hides, brings back the app that was in focus before you opened Sevak, and pastes the text (unless [pasting is unavailable](#)).

Example



Actions

What a row shows and what its keys do depends on what you copied:

You copy	The row shows		Other actions
Text	The first line, with its length in lines	Pastes the text	 +  copies it without pasting;  +  opens a long entry in the Text View

You copy	The row shows	<code>Enter ↵</code>	Other actions
An image (a screenshot, "Copy image" in a browser)	A thumbnail, <code>Image 1920 × 1080</code> and the file size	Pastes the image	<code>^ Ctrl</code> + <code>Enter ↵</code> copies it without pasting; <code>⇧ Shift</code> + <code>Enter ↵</code> Save image as... writes a PNG to the Desktop (else Downloads) as <code>Clipboard image <date> <time>.png</code> , never over an existing file, and shows it in the file manager
Files or folders (a file manager's copy)	The file names and how many	Pastes the files, as a file manager's paste would	<code>^ Ctrl</code> + <code>Enter ↵</code> shows the first one in the file manager; <code>⇧ Shift</code> + <code>Enter ↵</code> copies the files without pasting; <code>^ Ctrl</code> + <code>C</code> copies their paths as text

Files are only recorded by their path: if one has been moved or deleted by the time you paste, the rest are pasted, and if none is left Sevak says so. `^ Ctrl` + `K` lists every action of the selected entry, and the [preview pane](#) (tap `⇧ Shift` or `^ Ctrl` + `Y`) shows the full text, the full image, or the copied file.

Options

Setting	Default	What it does	Config section
Enabled	<code>false</code>	Turn clipboard history on or off	<code>[clipboard]</code> <code>enabled</code>
Max items	<code>200</code>	Keep this many entries; oldest are dropped (1–5000)	<code>[clipboard]</code> <code>max_items</code>
Max item size	<code>64 KB</code>	Longer text is not recorded (1 B–4 MB)	<code>[clipboard]</code> <code>max_item_bytes</code>
Images	<code>true</code>	Also record copied images (as PNG files)	<code>[clipboard]</code> <code>images</code>

Setting	Default	What it does	Config section
Files	<code>true</code>	Also record copied files and folders (their paths)	<code>[clipboard]</code> <code>files</code>
Max image size	<code>10 MB</code>	An image whose PNG is larger is not recorded (1 B–64 MB)	<code>[clipboard]</code> <code>max_image_bytes</code>
Ignore apps	<code>(empty)</code>	Never record copies made in these apps (case-insensitive names), e.g. <code>["KeePassXC", "1Password"]</code>	<code>[clipboard]</code> <code>ignore_apps</code>
Restore clipboard	<code>false</code>	Put back the clipboard's previous text after pasting	<code>[paste]</code> <code>restore_clipboard</code>

What is and isn't recorded

Recorded (up to `max_items` entries of all kinds together): - Plain text copies - Images (unless `images = false`), as PNG files with a small thumbnail - Copied files and folders (unless `files = false`), by their paths only - The timestamp (seconds since the Unix epoch) - The name of the app where it was copied from (where available)

Never recorded: - Content marked secret by the app (password managers on Windows and macOS set a marker; Linux has no marker, so use `ignore_apps`) - Copies made while one of the `ignore_apps` had focus - Text longer than `max_item_bytes` - Images whose PNG is larger than `max_image_bytes` (10 MB by default), and a copy of more than 1000 files - Empty or whitespace-only text - Copies Sevak itself made (pastes, or copies from its own actions), including the copy Universal Actions makes to read your selection - Rich text and HTML (the plain text of such a copy is recorded)

Images together are also kept under about 500 MB: the oldest go first.

The clipboard you started with: - When you enable clipboard history, it does not record what was already on the clipboard—only what you copy *after* enabling it.

Storage and privacy

- **Location:** `clipboard-history.json` in Sevak's data folder holds the text and the paths of copied files; each image is a PNG file (plus a small thumbnail) in the `clipboard` folder next to it (see [Files and data](#) for the path by platform). Everything is stored only while the history is on.

- **Permissions:** On Linux and macOS, the file is readable only by your user (mode `0600`). On Windows, it inherits NTFS permissions from its folder.
- **Clearing:** Type `cb clear` (any start of "clear clipboard history", at least 3 letters) to show a **Clear clipboard history** row, then press `Enter` on it to delete the entire history, image files included. There is no further prompt, and it cannot be undone. Trimming the history (`max_items`) deletes the image files of the entries it drops too.
- **On uninstall:** The `clipboard-history.json` file and the `clipboard` folder stay behind. Delete them manually if you want to remove the history.

Pasting: platform notes

Windows

Pasting works in most applications. **Exception:** applications running with elevated privileges (Admin, UAC elevation) cannot receive simulated keypresses, so Sevak copies the entry instead of pasting and the row says so.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → Sevak to paste. Without this permission, Sevak copies instead and the row shows "Copies to clipboard". Keyboard layouts that move the `v` key (e.g. Dvorak) also prevent pasting; use copy instead.

Linux (X11)

Pasting works in most applications.

Linux (Wayland)

Applications cannot simulate keypresses, so Sevak can only copy the entry. The row shows "Copies to clipboard". If you need pasting on Wayland, set `[actions] use_clipboard_fallback = true`, copy the entry into the clipboard yourself, and use Universal Actions (Ctrl+Alt+Space) to apply transformations.

Tips and troubleshooting



Password managers

If you use KeePassXC, 1Password, Bitwarden or another password manager, add its name to `[clipboard] ignore_apps` to prevent passwords and secret keys from being recorded. Sevak already skips content marked secret by the app (Windows and macOS), but Linux has no such marker.



Restore the clipboard

If you want pasting to restore the clipboard's previous content (so `cb <text>` does not permanently replace what you had copied), set `[paste] restore_clipboard = true`. This is useful when the text you're pasting is temporary and you want to keep your existing clipboard.



Limit the history size

Each entry takes a little disk space. If you have many very long entries (near the `max_item_bytes` limit), the history file can grow. Increase `max_items` to trim entries more aggressively, or lower `max_item_bytes` to skip long text.



History is not encrypted

The `clipboard-history.json` file contains the text you copied, in plain text, and the `clipboard` folder the images you copied as ordinary PNG files. A screenshot of a bank page is as readable there as it was on screen. If your computer is stolen or your home folder is otherwise compromised, the history can be read. Only enable clipboard history if you are comfortable with this risk, or keep sensitive copies out of it by using `ignore_apps` or turning images off (`images = false`).



App names must be exact (or close)

For `ignore_apps`, Sevak matches the application or window name case-insensitively. For example, `KeePassXC` and `keepassxc` both work, but `KeePass` (without `XC`) might not. If ignoring is not working, check the actual name of the app in the clipboard entry's subtitle (e.g. "Copied from KeePassXC").

Snippets

Paste text you define: boilerplate, email signatures, code templates, or anything you type often. Type `s <name>` to find a snippet and paste it with placeholders filled in, or turn on [expansion as you type](#) to replace a snippet's keyword in any app.

How to use it

Define snippets in your `config.toml`:

```
[[snippet]]
name = "Email signature"
keyword = "sig"
text = "Best regards, \nNinad\n{date}"

[[snippet]]
name = "License header"
keyword = "lic"
text = "# Apache License 2.0 - https://example.com"

[[snippet]]
name = "Meeting notes"
text = ""
## Attendees
-

## Agenda
-

## Action items
-
""
```

Then search by name or keyword:

Type	Shown	Press Enter ↵
<code>s email</code>	"Email signature" (matches name)	Pastes the text with <code>{date}</code> replaced by today
<code>s sig</code>	"Email signature" (matches keyword)	Pastes the text
<code>s lic</code>	"License header" (exact keyword match ranked highest)	Pastes the text

Type	Shown	Press <code>Enter ↵</code>
<code>s meet</code>	"Meeting notes" (fuzzy match on name)	Pastes the text (no placeholders to fill)

When you press `Enter ↵`, Sevak hides, brings back the app that was in focus, and pastes the snippet (unless [pasting is unavailable](#)).

Keyword matching

A `keyword` is optional and gives the snippet an alternative search term:

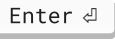
- Type `s signature` → finds "Email signature" by name.
- Type `s sig` → finds "Email signature" by keyword (and ranks higher).
- Type `s email sig` → finds "Email signature" because it matches either term.

An exact match on a keyword outranks any fuzzy match on the name, so `s sig` shows "Email signature" first even if you have other snippets with "sig" in the name.

Placeholders

Placeholders are **filled in when you press** `Enter ↵`, not while you type. This means `{time}` gives the time of the paste, and `{clipboard}` reads the clipboard only then.

Placeholder	Result	Example
<code>{date}</code>	Today in <code>YYYY-MM-DD</code> format	<code>2026-10-03</code>
<code>{time}</code>	Now in <code>HH:MM</code> format (24-hour)	<code>14:05</code>
<code>{datetime}</code>	Today and now together	<code>2026-10-03 14:05</code>
<code>{date:FORMAT}</code>	Today in <code>strftime</code> format	<code>{date:%d %B %Y}</code> → <code>03 October 2026</code>
<code>{time:FORMAT}</code>	Now in <code>strftime</code> format	<code>{time:%H:%M:%S}</code> → <code>14:05:23</code>

Placeholder	Result	Example
<code>{datetime:FORMAT}</code>	Now in strftime format	<code>{datetime:%Y-%m-%d at %H:%M}</code> → 2026-10-03 at 14:05
<code>{clipboard}</code>	The text on the clipboard when you pressed 	<i>(whatever you had copied)</i>
<code>{uuid}</code>	A fresh random UUID v4	550e8400-e29b-41d4-a716- 446655440000
<code>{{</code>	A literal <code>{</code>	<code>{{key}}</code> → <code>{key}</code>
<code>}}</code>	A literal <code>}</code>	<i>(same)</i>

Anything else in braces (`{unknown}` , `{{}` , a lone `{ }`) is left exactly as written, so code snippets with braces usually just work:

```
[[snippet]]
name = "Python lambda"
text = "lambda x: {x * 2}" # The {x * 2} is left as-is
```

Common formats

A few handy strftime formats for custom dates and times:

```
[[snippet]]
name = "ISO week date"
text = "{date:%G-W%V}" # 2026-W40

[[snippet]]
name = "US date"
text = "{date:%m/%d/%Y}" # 10/03/2026

[[snippet]]
name = "File timestamp"
text = "backup_{datetime:%Y%m%d_%H%M%S}" # backup_20261003_140523

[[snippet]]
name = "Markdown heading + date"
text = "## {date:%A, %B %-d, %Y}" # ## Thursday, October 3, 2026
```

Actions

- **Enter** : Paste the selected snippet into the app you were in.
- **^ Ctrl + K** (action panel): See all available actions for the selected snippet (typically just paste and copy).
- **^ Ctrl + C** : Copy the selected snippet's text to the clipboard without pasting into the previous app.
Placeholders are not filled.

Options

Setting	Default	What it does	Config section
Snippets	<i>(empty)</i>	List of <code>[[snippet]]</code> entries with name, optional keyword, and text	<code>[[snippet]]</code>
Restore clipboard	<code>false</code>	Put back the clipboard's previous text after pasting (also affects clipboard history)	<code>[paste]</code> <code>restore_clipboard</code>
Expand as you type	<code>false</code>	Replace a snippet's keyword when you type it in any app; watches keystrokes while on	<code>[snippets]</code> <code>auto_expand</code>
Prefix	<code>" "</code>	Typed before every keyword, e.g. <code>;</code>	<code>[snippets]</code> <code>prefix</code>
Expand on	<code>"immediate"</code>	<code>"immediate"</code> , or <code>"delimiter"</code> (a space or punctuation follows)	<code>[snippets]</code> <code>expand_on</code>
Case-sensitive	<code>true</code>	<code>false</code> : <code>SIG</code> and <code>sig</code> both expand	<code>[snippets]</code> <code>case_sensitive</code>
Ignore apps	<i>(empty)</i>	Never watch or expand in these apps	<code>[snippets]</code> <code>ignore_apps</code>
Expand in terminals	<code>false</code>	Also expand in terminal windows	<code>[snippets]</code> <code>expand_in_terminals</code>

Expand snippets as you type

Give a snippet a `keyword`, turn on **Settings** → **Plugins** → **Expand snippets as you type** (or `[snippets] auto_expand = true`), and typing the keyword in any app replaces it with the snippet's text, placeholders filled in:

```
[snippets]
auto_expand = true
prefix = ";"          # ";sig" expands, a plain "sig" does not

[[snippet]]
name = "Email signature"
keyword = "sig"
text = "Best regards, \nNinad\n{date}"
```

Sevak presses Backspace once for each character you typed, then pastes the text (the clipboard is saved first and put back afterwards, and kept out of clipboard history). Choose the moment with `expand_on: "immediate"` (the default) fires on the last character of the keyword, `"delimiter"` waits for a space or punctuation mark and keeps it after the text, which also lets `;a` and `;ab` both exist.

Rules worth knowing:

- A keyword starting with a letter or digit only counts at the start of a word, so `sig` does not fire inside `assign`. A keyword starting with punctuation (`;sig`, or any `prefix` that is punctuation) fires anywhere. A `prefix` is recommended: short keywords are easy to type by accident.
- The longest matching keyword wins. In `immediate` mode a keyword that is the start of another (`;a` and `;ab`) fires first, so `;ab` is unreachable; use `delimiter` mode or different keywords.
- Matching is case-sensitive unless `case_sensitive = false`. The expanded text is never changed to fit the case you typed.
- Expanding takes a moment (a few hundred milliseconds, mostly waiting for the app to read the pasted text). If you type another character right after the keyword, Sevak leaves the keyword alone rather than delete the wrong text.
- `Enter` and `Tab` do not expand: the app has already sent the line or moved the focus by the time Sevak could act. End the keyword with a space or punctuation in `delimiter` mode instead.
- Typing with `^ Ctrl`, `⌘ Alt` or `⌘ Cmd` held, arrow keys, `↑ Home` / `↓ End`, clicking, switching windows, `Esc` and `Enter` all make Sevak forget what you had typed, so a keyword has to be typed in one go. `Backspace` is followed.
- Input methods (Chinese, Japanese, Korean) and emoji pickers do not type through keys Sevak can follow; keywords containing such characters cannot expand. Letters typed with a dead key (` then `) are followed for the common accents on Windows only; on macOS, anything typed with `Option` held resets.

 This feature watches your keystrokes while it is on

What Sevak keeps and where it does not look:

- Only the last 64 characters you typed, in memory, to compare with your keywords. They are never written to disk, never logged and never sent anywhere, and Sevak has no telemetry. They are overwritten whenever the text could have changed under them (see above) and after every expansion.
- Nothing is observed while the setting is off: expansion does not listen to the keyboard. (On Windows the launcher shortcut can use the same keyboard hook when it needs to, for Win+Space; it only compares key presses with your shortcuts and reads nothing. See [Privacy](#).)
- Typing in Sevak's own windows, in terminals (unless `expand_in_terminals = true`) and in apps listed in `ignore_apps` (`[ "KeePassXC", "1Password" ]`, matched like [clipboard history](#)) is not recorded. Where the system can say so, a focused password box is skipped too: Windows edit controls in password mode, and macOS when "secure input" is on. A password field inside a web page is not detectable on Windows or Linux, so add your browser to `ignore_apps` if you type secrets next to your keywords, or use a `prefix` you never type in a password.
- Security software may flag any program that installs a keyboard hook. Sevak installs it only when you turn this on, or on Windows when a shortcut such as Win+Space needs it.

System	What happens
Windows	A low-level keyboard hook (<code>WH_KEYBOARD_LL</code>). It sees typing into every app at Sevak's own privilege level; it cannot see windows running as administrator unless Sevak does too.
macOS	A listen-only event tap. Needs <i>System Settings</i> → <i>Privacy & Security</i> → <i>Input Monitoring</i> → <i>Sevak</i> (macOS shows a prompt the first time; restart Sevak afterwards) and <i>Accessibility</i> , which pasting needs anyway.
Linux, X11	The X server's RECORD extension. Works without any permission. Dead keys and compose sequences reset instead of being followed.
Linux, Wayland	Not possible: Wayland does not let an app watch typing in other apps or press keys for them. Settings says so.

If the keyword was typed and nothing happened, check [the troubleshooting entry](#).

Pasting: platform notes

Windows

Pasting works in most applications. **Exception:** applications running with elevated privileges (Admin, UAC elevation) cannot receive simulated keypresses, so Sevak copies instead and the row says so.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → Sevak to paste. Without this permission, Sevak copies instead and the row shows "Copies to clipboard". Keyboard layouts that move the `⌘` key (e.g. Dvorak) also prevent pasting; use copy instead.

Linux (X11)

Pasting works in most applications.

Linux (Wayland)

Applications cannot simulate keypresses, so Sevak can only copy. The row shows "Copies to clipboard".

Editing snippets

The **Settings** window does not let you edit snippets (only the expansion switch, prefix and timing under **Plugins**) (to keep hand-written entries untouched, with proper formatting and comments). Edit them by opening `config.toml` directly:

1. Choose **Settings** → **Open config file** (or find it in [Files and data](#)).
2. Edit the `[[snippet]]` sections.
3. Save the file and choose **Reload index** from Sevak's tray or restart.

Each `[[snippet]]` must have a `name` and `text`; the `keyword` is optional. Names do not have to be unique (if you have duplicates, Sevak adds a number like "Email #2" to each).

Tips and troubleshooting



Shared snippets

Store `config.toml` in a synced folder (Dropbox, OneDrive, iCloud Drive, git) with `SEVAK_CONFIG_DIR=~/.Dropbox/sevak`. The `[[snippet]]` entries sync across devices. See [Configuration reference](#).

Long multiline snippets

Use triple-quoted TOML strings to avoid escaping newlines:

```
[[snippet]]
name = "Meeting notes"
text = """
## Attendees
-

## Agenda
-
"""
```



Generate UUIDs

Each time you paste a snippet with `{uuid}`, a fresh UUID is generated. Use it in multiple places in a snippet to get different UUIDs:

```
[[snippet]]
name = "Pair of UUIDs"
text = "{uuid}\n{uuid}"
```



Escaping braces in JSON or code

Snippets containing braces (JSON, programming languages) can conflict with placeholders. Use `{{` and `}}` to escape them:

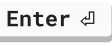
```
[[snippet]]
name = "JSON template"
text = "{{\"user\": \"{clipboard}\"}}"
# Pastes: {"user": "..."}"
```



Emoji picker

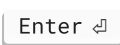
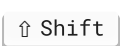
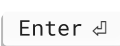
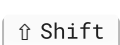
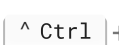
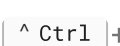
Find an emoji by name and paste it into the app you were using. Type `:` and a name (`:heart` , `:thumbs up`) or `emoji` and a name. The matches are shown as a [grid of tiles](#).

How to use it

Type	Shown	Press 
<code>:heart</code>	Every emoji whose name or keywords match "heart"	Pastes the selected emoji
<code>:thumbs up</code>	 and related emoji	Pastes it
<code>emoji party</code>	  and others, by keyword	Pastes it

The `:` keyword needs no space (`:smile` works). About 1,900 emoji are bundled with Sevak and found by their names and keywords, offline. Skin-tone variants are not listed: the base emoji is pasted and apps apply their own tone setting.

Actions

Key	Action
	Paste the emoji into the app you were using (it is copied where pasting is unavailable)
 + 	Copy the emoji instead of pasting it
 Left  Right  Up  Down	Move between tiles; ++pageup++ / ++pagedown++ jump three rows
 (tap) or  +  Y	Preview pane: the emoji large, with its name, keywords and code points
 +  K	All actions of the selected tile

The grid shows up to 60 tiles. Pasting works like snippets and clipboard history; see [Pasting: platform notes](#).

Options

Setting	Default	What it does	Config section
Keywords	<code>emoji</code> and <code>:</code>	Fixed; they cannot be changed	—
Turn it off	on	Add <code>"emoji"</code> to <code>[plugins] disabled</code> (<code>emoji:word</code> and <code>emoji:colon</code> turn off one keyword)	<code>[plugins]</code> <code>disabled</code>

Tips

 **Find by meaning**

Keywords come from the Unicode CLDR annotations, so a word describing an emoji often finds it even when it is not in the emoji's name.

System commands

Lock your screen, put the computer to sleep, restart, shut down, and more. Type a command's name as you would an app name (two or more characters); only the commands available on your machine are shown.

How to use it

Type the command name (or an alias):

Command	Aliases	What it does
lock	lock, lock computer, lock workstation	Lock the screen or session
sleep	suspend, standby	Put the computer to sleep
hibernate	(no aliases)	Hibernate (if supported)
restart	reboot	Restart the computer
shut down	shutdown, power off, poweroff, turn off	Power off the computer
log out	logout, log off, logoff, sign out, signout	End your session
empty trash / recycle bin	empty recycle bin, clear trash, recycle bin	Delete all trash (or Recycle Bin on Windows)

Press `Enter` to run the command. Destructive commands (restart, shut down, log out, empty trash) ask for confirmation first, unless you turn it off.

Aliases

Each command has aliases so you can type less. A query of at least two characters finds these commands (one letter alone might match app names like "S" for Slack, so Sevak requires more). Examples:

Query	Found
lo	Lock screen, Log out
sl	Sleep
sh	Shut down
re	Restart

System settings

Sevak also offers shortcuts to common operating system settings pages. Type the page name:

Page	Available on	Example queries
Bluetooth	Windows, macOS, Linux	bluetooth
Display	Windows, macOS, Linux	display , screen
Wi-Fi	Windows, macOS, Linux	wifi , network
Sound	Windows, macOS, Linux	sound , audio
Network	Linux	network
Power	Windows, Linux	power , battery

Actions

- **Enter** : Run the command (with confirmation if configured).
- **^ Ctrl + K** (action panel): See all available actions (typically just run).

Options

Setting	Default	What it does	Config section
Confirm	<code>true</code>	Ask before restart, shut down, log out, and empty trash	<code>[system]</code> <code>confirm</code>
Disabled	<i>(empty)</i>	Hide specific commands or settings pages: <code>"restart"</code> , <code>"settings:bluetooth"</code> , etc.	<code>[system]</code> <code>disabled</code>

Platform notes

Windows

- **Lock:** `LockWorkStation` (fast)
- **Sleep:** `SetSuspendState` (hybrid sleep mode)
- **Hibernate:** `shutdown /h` (only if hibernation is enabled)
- **Restart:** `shutdown /r /t 0`
- **Shut down:** `shutdown /s /t 0`
- **Log out:** `shutdown /l`
- **Empty Recycle Bin:** `SHEmptyRecycleBin` (permanent delete)
- **Settings pages:** `ms-settings:` (Bluetooth, Display, Network, Power, Sound, etc.)

macOS

- **Lock:** `pmset displaysleepnow` (lock screen; requires "Require password" set to immediately)
- **Sleep:** `pmset sleepnow`
- **Hibernate:** Not offered (use sleep instead)
- **Restart:** Uses System Events (requires Accessibility permission on first use)
- **Shut down:** Uses System Events (requires Accessibility permission on first use)
- **Log out:** Uses System Events (requires Accessibility permission on first use)
- **Empty Trash:** Finder (requires Accessibility permission on first use)
- **Settings pages:** System Settings panes

Linux

- **Lock:** `loginctl lock-session`
- **Sleep:** `systemctl suspend`
- **Hibernate:** `systemctl hibernate` (only if swap is set up)
- **Restart:** `systemctl reboot`
- **Shut down:** `systemctl poweroff`
- **Log out:** `gnome-session-quit` (GNOME), or KDE `qdbus`, or `loginctl terminate-session`
- **Empty Trash:** `gio trash --empty`
- **Settings pages:** `gnome-control-center <panel>` (if installed; GNOME only)

Tips and troubleshooting



Disable a command

To hide a specific command, add it to `[system] disabled`:

```
[system]
disabled = ["hibernate", "settings:bluetooth"]
```



Disable all confirmations

If you do not want to confirm destructive commands, set `confirm = false`:

```
[system]
confirm = false
```



Destructive commands ask for confirmation

Restart, shut down, log out, and empty trash are destructive and ask for confirmation first (if `confirm = true`). A warning dialog appears whose button names the action (for example **Restart**). Only that button runs it; **Cancel** or closing the dialog does nothing. This also applies when the command is bound to a [custom hotkey](#).



macOS Accessibility permission

Restart, shut down, log out, and empty trash on macOS need *System Settings* → *Privacy & Security* → *Accessibility* → *Sevak*. The first time you use any of these, macOS shows a permission prompt; grant it and try again.

Automation tasks

Ready-made actions for the operating system: toggle dark mode, set the volume, take a screenshot, quit or kill an app, eject a drive, keep the computer awake and more. Type a task's name as you would an app (two or more letters), or type `t` to list them all. Only the tasks that work on your machine are offered, and they are matched by name even without the `t`.

How to use it

Task (try typing)	Windows	macOS	Linux
Toggle dark mode (dark mode, theme)	app and system theme in the registry, then a settings broadcast	System Events appearance	GNOME <code>color-scheme</code> (<code>gsettings</code>), GNOME only
Show desktop (desktop)	<code>Win + D</code>	<code>F11</code> through System Events	<code>wmctrl -k</code> , if installed (X11)
Hide other apps (hide others)	<code>Win + ↑ Home</code> (minimizes all but the front window)	<code>⌘ Option + H</code>	not offered
Minimize all windows	<code>Win + M</code>	not offered	not offered
Take a screenshot (screenshot, snip)	<code>ms-screenclip:</code> (Snipping Tool)	the Screenshot app	<code>gnome-screenshot -i</code> , else <code>spectacle</code> , <code>flameshot gui</code> , <code>xfce4-screenshooter</code>
Open Downloads folder (downloads)	your Downloads folder	your Downloads folder	your Downloads folder
Open recent files (recent)	<code>shell:recent</code>	not offered	<code>gio open recent:///</code>

Task (try typing)	Windows	macOS	Linux
Flush DNS cache (<code>flush dns</code>)	not offered (needs administrator rights)	not offered (needs root)	<code>resolvectl flush-caches</code> ; may ask through polkit
Restart Explorer / Finder	ends <code>explorer.exe</code> (Windows starts it again)	<code>killall Finder</code>	not offered
Empty clipboard (<code>clear clipboard</code>)	yes	yes	yes
Mute / Unmute / Volume up / Volume down	the default output device (Core Audio)	Apple events (<code>set volume</code>)	<code>wpctl</code> , else <code>pactl</code> , else <code>amixer</code>
Toggle Wi-Fi (<code>wifi</code>)	Windows radio API, if the PC has a Wi-Fi radio	<code>networksetup -setairportpower</code>	<code>nmcli radio wifi</code> , else <code>rkill toggle wifi</code>
Toggle Bluetooth (<code>bluetooth on</code>)	Windows radio API, if the PC has a Bluetooth radio	<code>blueutil --power toggle</code> , if <code>blueutil</code> is installed	<code>rkill toggle bluetooth</code> , else <code>bluetoothctl power</code>
Keep awake (<code>caffeinate</code> , <code>stay awake</code>)	a power request held for the time	<code>caffeinate -d -i -t</code>	<code>systemd-inhibit</code>
Stop keeping awake	releases it	stops <code>caffeinate</code>	stops <code>systemd-inhibit</code>

Tasks that take an argument

Some tasks take what you type after them:

Type	Lists or does
<code>quit / quit sla</code>	The apps that have a window; <code>Enter ↵</code> asks the app to quit, <code>⇧ Shift + Enter ↵</code> force quits it
<code>force quit sla (or fq sla)</code>	The same list; <code>Enter ↵</code> force quits after asking
<code>kill chrome</code>	The running processes with that name, grouped, with their CPU and memory; <code>Enter ↵</code> ends them all after asking
<code>eject / eject usb</code>	The removable drives (USB sticks, SD cards, optical drives); <code>Enter ↵</code> ejects or unmounts
<code>vol 30, volume 30%</code>	Sets the volume (0 to 100); <code>vol up</code> , <code>vol down</code> , <code>vol mute</code> , <code>vol unmute</code> also work, and a bare <code>vol</code> offers presets
<code>awake 45, awake 2h, awake 1.5 hours</code>	Keeps the computer awake for that long (one minute to 24 hours); a bare <code>awake</code> offers presets

The list of apps, processes and drives is read from the system in the background, so the first keystroke after a pause may show the list a moment later; typing never waits for it.

Force quit, kill and restarting Explorer or Finder ask for confirmation first, and the system's own processes (`csrss`, `launchd`, `systemd` ...) and Sevak itself are never offered.

Bind a task to a hotkey

Tasks have stable result ids, so a `[[hotkey]]` or `sevak --run` can run them directly:

```
[[hotkey]]
key = "Ctrl+Alt+D"
run = "tasks:dark_mode"

[[hotkey]]
key = "Ctrl+Alt+M"
run = "tasks:volume:30"      # also "tasks:keep_away:45", "tasks:kill:chrome.exe"
```

Options

Setting	Default	What it does	Config section
Confirm	<code>true</code>	Ask before force quit, kill and restarting Explorer or Finder	<code>[tasks]</code> <code>confirm</code>
Disabled	<code>(empty)</code>	Hide tasks: <code>"dark_mode"</code> , <code>"screenshot"</code> , <code>"kill"</code> ... (the full list is in the configuration reference)	<code>[tasks]</code> <code>disabled</code>
Keyword	<code>t</code>	<code>t</code> lists the tasks; <code>" "</code> removes the keyword	<code>[tasks]</code> <code>keyword</code>
Global	<code>true</code>	Also match task names in ordinary searches	<code>[tasks]</code> <code>global</code>

To turn the whole plugin off, add `"tasks"` to `[plugins] disabled`.

Platform notes

Windows

Toggling Wi-Fi or Bluetooth needs *Settings* → *Privacy & security* → *Radios* to allow desktop apps. The window-shortcut tasks wait a moment so they act on the app you were using, not on Sevak. Quitting an app posts the same close request as its close button, so it may ask to save.

macOS

The first use of dark mode, volume, quit or hide-others asks for permission to control System Events; hide-others and show-desktop press keys and need *Privacy & Security* → *Accessibility* as well. Homebrew tools such as `blueutil` are found in `/opt/homebrew/bin` and `/usr/local/bin`.

Linux

Tasks that need a helper (`wmctrl`, `nmcli`, `udisksctl` ...) are simply not listed when it is not installed. Quit lists X11 windows (also XWayland); on a pure Wayland session it lists programs started with a display, minus a few desktop helpers.

Troubleshooting

A task you expected is missing? See [An automation task or media control is missing](#).

Media buttons (play, pause, next) are a separate plugin: [Media controls](#).

Media controls

Press the media buttons of whatever is playing (a music app, a browser tab, a video player) and see the current track, without switching to the player.

How to use it

Type `play`, `pause`, `next` or `skip`, `previous` or `back`, or `stop`. `Enter ↵` presses that button for the player you are listening to.

Type `play` (with the trailing space) or `music` to see the buttons together with a **now playing** row: *title – artist*, and the app.

Key on the now-playing row	Action
<code>Enter ↵</code>	Play or pause
<code>⇧ Shift</code> + <code>Enter ↵</code>	Next track
<code>⌘ Alt</code> + <code>Enter ↵</code>	Previous track

How it reaches the player

	Windows	macOS	Linux
Buttons	The system media session (the one in the volume flyout), else the media keys	The media keys (play/pause, next, previous); stop is sent to Music or Spotify	<code>playerctl</code> (MPRIS)
Now playing	The system media session	Music and Spotify (through Apple events, only if they are running)	<code>playerctl</code> <code>metadata</code>

On Linux the buttons are not offered until `playerctl` is installed. On macOS the system's own now-playing service is private, so only Music and Spotify are asked for the track.

Options

Setting	Default	What it does	Config section
Keyword	<code>play</code>	<code>play</code> lists the buttons and the track; <code>" "</code> removes the keyword	<code>[media]</code> <code>keyword</code>
Global	<code>true</code>	Also match <code>pause</code> , <code>next track</code> ... in ordinary searches	<code>[media]</code> <code>global</code>
Now playing	<code>true</code>	Show the playing track (title, artist, app) as a row	<code>[media]</code> <code>now_playing</code>

To turn the plugin off, add `"media"` to `[plugins]` `disabled`. Volume and mute are [automation tasks](#).

Privacy

The track is read from your media player when you search for it, never stored or sent anywhere. Set `now_playing = false` to turn the row off.

Shell commands

Run shell commands in a terminal window. Type `> <command>` (or `><command>` , no space needed); nothing happens until you press `Enter ↵`. Recent commands are offered again.

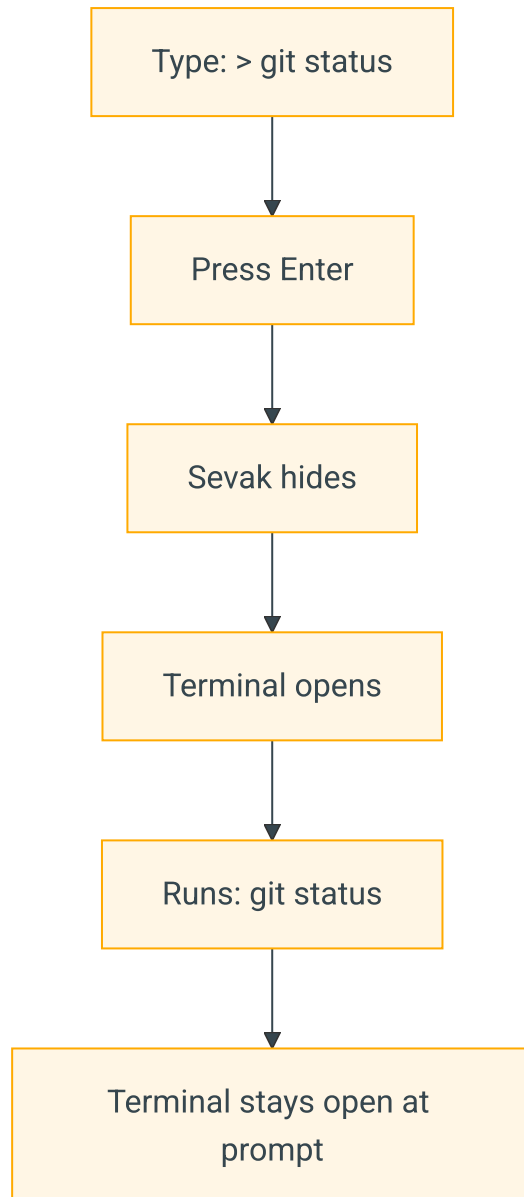
How to use it

Type `>` followed by the command:

Type	Shown	Press <code>Enter ↵</code>
<code>> git status</code>	"Run <code>git status</code> in terminal"	Opens your terminal and runs the command
<code>>npm test</code>	"Run <code>npm test</code> in terminal"	Opens your terminal and runs the command
<code>></code> (just <code>></code>)	"Open terminal" + recent commands	Opens a terminal at a shell prompt (if a recent is selected, runs it)

Nothing is executed until you press `Enter ↵`. Recent commands are stored in `usage.json` and are offered again, newest first and ranked by frequency.

Example



Keep the terminal open

By default, after your command finishes, the terminal stays open at a shell prompt (`[shell] keep_open = true`). If you want it to close when the command exits, set `keep_open = false` .

Choosing a terminal and shell

Sevak auto-detects a suitable terminal and shell for your platform, but you can specify one manually:

```
[shell]
terminal = ""          # "" = auto-detect
shell = ""             # "" = auto-detect
keep_open = true       # stay at a prompt after the command exits
```



Auto-detection order

Windows

Terminal: Windows Terminal (`wt`), else a console window.

Shell: `pwsh` (PowerShell Core), else `powershell`, else `cmd`.

For PowerShell, the command is encoded as Base64 to avoid quoting issues.

macOS

Terminal: Terminal.app (or `iTerm2` if you set `terminal = "iterm"`, `terminal = "iterm2"`, or `terminal = "/Applications/iTerm.app"`).

Shell: Your login shell (environment variable `$SHELL`), run by the terminal. Sevak does not choose the shell on macOS.

Linux

Terminal: `$TERMINAL` environment variable, then `x-terminal-emulator`, `gnome-terminal`, `konsole`, `kitty`, `alacritty`, `wezterm`, `foot`, `xterm`.

Shell: `$SHELL`, else `sh`.

Examples

Choose a specific terminal or emulator:

```
[shell]
# Use Windows Terminal explicitly
terminal = "wt"

# Or an alternative like ConHost (Windows console)
terminal = "conhost"

# macOS: use iTerm2 instead of Terminal.app
terminal = "iterm"

# Linux: use Kitty with custom arguments
terminal = "kitty --class sevak"

# Linux: use a terminal by full path
terminal = "/usr/bin/alacritty"
```

Choose a different shell:

```
[shell]
# Use bash on Windows (with Git Bash)
shell = "bash"

# Use zsh on Linux
shell = "zsh"

# PowerShell on Linux
shell = "pwsh"
```



Actions

- **Enter** : Run the command in a terminal window.
- **^ Ctrl** + **K** (action panel): See all available actions (typically just run).

Options

Setting	Default	What it does	Config section
Terminal	<i>(auto-detect)</i>	Terminal program: <code>wt</code> , <code>iterm</code> , <code>kitty</code> , <code>alacritty</code> , or a full path with optional args	[shell] terminal
Shell	<i>(auto-detect)</i>	Shell that runs the command: <code>pwsh</code> , <code>bash</code> , <code>zsh</code> , <code>fish</code> , etc.	[shell] shell
Keep open	<code>true</code>	Leave the terminal at a shell prompt after the command exits	[shell] keep_open

Platform notes

Windows

Commands run in the terminal without a shell context, so shell aliases and functions (from `.bashrc`, `.zshrc`, etc.) are not available. Use full paths or install executables on `PATH`. PowerShell scripts need `powershell - ExecutionPolicy Bypass -File <path> .`

macOS

Commands run through your login shell, so aliases and functions from your shell profile are available.

Linux

Commands run through `$SHELL` (or `sh`) in non-interactive mode, so aliases from `.bashrc` (and similar) are typically not available. Use `source ~/.bashrc && <command>` if you need aliases or custom functions.

Tips and troubleshooting



Keep recent commands

Recent commands are stored in `usage.json` (in Sevak's data folder). Clearing the usage store deletes them. Choose **Reload index** from the tray to rebuild, or restart Sevak.



Set a variable for the terminal

Some terminal programs recognize environment variables. For example, on Linux you can set `KITTY_LISTEN_ON` to enable remote scripting, or use `alacritty --class sevak` to apply different window decoration.



Elevated commands on Windows

Commands run as the current user. If you need to run a command as administrator, use `runas /user:Administrator <command>` instead (this will prompt for your admin password).



Terminal is missing

If Sevak cannot find a terminal on Linux, install one from the auto-detection list, or set `terminal` to a full path. Check that the terminal is on `PATH`: `which alacritty` or `which gnome-terminal`.



Shell not found

If the shell you specified is not installed, Sevak falls back to `sh` (Linux) or tries the next option in the auto-detect order (Windows). Verify the shell exists: `which bash` or `which pwsh`.

Selection actions

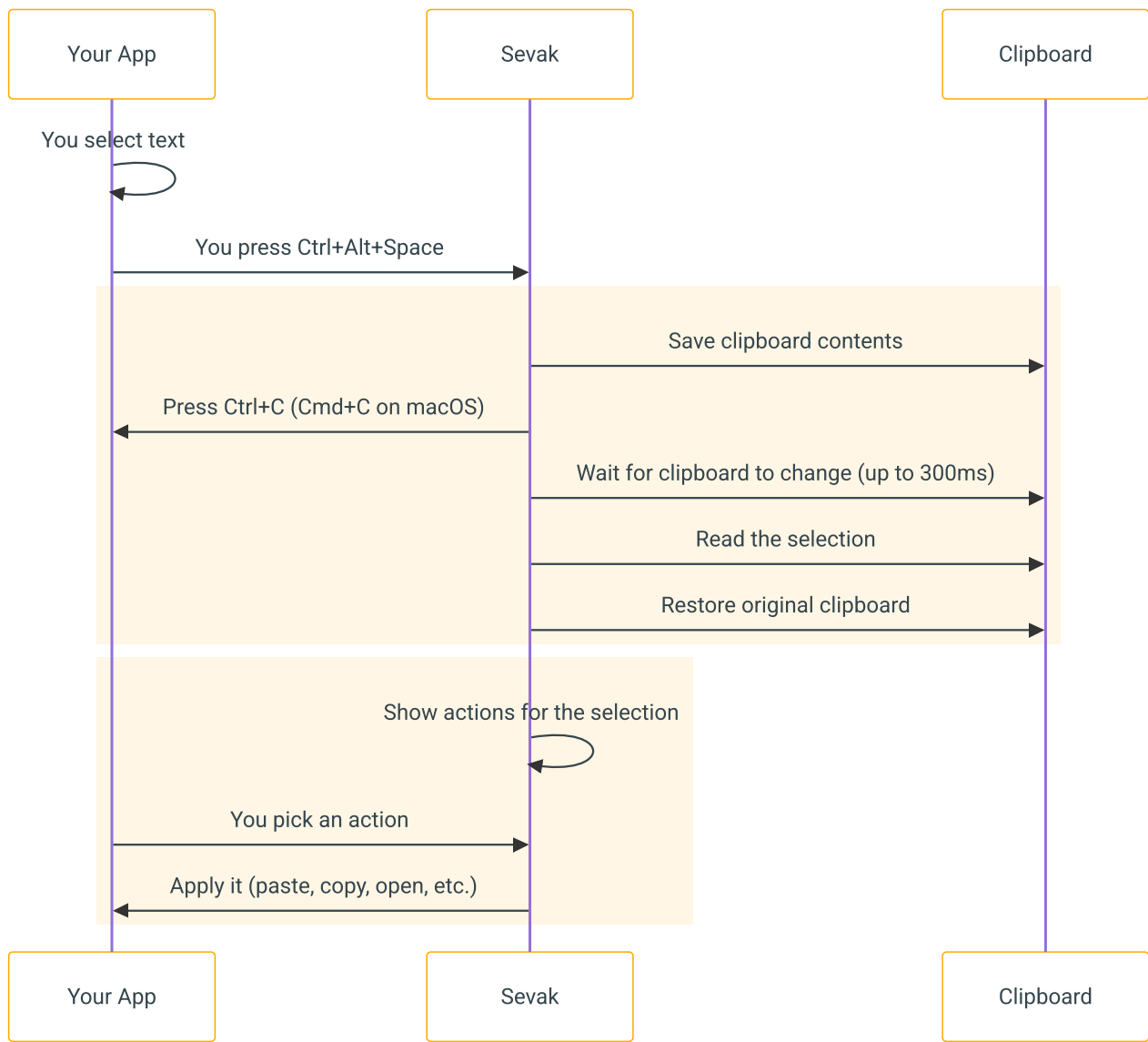
Act on text, links, or files selected in another app without copying and pasting manually. Select something, press `Ctrl+Alt+Space` (configurable), and Sevak shows actions for it. Nothing runs until you pick an action with `↑ Up` / `↓ Down` and `Enter ↵`, or `Ctrl+1` through `Ctrl+9`.

Turn it on

The feature is on by default. To change the hotkey or turn it off:

```
[general]
actions_hotkey = "Ctrl+Alt+Space" # "" turns it off
```

How it works



The selection itself is **never stored or logged**. Sevak reads it once, shows actions, and forgets it unless you run an action.

What you can do with selections

Text

Select any text:

Action	What it does
Search [Engine] for "..."	Opens your browser with a web search (one row per <code>[[web_search]]</code> engine)
Large Type	Shows the text huge across the screen (for reading across the room)
Copy	Copies the text to the clipboard
Paste as plain text	Pastes without formatting (font, color, etc.)
Calculate	If the text is a calculation or conversion, shows the result (e.g., <code>2*3+4</code> , <code>10 km in mi</code>)
Uppercase	Transform: <code>hello</code> → <code>HELLO</code> (replaces the selection where pasting works)
Lowercase	Transform: <code>HELLO</code> → <code>hello</code>
Title Case	Transform: <code>hello world</code> → <code>Hello World</code>
Trim whitespace	Remove leading and trailing spaces
URL-encode	Transform: <code>hello world</code> → <code>hello%20world</code>
URL-decode	Transform: <code>hello%20world</code> → <code>hello world</code>
Base64 encode	Transform: <code>hello</code> → <code>aGVsbG8=</code>
Base64 decode	Transform: <code>aGVsbG8=</code> → <code>hello</code> (only if valid)

Action	What it does
Pretty-print JSON	Format minified JSON with indentation
Minify JSON	Remove whitespace from JSON

Transformations **replace the selection** in the app (Sevak pastes over it) where pasting works. `Ctrl+Enter` copies the result instead of pasting. If pasting is not available (Wayland, or macOS without permission), they copy.

Workflows

A [workflow](#) with a **Universal Actions** trigger appears in this list for the kinds of selection it accepts (text, links, or files and folders). Picking it runs the workflow with the selection as its argument, for example to tidy the text and paste it back.

URLs and links

Select one or more URLs (starting with `http://`, `https://`, `mailto:` or `www.`):

Action	What it does
Open	Opens the URL(s) in your default browser
Copy	Copies to the clipboard (one line per URL)
Large Type	Shows the URL(s) huge on screen

Files and folders

Select file paths or drag files from a file manager:

Action	What it does
Open	Opens the file with its default application, or a folder in the file manager
Show in folder	Opens the file manager at the file's location

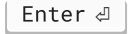
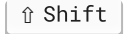
Action	What it does
Copy path	Copies the full path to the clipboard (one line per file)
Open in terminal	Opens a terminal in the folder (folders only)
Run as administrator	Runs the program with elevated privileges (Windows .exe programs only)
Send to Sevak	Fills Sevak's search box with the path for browsing (path browsing)

Text that looks like a path

If you select text that matches a file path (`C:\Users\me\Documents` , `/home/me/Projects` , `~/Downloads`), Sevak also offers the file actions above, then the text actions (search, calculate, etc.).

Actions and modifiers

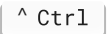
Press  to run the highlighted action. Modifiers change the behavior:

Key	Effect	Example
	Run the action (as shown)	Paste the transformation, open the URL, etc.
 + 	Copy instead of pasting (transformations only)	Copy <code>HELLO</code> to clipboard instead of pasting
 + 	Copy the URL (web searches only)	Copy the search URL without opening it

The row under each action hints at the modifiers.

Pick an action with number keys

Instead of arrow keys and Enter, use `Ctrl+1` through `Ctrl+9` to run the 1st through 9th action directly:

Key	Effect
 + 	Run the 1st action
 + 	Run the 2nd action
...	...
 + 	Run the 9th action

Dismiss the action panel

-  : Close the action panel without running anything.
-  : Close the action panel (same as Esc).
- Any other key: Close and continue typing in Sevak.

Platform notes

Windows

Selection capture works in every app **except windows running with elevated privileges** (Admin, UAC). Sevak tries anyway and shows what happens. For admin windows, select manually and set `[actions] use_clipboard_fallback = true` to act on what you copied.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → *Sevak*. Without this permission, Sevak cannot press Cmd+C in other apps and shows "Cannot read selection". Grant the permission and try again.

Files in Finder and text both work. Pasting transformations also needs Accessibility permission and respects your keyboard layout.

Linux (X11)

By default, Sevak reads the **PRIMARY selection** (text you just highlighted) first, without pressing any key (configurable: `[actions] use_primary_selection = true`). This is instant. If nothing is highlighted, it falls back to Ctrl+C.

You can also turn off primary selection and always use Ctrl+C: `use_primary_selection = false`.

Files are read from the clipboard. Terminal windows are skipped (Ctrl+C would interrupt the running program); use manual selection and clipboard fallback instead.

Linux (Wayland)

Applications cannot read another app's selection or press keys. Sevak shows "Cannot read selection on this desktop" or similar. Workaround:

1. Select the text manually.
2. Copy it (Ctrl+C or Cmd+C).
3. Set `[actions] use_clipboard_fallback = true` in your config.
4. Press Ctrl+Alt+Space again.

Sevak will then act on what you copied. This is not automatic, but it lets you use transformations and web searches.

Special cases

Very large selections

Sevak refuses selections larger than 256 KB to avoid lag. If you have a massive block of text selected, copy a smaller part and try again.

Terminal windows

On Windows and Linux, terminal windows (Windows Terminal, `cmd`, PowerShell, `gnome-terminal`, `konsole`, etc.) do not receive Ctrl+C from Sevak because it would interrupt the running program. Select manually or use clipboard fallback instead.

No admin permission needed on macOS

Unlike pasting, selection capture on macOS needs the same Accessibility permission but does not require you to run Sevak as admin.

Configuration

```
[general]
actions_hotkey = "Ctrl+Alt+Space"      # Hotkey to trigger actions; "" turns it off

[actions]
use_primary_selection = true           # Linux X11: read highlighted text first
use_clipboard_fallback = false        # Act on clipboard when capture fails
```

See [Configuration reference](#) for details on all settings.

Tips and troubleshooting

Bind actions to a different key

Choose a key not used by your desktop. On some layouts, Ctrl+Alt+Space types a no-break space; if that is your case, change the hotkey to `Ctrl+Alt+C` or another combination.

Use clipboard fallback on Wayland

If you are on Wayland and want to use transformations without the selection capture workaround, set `use_clipboard_fallback = true`. Then copy what you want to transform, press the hotkey, and pick an action. The result pastes or copies as normal.



Selection capture blocks keyboard input briefly

When you press the hotkey, Sevak waits for the modifiers to be released (up to 600ms) to avoid sending Ctrl+C while Ctrl is still down. If the hotkey's modifiers are held too long, Sevak releases them for the app. This ensures the copy succeeds, but it means your key bindings using those modifiers might behave unexpectedly for a moment.

Contacts

Search your address book and copy an e-mail address, write an e-mail, call a number or open the contact card. Type `c` or `@` and part of a name, address, number or company: `c ada` , `@lovelace` , `c 555 0100` .

This plugin is **off by default**.

Turn it on

Add to `config.toml` , then choose **Reload index** from the tray menu:

```
[contacts]
enabled = true
vcard_files = ["~/contacts.vcf"] # optional: .vcf files or folders of them
```

The Settings window has no fields for this section; its plugin list can still switch the plugin off.

How to use it

Names, e-mail addresses, phone numbers and company names are searched. Each row shows the person with their main address, number and company.

Key	Action
<code>Enter ↵</code>	Copy the e-mail address (the phone number if there is no address)
<code>^ Ctrl</code> + <code>Enter ↵</code>	Write an e-mail (opens your mail program)
<code>⇧ Shift</code> + <code>Enter ↵</code>	Copy the phone number
<code>⌘ Alt</code> + <code>Enter ↵</code>	Call the number (<code>tel:</code> , handled by whichever phone app you have)
<code>^ Ctrl</code> + <code>L</code>	Show the phone number as Large Type
<code>→ Right</code> or <code>^ Ctrl</code> + <code>K</code>	List these and more: open the contact card (macOS), copy other addresses and numbers

Where the contacts come from

Source	
Everywhere	vCard (<code>.vcf</code>) files and folders listed in <code>vcard_files</code> . Export one from your address book, Outlook, Google Contacts or a phone. No permission needed.
macOS	The Contacts app. The first time you type <code>c</code> , a row asks to allow access; press <code>Enter ↵</code> and answer macOS's question. Until you do, only vCard files are searched.
Windows	The People store behind the Mail and People apps. If your account's contacts are not in it, export a vCard file instead.
Linux	Evolution's local address books (<code>~/.local/share/evolution/addressbook</code>). Other address books: export a vCard file.

Set `use_system = false` to read vCard files only.

Options

Setting	Default	What it does	Config section
Enabled	<code>false</code>	Turn the plugin on	<code>[contacts]</code> <code>enabled</code>
Keyword	<code>c</code>	Keyword to search contacts; the <code>@</code> keyword always works too	<code>[contacts]</code> <code>keyword</code>
Use system	<code>true</code>	Also read the system address book (macOS Contacts, Windows People, Evolution)	<code>[contacts]</code> <code>use_system</code>
vCard files	<i>(empty)</i>	<code>.vcf</code> files or folders of them, e.g. <code>["~/contacts.vcf"]</code>	<code>[contacts]</code> <code>vcard_files</code>

Privacy

Contacts are loaded into memory when Sevak starts and every ten minutes, never written to disk, never logged and never sent anywhere. Searches and picks in this plugin are kept out of the search history and usage statistics, so no names end up in `usage.json`.

Troubleshooting

See `c`, `1p`, `define` or `spell` does nothing.

1Password

Find your 1Password logins by title or website and open the site or the item: `1p github`. Sevak uses the official [1Password command-line tool](#) ([op](#)) and **never reads a password, one-time code, note or any other secret**.

This plugin is **off by default**.

Turn it on

1. Install the 1Password CLI ([op](#)).
2. In the 1Password app, turn on *Settings* → *Developer* → *Integrate with 1Password CLI*.
3. Add to `config.toml`, then choose **Reload index**:

```
[onepassword]
enabled = true
```



How to use it

Type `1p` followed by a space and part of a login's title or website.

Key	Action
<code>Enter ↵</code>	Open the login's website in your browser (opens the item in 1Password if it has no website)
<code>^ Ctrl + Enter ↵</code>	Open the item in the 1Password app
<code>↑ Shift + Enter ↵</code>	Copy the username, when 1Password lists one
<code>→ Right</code> or <code>^ Ctrl + K</code>	Also: copy the website address

`op` runs only when you type `1p` followed by a space, never at startup or while you type a normal search. The first time (and again after `cache_minutes`, default 10) 1Password may ask you to unlock with Touch ID, Windows Hello or your system password; the row says "Asking 1Password..." until it answers. If you dismiss the prompt, Sevak does not ask again by itself: press `Enter ↵` on the "Try again" row.

Options

Setting	Default	What it does	Config section
Enabled	false	Turn the plugin on	[onpassword] enabled
Keyword	1p	Keyword to search logins	[onpassword] keyword
op path	" "	Path to op ; empty searches PATH and the usual install folders	[onpassword] op_path
Account	" "	Which account when several are signed in (address, short name or ID); empty uses op 's default	[onpassword] account
Cache	10	Minutes the list of logins is kept in memory (1–1440)	[onpassword] cache_minutes

Privacy

Sevak asks op only for the list of logins (`op item list --categories Login`), which contains titles, vault names, website addresses and usernames, and keeps that list in memory. To fill in a password, open the item in 1Password. What you search for and pick here is kept out of the search history and usage statistics. op is 1Password's own program and talks to 1Password as it normally does; Sevak itself sends nothing.

Troubleshooting

See `c`, `1p`, `define` or `spell` does nothing.

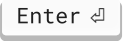
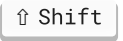
Dictionary and spelling

Look up a word's meanings or fix its spelling, offline. Type `define serendipity` or `spell recieve`.

How to use it

Define

`define serendipity` shows the meanings of a word, one row each with its part of speech.

Key	Action
	Copy the definition
 + 	Copy the definition with the word
 + 	Show it as Large Type
 on a "Did you mean" row	Look up that word instead

For a word that is not found, "Did you mean" rows offer close words. Inflected forms are explained through their base word (`running` shows `run`).

Spell

`spell recieve` lists corrections, best first.

Key	Action
	Paste the right spelling into the app you were using (copies where pasting is not possible)
 + 	Copy it

Typing a whole sentence checks it and corrects the first wrong word. Capitalization is kept: `Recieve` gives `Receive`.

Where the words come from

Everything is offline. macOS uses its own Dictionary for `define`, Windows its spell checker for `spell`, and otherwise Sevak uses a bundled English dictionary (WordNet, about 3 MB in the program). On Linux the word lists in `/usr/share/hunspell` also count as correct spellings.

Options

Setting	Default	What it does	Config section
Define keyword	<code>define</code>	Keyword for definitions	<code>[dictionary]</code> <code>define_keyword</code>
Spell keyword	<code>spell</code>	Keyword for spelling	<code>[dictionary]</code> <code>spell_keyword</code>
Use system	<code>true</code>	Prefer the OS dictionary and spell checker; <code>false</code> always uses the bundled data	<code>[dictionary]</code> <code>use_system</code>

The plugin is on by default. Turn it off by adding `"dict"` to `[plugins] disabled`.

Privacy

The words you look up are not saved in the search history, and nothing is sent anywhere. The bundled dictionary's licence is in [THIRD_PARTY_NOTICES.md](#).

Troubleshooting

See [c](#), [1p](#), `define` or `spell` does nothing.

Script plugins

Extend Sevak with custom search results by writing a script in Python, Node.js, PowerShell, or any language. A script plugin is a folder with a manifest (`plugin.toml`) and a script; drop it into your plugins folder and Sevak loads it on the next startup (or after **Reload index**).

This page covers installing and using script plugins. For writing one, see [Writing plugins](#).

Install a plugin

1. **Get the plugin folder.** Download or clone a script plugin (examples are in the [Sevak repository](#)).
2. **Find your plugins folder:**
3. **Windows:** `%APPDATA%\sevak\plugins`
4. **macOS:** `~/Library/Application Support/sevak/plugins`
5. **Linux:** `~/.config/sevak/plugins`

Or in Sevak, choose **Settings** → **Open config file**, then go up one folder to find the plugins folder.

1. **Copy the plugin folder** (with its `plugin.toml` and script files) into the plugins folder. The folder structure looks like:

```
~/.config/sevak/  
├─ config.toml  
├─ plugins/  
│   ├─ hello-python/  
│   │   ├─ plugin.toml  
│   │   └─ main.py  
│   └─ timestamp-powershell/  
│       ├─ plugin.toml  
│       └─ timestamp.ps1
```

1. **Approve the plugin** (first time only):
2. Restart Sevak or choose **Reload index** from the tray menu.
3. Sevak finds the new plugin and shows a warning dialog, **Sevak: new script plugin**, with its name, keyword, folder and the exact command it runs.
4. Click **Allow** only if you trust where the plugin came from: it runs with your account's permissions. **Not now** leaves it off and asks again the next time Sevak starts.
5. Sevak remembers the answer per plugin and command. If a plugin's command changes, you are asked again.

The plugin is now active. Type its keyword (defined in `plugin.toml`) to search it.

How to use a script plugin

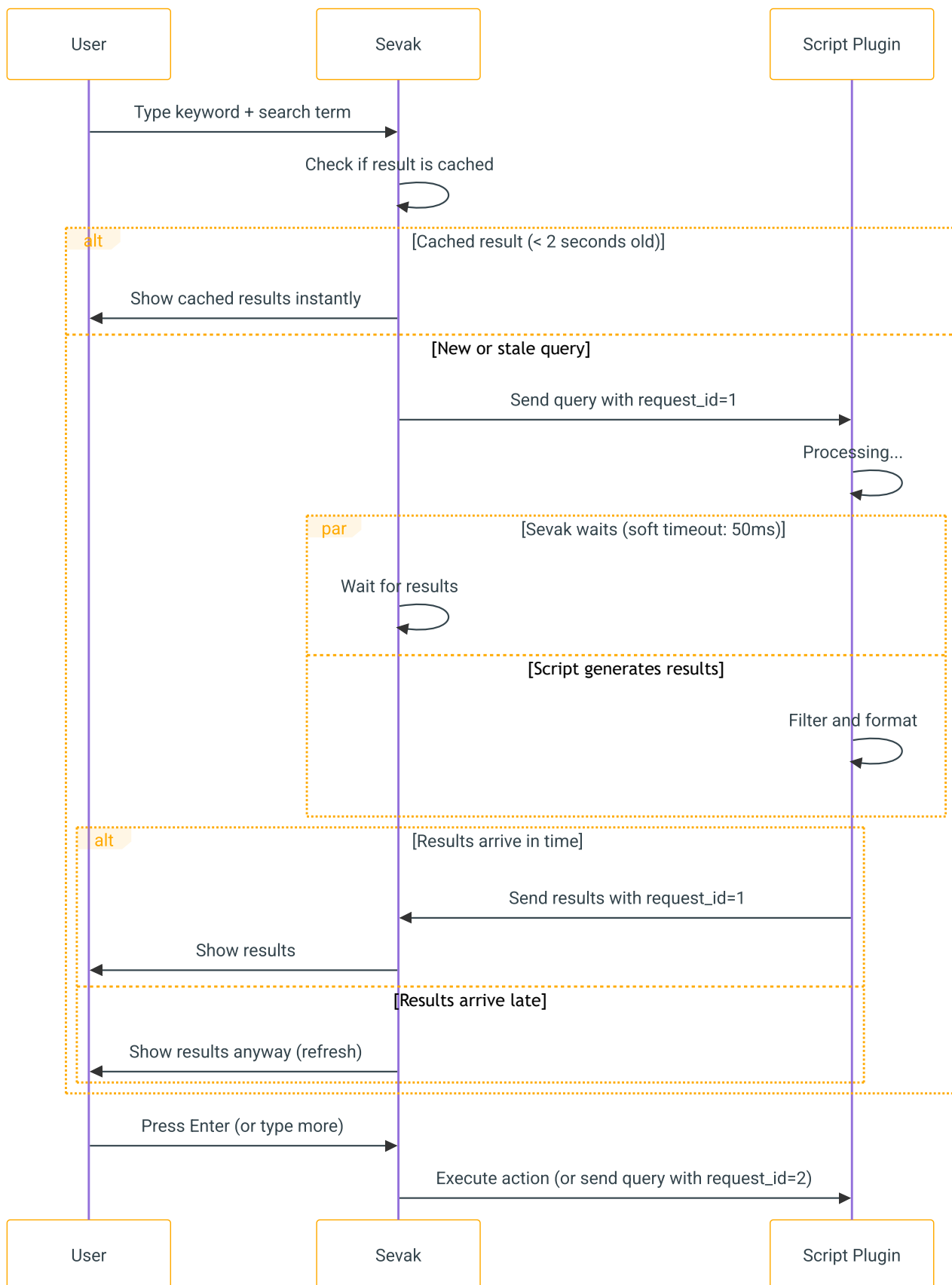
Each script plugin has a **keyword** (defined in its `plugin.toml`). Type the keyword followed by your search term:

Type	Shown	Press Enter ↵
<code>hello john</code>	Results from the "Hello" plugin matching "john"	Runs the action for the selected result
<code>ts</code>	Results from the "Timestamp" plugin (if ts is the keyword)	Runs the action

Results come from the script's output; the script decides what actions do (copy, paste, open a URL, etc.).

How script plugins communicate

A persistent-mode script runs once and listens on stdin for queries. Sevak and the script speak JSON lines:



Example: built-in plugins

Sevak's built-in plugins work the same way:

- `cb` is clipboard history's keyword
- `s` is snippets' keyword
- `>` is shell commands' keyword
- `f` is file search's keyword

Script plugins work exactly like these: type the keyword, get results, press Enter. If a script plugin's keyword is also used by a built-in search, a web search engine, a workflow or another script plugin, Sevak logs a warning and shows it in the plugin's description in Settings; the plugin still loads and both sets of results appear.

Built-in examples

The Sevak repository includes example plugins you can install:

hello-python

A Python plugin that greets you. Demonstrates a persistent process answering queries on stdin/stdout.

- **Location:** `examples/plugins/hello-python/`
- **Keyword:** `hello`
- **How to use:** Type `hello john` to see a greeting.

timestamp-powershell

A PowerShell plugin that generates timestamps. Demonstrates using PowerShell on Windows and other platforms.

- **Location:** `examples/plugins/timestamp-powershell/`
- **Keyword:** `ts`
- **How to use:** Type `ts` to see timestamp options (now, yesterday, tomorrow, etc.).

case-converter-node

A Node.js plugin that converts text case (uppercase, lowercase, title case, etc.). Demonstrates a one-shot plugin (fresh process per query).

- **Location:** `examples/plugins/case-converter-node/`
- **Keyword:** `case`
- **How to use:** Type `case hello` to see case-conversion options.

Views, modifiers and the gallery

- A script can ask for its rows to be shown as a **grid of tiles** (pictures, icons) or mark a row whose long text opens in the **Text View**; see [Views: text and grid](#). The [preview pane](#) works for script results too.
- Alfred's `mods` (secondary actions on `^ Ctrl` + `Enter`, `Alt` + `Enter`, `Shift` + `Enter`) are supported; see [Modifiers](#).
- **Settings** → **Gallery** can install ready-made script plugins, but only after you press **Load gallery** and **Install**; see [The gallery](#). An installed plugin still asks for permission before it runs.
- To chain a script with other steps (open a link, paste, show a notification), use a [workflow](#) with a script filter.

Enable and disable plugins

- **Enable:** Place the plugin folder in the plugins folder and reload.
- **Disable all script plugins:** In `config.toml`, add `"script"` to `[plugins] disabled`:

```
[plugins]
disabled = ["script"]
```

- **Disable a specific plugin:** Add its id (typically `script:<name>`) to the disabled list:

```
[plugins]
disabled = ["script:hello"]
```

Disabled plugins do not run, but their folders stay in place.

Script plugin types

Script plugins can work in two modes, defined in their `plugin.toml`:

Mode	How it works	When to use
Persistent (default)	One long-lived process speaks JSON lines on stdin/stdout	Complex plugins that benefit from caching or keeping state across queries
One-shot	A fresh process per query; it prints JSON and exits	Simple plugins, or quick ad-hoc scripts that do not need to stay running

A plugin's `plugin.toml` declares which mode it uses (`mode = "persistent"` or `mode = "oneshot"`). As a user, you do not need to know the difference—both work the same way.

Troubleshooting



Plugin did not load

1. Check that the plugins folder exists (create it if it does not).
2. Restart Sevak or choose **Reload index** from the tray.
3. Check Sevak's log for errors. Choose **Settings** → **Troubleshooting** (if available) or look at the log file (see [Files and data](#)).
4. Make sure the `plugin.toml` is valid TOML and has a `protocol`, `keyword`, and `script` or `command`.



Plugin asked for approval but I clicked the wrong thing

1. Note the plugin's id (shown in the approval prompt, e.g., `script:hello`).
2. Open `config.toml` and remove the id from `[plugins] disabled` if it was added.
3. Reload or restart Sevak.



Plugin appears but does not return results

1. Check the plugin's `plugin.toml` for the `script` or `command` field.
2. Verify the script file exists in the plugin folder.
3. On Linux and macOS, make sure the script is executable: `chmod +x script-file`.
4. Run the script manually from the plugin folder to check for errors (e.g., `python3 main.py hello` or `./main.py hello`).
5. Check Sevak's log for error messages.



Plugin hangs or is very slow

Script plugins have a timeout (soft: 50ms per query, hard: 3 seconds total). If your script is slow: 1. Check the plugin's `plugin.toml` for `timeout_ms` and `hard_timeout_ms` settings. 2. Optimize the script to respond faster (e.g., cache results, avoid network calls on every query). 3. For persistent plugins, make sure it is staying running between queries (check Sevak's log).



Share a script plugin

If you write a useful plugin, consider sharing it. Open an issue on the Sevak GitHub repository or link it in Discussions.

Learn more

For details on writing your own script plugin, see [Writing plugins](#).

Workflows

A workflow chains a **trigger** (a keyword, a shortcut, Universal Actions, a command) to **actions** and **outputs**: type `issue 14` and open issue 14, select text in any app and tidy it up, run your own script and show its result. You build workflows as boxes and connectors in **Settings** → **Workflows**, or write the small `workflow.toml` file yourself.

- [Try one in two minutes](#)
- [The builder](#)
- [Triggers](#)
- [Nodes](#)
- [Placeholders, the argument and variables](#)
- [How a run works](#)
- [Permission, privacy and limits](#)
- [The gallery](#)
- [The file format](#)
- [Coming from Alfred](#)
- [Troubleshooting](#)

Try one in two minutes

1. Open **Settings** → **Workflows** and press **New from template...**, then **Use this** next to *Open a URL with the query*. The builder opens it.
2. Press **Save** (Ctrl+S). The template runs no code, so Sevak does not ask for permission.
3. Open Sevak and type `issue 14`, then `issue dark mode`. The first opens issue 14 in your browser; the second searches the issues.

In the builder you can see why: the **Conditional** box looks at what you typed and follows its *then* connector for a number and its *else* connector for anything else.

The builder

Settings → **Workflows** lists every workflow, with a switch to turn each on or off, **Edit**, **Delete**, and **Review...** for one that waits for your permission. **New workflow** starts a blank one; **New from template...** offers three starting points (search a site, open a URL with the query, a script filter).

In the builder:

To	Do
Add a node	Pick it from Add a node... ; it appears at the top-left of what you see
Move a node	Drag its box (or focus it and use the arrow keys; <code>Shift</code> moves further)
Connect	Drag from the dot on the right edge of a box to another box. A conditional has two dots, <i>then</i> and <i>else</i>
Connect without a mouse	Select the node and use Connect to... in the side panel
Edit a node	Click it; its settings are in the side panel
Delete a node or a connection	Select it and press <code>Delete</code>
Arrange the boxes	Tidy
Save	Save or <code>Ctrl+S</code>

The builder refuses connections that cannot be right (to a trigger, to the same node, or one that would make a loop), and the **Problems** list under the side panel shows what else is wrong; a node with a red dot has an error, a yellow dot a warning. **Save** is only available when there are no errors. Warnings, such as a node nothing leads to, do not stop you.

Saving rewrites `workflow.toml` in the builder's own layout, so comments and formatting you added by hand are not kept. The builder never touches the scripts and other files in the folder.

Triggers

Every workflow starts at one or more triggers. A workflow may have several.

Trigger	Starts when	Argument
Keyword	you type the keyword (<code>docs rust</code>) and press Enter on its result	the text after the keyword

Trigger	Starts when	Argument
Script filter	you type its keyword and pick a row of the results its script printed	the row's <code>arg</code>
Hotkey	you press a global shortcut	empty
Universal Actions	you select text, a link or files, press <code>Ctrl+Alt+Space</code> and pick the workflow	the selection
External trigger	<code>sevak --trigger <folder>/<node id> [text] runs</code>	the text after it

Keyword. `argument` says whether text must follow: *optional* (the default), *required* (the result says "Type some text after the keyword" and does nothing until you do) or *none*. The label of the node is the result's title and may use `{query}`. Pressing `Tab` on the bare keyword completes it to `keyword`.

Keyword clashes. A keyword that is already used by a built-in search, a web search engine, a script plugin or another workflow (compared without regard to case) is only a warning, so the workflow still runs and both sets of results are shown. The warning names the other owner (for example `Keyword "g" is also used by web search Google; both will show results.`) and appears next to the node in the builder, on the Workflows page and in the log. Give the keyword another name to silence it.

Hotkey. The key uses the same syntax as the shortcut settings (`Ctrl+Alt+K`). Sevak registers it like a `[[hotkey]]` entry in `config.toml` (without writing it to that file), so it needs a system that lets Sevak grab keys: Windows, macOS and Linux with X11. On Wayland use an *external trigger* and bind a key to the command in your desktop's settings. Registration problems, such as a key another app owns, show on the workflow's row. The id of the result the key runs is `workflow:<folder>:run:<node id>`, so `sevak --run workflow:my-flow:run:key` does the same from a script.

Universal Actions. The node's *Offer it for choice* picks the kinds of selection that show the workflow in the actions list: *Text* (any text, including a link), *Links* (a selection that is only links) and *Files and folders*. The selection becomes the argument, one link or path per line when there are several; the variable `selection_kind` is `text`, `url` or `file`. See [Universal Actions](#) for how Sevak reads the selection.

External trigger. For anything that can run a command: a desktop shortcut on Wayland, a script, a file manager action, a scheduled task.

```
sevak --trigger my-flow/go           # no text
sevak --trigger my-flow/go some text # several words are one text
sevak --trigger my-flow/go -- --starts-with-dashes
```



`my-flow` is the workflow's folder name and `go` the id of the external trigger node (shown under *Id* in the builder). Sevak must be allowed to run the workflow, enabled, and valid; otherwise the launcher opens and says why.

Nodes

Actions

Node	Does	Notes
Run script	runs a program; what it prints becomes the argument	details below
Open URL	opens a web or mail link	only <code>http</code> , <code>https</code> and <code>mailto</code> ; placeholders are URL-encoded
Open file	opens a file or folder with its default program	<code>~</code> is your home folder; a relative path is inside the workflow's folder
Launch app	starts an application by name (as in the launcher) or by path	runs code, so needs permission
System command	<code>lock</code> , <code>sleep</code> , <code>hibernate</code> , <code>restart</code> , <code>shutdown</code> , <code>logout</code> , <code>empty_trash</code>	needs permission; the destructive ones ask each time
Terminal command	opens your terminal and runs a command line	needs permission; see quoting
Copy	puts text on the clipboard	
Paste	pastes text into the app you were using	copies instead where pasting is unavailable

Run script

Give either a **program and arguments** (one per line, the program first) or a **script file** in the workflow's folder, which Sevak starts the way it starts script plugins (`.py` with Python, `.ps1` with PowerShell, `.js` with Node, `.sh` with `sh`, anything else as an executable). **No shell is involved**: every argument is passed as it is, so `{query}` in an argument cannot run commands, no matter what it contains.

- The script runs in the workflow's folder.

- *More arguments* and *Standard input* may use placeholders.
- Its variables arrive as **environment variables**, as do `SEVAK_QUERY` (the argument, if under 16 kB), `SEVAK_WORKFLOW_ID`, `SEVAK_WORKFLOW_DIR`, `SEVAK_WORKFLOW_DATA` (a folder for the script's own files), `SEVAK_VERSION`, and the names Alfred workflows read (`alfred_workflow_*`). Variables named like `PATH`, `HOME`, `LD_*` and a few others are never exported, so a value cannot change how programs start. A node's own *Environment variables* are always passed.
- What it prints (standard output, up to 1 MiB, minus the final newline) becomes the **argument** of the next nodes. If it prints Alfred's envelope `{"alfredworkflow": {"arg": "...", "variables": {"name": "value"}}}`, the argument and the variables are set from it.
- It is stopped after its **timeout** (10 seconds unless you set another, up to 5 minutes). A non-zero exit ends the branch with "exited with code N".
- Its error output is **discarded**, because it may hold your data. Tick *Log its error output* while debugging to write its first 500 characters to Sevak's log.

Utilities

Node	Does
Set variable	<code>name = value</code> (placeholders allowed) for the nodes after it
Transform	changes the argument, or stores the result in a variable (<i>Store in variable</i>): set, UPPER, lower, Title Case, trim, URL-encode/decode, Base64 encode/decode, replace text, replace with a regular expression (<code>\$1</code> is a group), first line, split at some text and keep part <i>n</i>
Conditional	continues along <i>then</i> when the test holds, <i>else</i> otherwise: equals, does not equal, contains, does not contain, starts with, ends with, matches a regular expression, is empty, is not empty (optionally ignoring case)
Delay	waits up to 60 seconds

Regular expressions are Rust's [regex syntax](#): linear time, no look-around, and limited in size, so a pattern cannot hang Sevak.

Outputs

Node	Shows
Notification	a system notification (heading defaults to the workflow's name)
Large Type	the text huge on screen (dismiss with any key or a click)
Text view	the text in the launcher window, below the search bar

Script filter

A script filter is both a trigger and an input: it has its own keyword, runs a script whenever the text after the keyword changes (the text is the last argument), and shows the rows the script prints as results. Picking a row starts the nodes after it with the row's `arg` as the argument and its `variables` as variables. It speaks [Alfred's Script Filter JSON](#); the same rules as [script plugins](#) apply (a late script never blocks typing, results are cached for a moment, etc.), plus:

Item field	In a workflow
<code>arg</code>	the argument of the next nodes (the first string of an array)
<code>variables</code>	variables for the next nodes (text, numbers and booleans)
<code>valid: false</code>	the row only informs; Enter does nothing
<code>autocomplete</code>	what <code>Tab</code> types (after the keyword)
<code>mods</code>	secondary actions: <code>cmd</code> is <code>Ctrl+Enter</code> (<code>⌘↵</code> on macOS), <code>alt</code> is <code>Alt+Enter</code> , <code>shift</code> is <code>Shift+Enter</code> ; <code>ctrl</code> , <code>fn</code> and combinations are in the action panel (<code>→</code> or <code>Ctrl+K</code>). A mod's own <code>arg</code> and <code>variables</code> replace the row's, and the variable <code>mod</code> tells the next nodes which one was picked (<code>alt</code> , <code>cmd+alt</code> , ...)
<code>uid</code> , <code>title</code> , <code>subtitle</code> , <code>icon.path</code>	as in script plugins

Use a *Conditional* on `{var:mod}` to treat an `alt` pick differently.

The script filter uses the workflow's variables as environment variables (and `SEVAK_WORKFLOW_*`), and stops a run after *Stop the script after* (3 seconds by default). A script filter that gives both a `script` and extra arguments looks for its interpreter when the workflow loads, so choose **Reload index** after installing Python or Node.

Placeholders, the argument and variables

Text fields of nodes can use placeholders:

Write	Becomes
<code>{query}</code>	the argument that reached the node
<code>{var:name}</code>	the variable <code>name</code> (empty when it was never set)
<code>{query\ upper}</code>	filters, left to right

Filters: `url` (percent-encode), `raw` (insert as it is: no encoding in a link, no quoting in a command line), `upper`, `lower`, `title`, `trim`, `json` (escape for a JSON string), `sh` (quote as one POSIX shell word) and `ps` (quote as one PowerShell string). Other text in braces, such as JSON or `${HOME}`, is left alone.

- In **Open URL**, every placeholder is **percent-encoded**, so `https://example.com/?q={query}` is safe for any text. A variable that holds a whole address needs `|raw: {var:site|raw}/search?q={query}`.
- In **Terminal command**, every placeholder is **quoted as one literal word for the shell that runs the line** (the one `[shell]` picks: single quotes for `sh`, `bash`, `zsh` and the macOS login shell, PowerShell single quotes for `pwsh` and `powershell`, double quotes for `cmd`). So `grep {query} notes.txt` searches for the whole argument even if it is `x; rm -rf ~`, and nothing in it runs. `cmd` cannot quote `"`, `%`, `!` or a line break: an argument with one of them makes the node fail with an error rather than run. To put text into the command line *as commands*, opt out with `|raw (git {query|raw})`, and only for text you trust. `|sh` and `|ps` still pick a quoting style by hand. **Run script** passes arguments without any shell and needs none of this.
- Expanded text is capped at 4 MiB.

The **argument** starts as the trigger's text. **Run script** replaces it with what the script prints; **Transform** replaces it (or stores into a variable); every other node passes it on. **Variables** start from the workflow's own `[variables]` (edit them in the builder with nothing selected) and the trigger's (a script filter row's), and nodes can add more.

How a run works

- A node with several connections runs them **one after the other in the order they appear in the file** (the order you connected them), each with its own copy of the argument and variables. So a workflow always does the same things in the same order.

- A node that fails (a script that exits with an error or times out, a link that cannot open) **ends its branch** only; the other branches carry on. Sevak shows a notification naming the node and what happened.
- Everything runs on a thread of its own, never in the launcher or while you type. The launcher hides before a workflow starts, so a *Paste* node returns to the app you were using.
- Limits: at most 200 steps per run (a node on two branches counts twice), 15 minutes in total, 8 workflows at once. *Reload index*, saving settings or quitting stops workflows that are still running.
- The log names the workflow and the node ids (`workflow finished` , `node failed` and the kind of node), never what you typed, selected or copied, and never a program's output.

Permission, privacy and limits

Permission. A workflow whose nodes can run code or commands (**Run script**, **Script filter**, **Launch app**, **System command**, **Terminal command**) does nothing until you allow it. Sevak shows a dialog with the workflow's name, what starts it, what it runs and whether it will receive your Universal Actions selection, and remembers your answer in `script-plugin-approvals.json` (the file script plugins use). **Not now** asks again at the next start; the workflow's row has **Review...** if you change your mind.

What you allow is **what can run**: the settings of those nodes, the connections, the workflow's variables and the *contents* of the script files they name. If any of that changes (an edit in the builder, or a script file changed on disk), Sevak asks again after the next reload. Moving boxes, renaming and retitling do not. A workflow that only opens links and files, copies, pastes and shows text runs without asking, because it can do no more than the actions in the launcher can; it still never runs anything you did not trigger.

Two caveats. A script may read other files the workflow folder refers to (a module it imports) without Sevak noticing a change in them: install workflows you trust. And workflows are not sandboxed: a script has your account's permissions.

Privacy. Workflows never send anything anywhere on their own. What a script does on the network is up to the script. The Universal Actions selection reaches a workflow only when you pick it, lives in memory, is not in result ids, and does not touch the usage history or search history (workflows triggered by Universal Actions, hotkeys and `-- trigger` record nothing there; keyword results are recorded like any result).

The gallery

Settings → **Gallery** lists ready-made workflows and script plugins. It is **opt-in and on request**:

1. Opening the page requests nothing. **Load gallery** downloads one small file, `gallery/index.json` from the Sevak repository on GitHub (`raw.githubusercontent.com`), and shows what it lists.
2. **Install** on an entry downloads that one package, a zip, over HTTPS, checks it against the SHA-256 in the index and **discards it if it does not match**, refuses any path that would leave the folder (and links, oversized or too-many files), checks that the workflow or plugin inside is valid and unpacks it into the workflows or plugins folder. An existing folder is never overwritten.
3. The new folder is not allowed yet: Sevak shows its usual permission dialog before anything in it runs.

Each request sends only what any web request does (your address, and a user agent `Sevak/<version> (gallery)`). Nothing else leaves your computer, and Sevak keeps no account or identifier. The bundled examples are in [examples/](#) and their zips in [gallery/packages/](#).

To offer your own package: pack the folder (`cargo run -p sevak-plugins --example gallery_pack -- <folder> <out.zip>` prints the SHA-256), add an entry to `gallery/index.json` and open a pull request. See [plugins.md](#).

The file format

A workflow is the folder `<config folder>/workflows/<name>/` with `workflow.toml` (and scripts) in it. The config folder is next to `config.toml`: `%APPDATA%\sevak` on Windows, `~/Library/Application Support/sevak` on macOS, `~/.config/sevak` on Linux. Folders starting with `.` are ignored.

```
format = 1                                # the file format version
name = "Search docs"
description = "docs rust opens the Rust docs search."
author = "you"
version = "1.0"
enabled = true                            # false keeps it installed but off

[variables]                               # start-of-run variables (also environment variables for scripts)
site = "https://doc.rust-lang.org"

[[node]]
id = "kw"                                # unique; 1-40 letters, digits, - and _
type = "keyword"
title = "Search the docs for {query}"
x = 40                                    # canvas position; layout only
y = 80
keyword = "docs"
argument = "required"                    # none | optional | required

[[node]]
id = "open"
type = "open_url"
url = "{var:site|raw}/std/?search={query}"

[[connection]]
from = "kw"
to = "open"                             # port = "then" / "else" after a conditional
```

Node types and their fields: `keyword` (`keyword`, `argument`, `subtitle`), `hotkey` (`key`), `selection` (`accepts`), `external`, `script_filter` (`keyword`, `command` or `script`, `args`, `timeout_ms`, `hard_timeout_ms`), `run_script` (`command` or `script`, `args`, `stdin`, `env`, `timeout_ms`, `log_stderr`), `open_url` (`url`), `open_file` (`path`), `launch_app` (`app`, `args`), `system_command` (`command`), `terminal_command` (`command`), `copy` (`text`), `paste` (`text`, `restore_clipboard`), `set_variable` (`name`, `value`), `transform` (`op`, `input`, `find`, `replace`, `index`, `into`), `conditional` (`left`, `test`, `right`, `ignore_case`), `delay` (`ms`), `notification` (`heading`, `body`), `large_type` (`text`) and `text_view` (`heading`, `text`). Every node may also have `title`, `x` and `y`. Unknown type values make the whole workflow fail to load ("Not loaded: ..." on its row) rather than half-work. The examples in [examples/workflows/](#) show real files.

To disable every workflow, use `[plugins] disabled = ["workflow"]` in `config.toml`; the switch on a workflow's row (its `enabled` field) turns off one. `workflow:<folder>` also works in that list.

Coming from Alfred

Workflows borrow Alfred's ideas, not its file format: there is no importer for `.alfredworkflow` packages, and Sevak's nodes are fewer. Script Filter scripts and `{query}` / `{var:name}` placeholders work as in Alfred, and Alfred's `alfredworkflow` envelope and `alfred_workflow_*` environment variables are understood. Differences worth knowing:

- Nodes run in a fixed order, one branch after the other; Alfred runs branches in parallel.
- Scripts are never run through a shell: use `command` or `script` with `args`. `{query}` is substituted into arguments, not into a script's source.
- Modifier keys: Alfred's `cmd` is Sevak's `Ctrl` (Command on macOS).
- Anything that runs code needs your permission first, and the permission is tied to what runs.

Troubleshooting

Symptom	Likely cause
The workflow's row says Waiting for your permission	It runs code. Press Review... (or choose Reload index in the tray)
The row says Not loaded	The file has an error; the message says which. Fix... opens it in the builder
The keyword shows nothing	The workflow is switched off or waiting for permission; another plugin may also answer the keyword (both answer; the Workflows page and the builder warn about this)

Symptom	Likely cause
A hotkey does nothing	The row shows "The shortcut ... is not active" if another app owns it; on Wayland use an external trigger
A script node fails at once	The interpreter is not installed (the notification says "none of ... is on PATH"), or the path is wrong. Tick <i>Log its error output</i> and look at Reveal logs folder
<i>Paste</i> only copied	The system cannot paste (Wayland, or macOS without Accessibility permission); the text is on the clipboard
<code>sevak --trigger</code> opens the launcher with a message	The workflow or node does not exist, is off, or is not allowed yet

Reference

Configuration file reference

Sevak stores settings in a TOML file. You can edit it by hand or use the Settings window. This page documents every configuration key.

Where the file is

Windows

```
%APPDATA%\sevak\config.toml
```

Example: `C:\Users\Ninad\AppData\Roaming\sevak\config.toml`

macOS

```
~/Library/Application Support/sevak/config.toml
```

Linux

```
~/.config/sevak/config.toml
```

You can override the config folder with the `SEVAK_CONFIG_DIR` environment variable or the `--config` flag. The Settings window's "Open config file" button opens it in your editor.

Editing and reloading

After making changes to `config.toml`:

- Choose **Reload index** from the tray menu or menu bar (fastest).
- Or restart Sevak.

If the file has a mistake (invalid TOML or a value of the wrong type), Sevak never overwrites it. At startup it runs with the default settings; on **Reload index** it keeps the settings already in use. Either way the problem is written to the [log](#), so check there if a change seems to have no effect. Comments and key order are preserved when Settings saves changes.

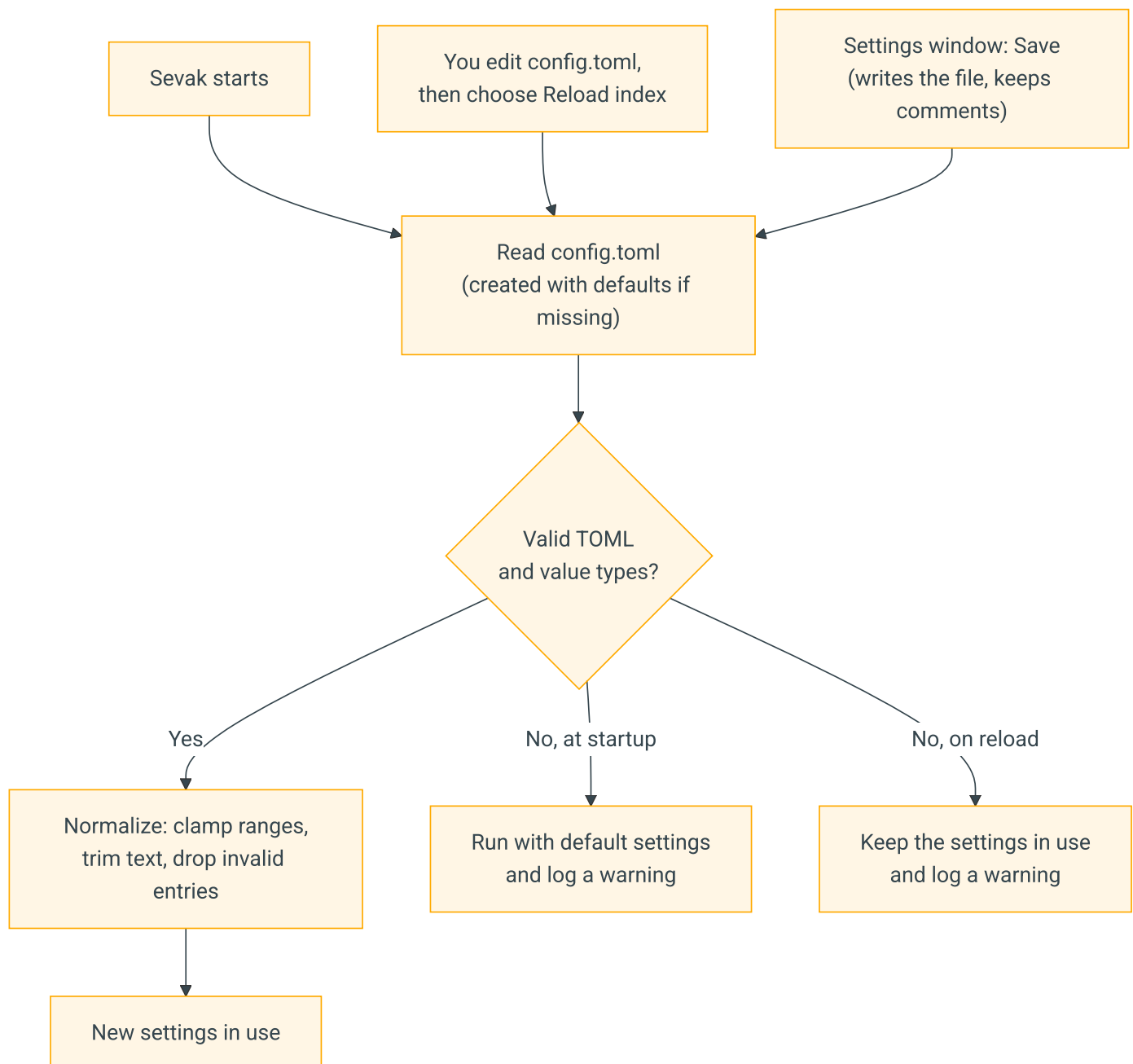
A minimal config works: missing fields use their defaults, so a nearly empty file is valid.

Keywords

Every keyword is one word, and no two searches may share one: the configurable keywords (`[files]` `keyword` / `index_keyword` / `content_keyword`, `[bookmarks]`, `[tasks]`, `[media]`, `[contacts]`, `[onpassword]`, `[dictionary]` `define_keyword` / `spell_keyword`), the fixed ones (`>`, `cb`, `s`, `emoji`, `:`, `@`, `uuid`) and your web search keywords. Settings refuses to save a clash; in a hand-edited file a clashing keyword is ignored and logged. An empty keyword turns that keyword off.

Keywords of [workflows](#) and [script plugins](#) are checked the same way but only produce a warning (in Settings and in the log), because both plugins then answer the keyword and show their results.

How configuration is loaded



Configuration sections

[general]

Main hotkeys and startup behaviour.

Key	Type	Default	Description
<code>hotkey</code>	string	<code>"Super+Space"</code>	Global keyboard shortcut to show/hide Sevak. <code>Super</code> is the Windows key (Cmd on macOS; <code>Win</code> , <code>Windows</code> and <code>Meta</code> are accepted spellings). Examples: <code>"Alt+Space"</code> , <code>"Ctrl+Space"</code> , <code>"Ctrl+Shift+K"</code> . On macOS, <code>Alt</code> is the Option key. Super+Space is used by the system too; Sevak takes it over (a keyboard hook on Windows; a question on macOS and GNOME) - see Troubleshooting . Existing config files keep the key they have. On Linux Wayland, run <code>sevak --setup-hotkey</code> to bind this in GNOME instead.
<code>actions_hotkey</code>	string	<code>"Ctrl+Alt+Space"</code>	Hotkey for Universal Actions: capture the selection in the foreground app and offer actions on it. Empty string <code>" "</code> turns Universal Actions off. On Wayland, run <code>sevak --setup-hotkey</code> to bind this in GNOME.
<code>hide_on_blur</code>	boolean	<code>true</code>	Hide the launcher when it loses focus to another window. Press Esc or click elsewhere to close; this setting hides it automatically.
<code>launch_at_login</code>	boolean	<code>false</code>	Start Sevak when you log in to your desktop.
<code>check_for_updates</code>	boolean	<code>true</code>	Check GitHub for a new version at startup and once daily. Updates are only installed after you confirm. Apart from optional currency rates, this is the only automatic network request.

```
[general]
hotkey = "Super+Space"
actions_hotkey = "Ctrl+Alt+Space"
hide_on_blur = true
launch_at_login = false
check_for_updates = true
```

[window]

Launcher window appearance.

Key	Type	Default	Range	Description
width	integer	720	400–1600 (logical pixels)	Width of the search bar window. Logical pixels scale with the display's DPI.

```
[window]
width = 720
```

[linux]

Linux-specific settings (ignored on Windows and macOS).

Key	Type	Default	Description
wayland_use_xwayland	boolean	true	On Wayland sessions, draw Sevak through XWayland (X11 compatibility layer). Native Wayland windows cannot position themselves or request focus reliably, so XWayland keeps the launcher centred and typeable. Set to <code>false</code> to use the native Wayland backend (window may appear off-centre and not accept input). Only used on Wayland; X11 sessions ignore this.

```
[linux]
wayland_use_xwayland = true
```

[search]

Search behaviour and fallback when no results match.

Key	Type	Default	Range	Description
<code>max_results</code>	integer	<code>8</code>	1–20	Maximum number of results shown at once.
<code>fallback_web_search</code>	string or list	<code>"g"</code>	any keyword from <code>[[web_search]]</code> entries	When a query has no local results, offer a web search with this engine. Write as a single string <code>"g"</code> or a list to try several in order: <code>["g", "yt", "gh"]</code> . Empty string <code>""</code> disables the fallback. Keywords are matched case-insensitively; only one keyword per engine.
<code>query_history</code>	boolean	<code>true</code>	—	Remember the last 50 queries you ran. Press Up/Down on an empty search box to recall them. <code>false</code> disables history and clears it.

```
[search]
max_results = 8
fallback_web_search = "g"
query_history = true
```



[appearance]

Launcher theme and styling.

Key	Type	Default	Range/Values	Description
<code>theme</code>	string	<code>"system"</code> <code>"</code>	<code>"system"</code> , <code>"light"</code> , <code>"dark"</code>	<code>"system"</code> follows your OS dark-mode preference. Invalid values fall back to <code>"system"</code>

Key	Type	Default	Range/Values	Description
				and are reported in the log and Settings.
<code>accent</code>	string	<code>""</code> (empty)	<code>"#rgb"</code> , <code>"#rrggbb"</code> , <code>"rgb(r, g, b)"</code> , or <code>""</code>	Override the theme's accent colour for buttons and highlights. <code>""</code> uses the theme's own accent. Invalid values fall back to <code>""</code> with a warning.
<code>font_size</code>	integer	<code>15</code>	12–22 (pixels)	Size of result titles. The rest of the launcher (icons, text, spacing) scales with it. Clamped to the range.
<code>font_family</code>	string	<code>""</code> (empty)	any font family name, comma-separated	Comma-separated font family list, e.g. <code>"Fira Sans, sans-serif"</code> . Empty uses your system font. CSS font-family syntax; if the font is missing, the next in the list is used.
<code>opacity</code>	integer	<code>100</code>	30–100 (percent)	Background opacity of the search bar. 100 is fully opaque; 30 is quite transparent. Whole numbers only.
<code>radius</code>	integer	<code>14</code>	0–32 (pixels)	Corner radius of the search bar. 0 is sharp corners; 32 is very rounded. Whole numbers only.
<code>theme_file</code>	string	<code>""</code> (empty)	path in config folder, or <code>""</code>	A theme file, such as <code>"themes/Nord.toml"</code> , made, imported or installed in Settings → Appearance → Theme editor . Applied

Key	Type	Default	Range/Values	Description
				between the built-in light/dark look and the settings above; "" uses none. See Theme files and the editor .
custom_css	string	"" (empty)	filename in config folder, or ""	Stylesheet to override theme colours and layout. Must be a file in the config folder (same folder as config.toml), e.g. "theme.css". "" loads none. See Themes guide for available CSS variables.

```
[appearance]
theme = "system"
accent = ""
font_size = 15
font_family = ""
opacity = 100
radius = 14
theme_file = ""
custom_css = ""
```

[plugins]

Which plugins are active. Each built-in plugin can be disabled.

Key	Type	Default	Description
disabled	array of strings	[]	Plugin ids to turn off. Available ids: "apps", "calculator", "web" (all engines) or "web:<keyword>" (one engine), "files" (its instances "files:names" and "files:content" are the ff and in searches), "bookmarks", "system", "tasks" (automation tasks), "media" (media controls), "shell", "clipboard", "snippets", "emoji" (the emoji picker; "emoji:word" and "emoji:colon" are its two keywords), "selection" (Universal Actions), "contacts", "1password", "dict" (define and spell), "uuid", "script" or "script:<name>"

Key	Type	Default	Description
			(script plugins) and "workflow" or "workflow:<folder>" (workflows ; each also has its own switch in Settings → Workflows). Example: <code>disabled = ["files", "web:yt"]</code> turns off file search and YouTube search. Unknown ids are ignored.

```
[plugins]
disabled = []
```



[calculator]

Calculator and unit conversion settings.

Key	Type	Default	Description
<code>currency</code>	boolean	<code>false</code>	Enable currency conversion (e.g. <code>100 usd in eur</code>). Off by default because it requires network access: Sevak downloads the European Central Bank's daily reference rates (a small XML file, no API key) at most once per day and caches them. Unit conversion (e.g. <code>10 km in mi</code>) never needs the network and is always on.

```
[calculator]
currency = false
```



[files]

Indexed file search settings.

Key	Type	Default	Description
<code>directories</code>	array of strings	<code>["~/Desktop", "~/Documents", "~/Downloads"]</code>	Root folders to index recursively. <code>~</code> expands to your home directory and works on all platforms. Relative paths are anchored at the working directory when Sevak starts (usually your home). Forward slashes work everywhere; avoid unescaped

Key	Type	Default	Description
			backslashes. The index caps at 100,000 entries; reduce roots or depth if you hit this.
<code>max_depth</code>	integer	<code>4</code>	How many folder levels below each root are indexed. <code>1</code> means only files directly in the root.
<code>include_hidden</code>	boolean	<code>false</code>	Index dot-files and dot-folders (starting with <code>.</code>). <code>false</code> ignores them.
<code>keyword</code>	string	<code>"f"</code>	Prefix to search only files: type <code>f filename</code> . Leave empty <code>"</code> to disable keyword search.
<code>global</code>	boolean	<code>true</code>	Show file results even in ordinary queries without the keyword. <code>false</code> requires <code>f <name></code> to search files.
<code>use_os_index</code>	boolean	<code>true</code>	Whole-disk (<code>ff</code>) and content (<code>in</code>) search through the operating system's own file index (Windows Search, Spotlight, <code>locate</code> , Tracker or Baloo). Queries go only to that local index, never over the network. <code>false</code> turns both off. See what each OS needs .
<code>index_keyword</code>	string	<code>"ff"</code>	Keyword for file names anywhere on the disk: <code>ff report</code> . <code>"</code> turns off just this search.
<code>content_keyword</code>	string	<code>"in"</code>	Keyword for words inside files: <code>in invoice 2026</code> . <code>"</code> turns off just this search.

```
[files]
directories = ["~/Desktop", "~/Documents", "~/Downloads"]
max_depth = 4
include_hidden = false
keyword = "f"
global = true
use_os_index = true
index_keyword = "ff"
content_keyword = "in"
```

[bookmarks]

Browser bookmark search settings.

Key	Type	Default	Description
browsers	array of strings	[]	Browser profiles to index. Empty [] means every browser found on your system. Supported ids: "chrome", "edge", "brave", "vivaldi", "chromium", "opera", "opera-gx", "firefox", "librewolf", "zen". All profiles of each browser are searched. Unknown ids are ignored.
keyword	string	"b"	Prefix to search only bookmarks: type <code>b term</code> . Leave empty "" to disable keyword search.
global	boolean	true	Show bookmark results in ordinary queries without the keyword. <code>false</code> requires <code>b <term></code> to search bookmarks.

```
[bookmarks]
browsers = []
keyword = "b"
global = true
```

[system]

System commands (lock, sleep, restart, etc.) and settings pages.

Key	Type	Default	Description
<code>confirm</code>	boolean	<code>true</code>	Ask for confirmation before restart, shut down, log out and emptying the trash. Power commands are destructive so default to "ask".
<code>disabled</code>	array of strings	<code>[]</code>	Commands and settings pages to hide. Options: <code>"lock"</code> , <code>"sleep"</code> , <code>"hibernate"</code> , <code>"restart"</code> , <code>"shutdown"</code> , <code>"logout"</code> , <code>"empty_trash"</code> , <code>"settings"</code> (all settings pages), or specific pages like <code>"settings:bluetooth"</code> , <code>"settings:wifi"</code> . Case-insensitive. Unknown entries are ignored. To turn off the entire plugin, add <code>"system"</code> to <code>[plugins]</code> <code>disabled</code> instead.

`[system]`

```
confirm = true
disabled = []
```



[tasks]

Automation tasks: dark mode, show the desktop, volume, screenshots, quit or kill an app, eject a drive, keep the computer awake and more.

Key	Type	Default	Description
<code>confirm</code>	boolean	<code>true</code>	Ask before force quitting an app, ending a process and restarting Explorer or Finder.
<code>disabled</code>	array of strings	<code>[]</code>	Tasks to hide: <code>"dark_mode"</code> , <code>"show_desktop"</code> , <code>"hide_others"</code> , <code>"minimize_all"</code> , <code>"screenshot"</code> , <code>"downloads"</code> , <code>"recent_files"</code> , <code>"flush_dns"</code> , <code>"restart_shell"</code> , <code>"empty_clipboard"</code> , <code>"mute"</code> , <code>"unmute"</code> , <code>"volume_up"</code> , <code>"volume_down"</code> , <code>"volume"</code> , <code>"wifi"</code> , <code>"bluetooth"</code> , <code>"keep_away"</code> , <code>"stop_keep_away"</code> , <code>"quit_app"</code> , <code>"force_quit_app"</code> , <code>"kill"</code> , <code>"eject"</code> . To turn the whole plugin off, add <code>"tasks"</code> to <code>[plugins]</code> <code>disabled</code> instead.

Key	Type	Default	Description
keyword	string	"t"	t lists the tasks. "" removes the keyword.
global	boolean	true	Also match task names in ordinary searches (dark mode , kill chrome).

```
[tasks]
confirm = true
disabled = []
keyword = "t"
global = true
```



[media]

Media controls: play/pause, next, previous, stop and what is playing now.

Key	Type	Default	Description
keyword	string	"play"	play lists the buttons and the track. "" removes the keyword.
global	boolean	true	Also match pause , next track ... in ordinary searches.
now_playing	boolean	true	Show the playing track (title, artist, app) as a row. It is read from the system's media player on request; nothing is stored or sent anywhere.

To turn the plugin off, add "media" to [plugins] disabled .

```
[media]
keyword = "play"
global = true
now_playing = true
```



[shell]

Shell command execution settings (type > <command> to run a command).

Key	Type	Default	Description
<code>terminal</code>	string	<code>""</code> (auto-detect)	Terminal program to run commands in. Empty means auto-detect: Windows Terminal (else a console window) on Windows, Terminal.app on macOS, <code>\$TERMINAL</code> environment variable then common terminals on Linux. You can give a program name (<code>"wt"</code> , <code>"iterm"</code> , <code>"kitty"</code>), a full path, or a command with arguments (<code>"alacritty --class sevak"</code>).
<code>shell</code>	string	<code>""</code> (auto-detect)	Shell that runs the command. Empty means auto-detect: <code>pwsh</code> , then <code>powershell</code> , then <code>cmd</code> on Windows; <code>\$SHELL</code> environment variable or <code>sh</code> on Linux. Ignored on macOS, where the terminal starts your login shell. You can give a program name or full path.
<code>keep_open</code>	boolean	<code>true</code>	Keep the terminal open at a shell prompt after the command exits. <code>false</code> closes it when done.

```
[shell]
terminal = ""
shell = ""
keep_open = true
```



[paste]

How text is pasted into the previously focused app.

Key	Type	Default	Description
<code>restore_clipboard</code>	boolean	<code>false</code>	After pasting from clipboard history or snippets, restore the previous clipboard contents. <code>false</code> leaves the pasted text in the clipboard.

```
[paste]
restore_clipboard = false
```



[actions]

Universal Actions: how the selection in another app is captured.

Key	Type	Default	Description
<code>use_primary_selection</code>	boolean	<code>true</code>	Linux X11 only: read the PRIMARY selection (highlighted text) first, without pressing Ctrl+C. Ignored on Wayland (no PRIMARY selection) and other platforms.
<code>use_clipboard_fallback</code>	boolean	<code>false</code>	When the selection cannot be captured (Wayland, terminal windows, missing macOS Accessibility permission), act on the current clipboard contents instead. <code>true</code> adds a fallback; <code>false</code> does nothing if capture fails.

[actions]

```
use_primary_selection = true
use_clipboard_fallback = false
```



[clipboard]

Clipboard history settings (opt-in feature).

Key	Type	Default	Description
<code>enabled</code>	boolean	<code>false</code>	Enable clipboard history (<code>cb <text></code> to search). Off by default: turning it on makes Sevak watch your clipboard and keep what you copy in its data folder, unencrypted: text and the paths of copied files in <code>clipboard-history.json</code> , images as PNG files in the <code>clipboard</code> folder. Content marked as secret by apps (password managers) is never recorded.

Key	Type	Default	Description
<code>max_items</code>	integer	<code>200</code>	How many clipboard items to keep, of all kinds together. Older items are dropped (with their image files). Range: 1–5000.
<code>max_item_bytes</code>	integer	<code>65536</code> (64 KiB)	Maximum size of a clipboard item in bytes. Longer text is not recorded. Range: 1–4,194,304 (4 MiB).
<code>images</code>	boolean	<code>true</code>	Also record copied images (as PNG files with thumbnails).
<code>files</code>	boolean	<code>true</code>	Also record copied files and folders (their paths only).
<code>max_image_bytes</code>	integer	<code>10485760</code> (10 MiB)	An image whose PNG is larger is not recorded. Range: 1–67,108,864 (64 MiB).
<code>ignore_apps</code>	array of strings	<code>[]</code>	Apps whose copies are never recorded, e.g. <code>["KeePassXC", "1Password"]</code> . Matched case-insensitively against the program or app name.

```
[clipboard]
enabled = false
max_items = 200
max_item_bytes = 65536
images = true
files = true
max_image_bytes = 10485760
ignore_apps = []
```

[file_buffer]

The [file buffer](#) (`Alt` + `Down` on a file result collects it). There is no settings page for it.

Key	Type	Default	Description
<code>keep_between_shows</code>	boolean	<code>false</code>	Keep the collected files when the launcher hides. By default the buffer is emptied

Key	Type	Default	Description
			whenever the launcher hides.

```
[file_buffer]
keep_between_shows = false
```

[contacts]

Contacts (`c <name>` or `@name`). Off by default. Contacts are read into memory only; nothing is written to disk or sent anywhere. The Settings window has no fields for this section.

Key	Type	Default	Description
<code>enabled</code>	boolean	<code>false</code>	Turn the plugin on.
<code>keyword</code>	string	<code>"c"</code>	Keyword to search contacts. The <code>@</code> keyword always works too. <code>" "</code> keeps the default.
<code>use_system</code>	boolean	<code>true</code>	Also read the system address book: macOS Contacts (asks for permission the first time you use it), the Windows People store and Evolution's local address books on Linux.
<code>vcard_files</code>	array of strings	<code>[]</code>	vCard files (<code>.vcf</code>) or folders of them, e.g. <code>["~/contacts.vcf", "~/Contacts"]</code> . Works everywhere and needs no permission.

```
[contacts]
enabled = false
keyword = "c"
use_system = true
vcard_files = []
```

[onpassword]

1Password logins (`1p github`) through the official `op` command-line tool. Off by default. Only titles, vault names, websites and usernames are read, never passwords or one-time codes. The Settings window has no fields for this section.

Key	Type	Default	Description
<code>enabled</code>	boolean	<code>false</code>	Turn the plugin on. <code>op</code> only runs when you type the keyword followed by a space.
<code>keyword</code>	string	<code>"1p"</code>	Keyword to search logins.
<code>op_path</code>	string	<code>" "</code>	Path to the <code>op</code> program. <code>" "</code> looks on <code>PATH</code> and in the usual install folders.
<code>account</code>	string	<code>" "</code>	Which account to use when several are signed in: its address (<code>my.1password.com</code>), short name or ID. <code>" "</code> uses <code>op</code> 's default.
<code>cache_minutes</code>	integer	<code>10</code>	How long the list of logins is kept in memory before <code>1p</code> refreshes it. Range: 1–1440.

```
[onpassword]
enabled = false
keyword = "1p"
op_path = " "
account = " "
cache_minutes = 10
```



[dictionary]

Dictionary and spelling, all offline. On by default; turn it off with `"dict"` in `[plugins] disabled`. The Settings window has no fields for this section.

Key	Type	Default	Description
<code>define_keyword</code>	string	<code>"define"</code>	Keyword for definitions.
<code>spell_keyword</code>	string	<code>"spell"</code>	Keyword for spelling corrections.
<code>use_system</code>	boolean	<code>true</code>	Use the macOS Dictionary and the Windows spell checker where available. <code>false</code> always uses the

Key	Type	Default	Description
			bundled English dictionary (WordNet) and word list.

```
[dictionary]
define_keyword = "define"
spell_keyword = "spell"
use_system = true
```



[snippets]

Expanding snippets as you type in any app. **Off by default:** while it is on, Sevak watches your keystrokes (in memory only, the last 64 characters, never stored or logged) to notice a snippet `keyword`. Not available on Wayland. The snippets themselves are the `[[snippet]]` entries below.

Key	Type	Default	Description
<code>auto_expand</code>	boolean	<code>false</code>	Expand snippet keywords as you type. Reloading applies changes; Sevak stops watching typing when this is turned off.
<code>prefix</code>	string	<code>" "</code>	Typed before every keyword, e.g. <code>" ;"</code> so that <code> ;sig</code> expands and <code> sig</code> does not. No spaces.
<code>expand_on</code>	string	<code>"immediate"</code>	<code>"immediate"</code> expands the moment the keyword is typed; <code>"delimiter"</code> waits for a space or punctuation mark, which is kept after the text. An unknown value is treated as <code>"delimiter"</code> .
<code>case_sensitive</code>	boolean	<code>true</code>	<code>false</code> : <code>SIG</code> and <code>sig</code> both expand.
<code>ignore_apps</code>	array of strings	<code>[]</code>	Never watch or expand in these apps, e.g. <code>["KeePassXC", "Firefox"]</code> . Matched case-insensitively against the program or app name, like <code>[clipboard]</code> <code>ignore_apps</code> .

Key	Type	Default	Description
<code>expand_in_terminals</code>	boolean	<code>false</code>	Terminal windows are skipped unless this is on.

```
[snippets]
auto_expand = false
prefix = ""
expand_on = "immediate"
case_sensitive = true
ignore_apps = []
expand_in_terminals = false
```

[[snippet]]

Snippets: text you paste often, with placeholders that expand.

Key	Type	Required	Description
<code>name</code>	string	yes	Name of the snippet; type <code>s <name></code> to search for it. Must not be empty.
<code>text</code>	string	yes	Text to paste. Can include newlines (<code>\n</code>). Must not be empty.
<code>keyword</code>	string	optional	Extra word to search by, e.g. <code>keyword = "sig"</code> lets you find it with <code>s sig</code> . With <code>[snippets] auto_expand</code> on, typing it in any app expands the snippet. Trimmed; empty keywords are ignored.

Placeholders in `text` :

- `{date}` — today's date in YYYY-MM-DD format
- `{time}` — current time in HH:MM:SS format (24-hour)
- `{datetime}` — date and time, e.g. 2025-10-03 14:30:00
- `{date:%d %B %Y}` — date in a custom format, e.g. 03 October 2025 (uses Rust strftime syntax)
- `{clipboard}` — contents of the clipboard at paste time
- `{uuid}` — a random UUID v4
- `{{ and }}` — literal `{` and `}` in the output

The Settings window never reformats hand-written snippets, so they survive a save. Unknown placeholders are left as-is.

```
[[snippet]]
name = "Email signature"
keyword = "sig"
text = "Best regards, \nNinad\n{date}"

[[snippet]]
name = "Meeting note"
text = "{datetime}: {clipboard}"
```

[[web_search]]

Web search engines. Type the keyword followed by your search terms, e.g. `g rust traits`.

Key	Type	Required	Description
keyword	string	yes	Short name to type before the query, e.g. "g" for Google. Must be unique and contain no spaces. Trimmed.
name	string	yes	Display name shown in results, e.g. "Google" .
url	string	yes	URL template. Must contain {query} (replaced by the URL-encoded search terms) and start with http:// or https:// .

If you define any `[[web_search]]` entries, they replace the defaults entirely. The defaults are:

```
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"
```

[[hotkey]]

Extra global hotkeys: bind a key to open Sevak with a query or run a result directly.

Key	Type	Required	Description
<code>key</code>	string	yes	Keyboard shortcut, e.g. <code>"Ctrl+Alt+T"</code> . Parsed like <code>[general] hotkey</code> . Must not be empty.
<code>query</code>	string	optional*	Open Sevak with this text already typed. Example: <code>query = "> "</code> opens a shell command prompt. Either <code>query</code> or <code>run</code> , not both.
<code>run</code>	string	optional*	Run the result with this id without showing the launcher. Example: <code>run = "apps:firefox.desktop"</code> . Either <code>query</code> or <code>run</code> , not both. Must not be empty.

*Exactly one of `query` or `run` must be present.

Result ids:

- Apps: `apps:firefox.desktop` (Linux desktop-file id), `apps:Chrome` or `apps:AppUserModelID` (Windows), `apps:Safari` (macOS)
- Files: `files:<full path>`
- Bookmarks: hover over a bookmark in Sevak to see its id
- System commands: `system:lock` , `system:sleep` , `system:restart` , `system:shutdown` , `system:logout` , `system:settings` , `system:settings:bluetooth` , `system:settings:wifi` , etc.
- Snippets: `snippets:<name>`
- Shell commands: `shell:<command>`
- Automation tasks: `tasks:dark_mode` , `tasks:volume:30` , `tasks:keep_awake:45` , `tasks:kill:chrome.exe` (see [Automation tasks](#))
- Workflows: `workflow:<folder>:run:<node id>` , the result a workflow's hotkey trigger runs; usually it is simpler to give the workflow its own hotkey trigger in the builder (see [Triggers](#))

Calculator, web search and clipboard history results have no stable id and cannot be bound.

On Wayland, run `sevak --setup-hotkey` after editing to create GNOME desktop shortcuts. Delete old entries in GNOME's keyboard settings after removing them from `config.toml`. On X11 and other desktops, the keys are registered directly by Sevak.

```
[[hotkey]]
key = "Ctrl+Alt+T"
query = "> "

[[hotkey]]
key = "Ctrl+Alt+L"
run = "system:lock"

[[hotkey]]
key = "Super+F"
run = "apps:firefox.desktop"
```

Files next to config.toml

Some settings are files of their own in the config folder rather than keys in `config.toml`:

Folder	Contents
<code>plugins/</code>	Script plugins , one folder each with a <code>plugin.toml</code>
<code>workflows/</code>	Workflows , one folder each with a <code>workflow.toml</code> ; the builder in Settings → Workflows writes them
<code>themes/</code>	Theme files made, imported or installed in Settings → Appearance ; <code>[appearance]</code> <code>theme_file</code> picks one

See [Files and data](#) for the data folder.

Complete example

This is the full default configuration written on first run:



```
# Sevak configuration
#
# Created with default values on first run. Edit it, then choose "Reload index"
# from the tray menu (or restart Sevak) to apply changes.

[general]
# Shortcut that shows and hides Sevak. Super is the Windows key (Cmd on macOS).
# Examples: "Super+Space", "Alt+Space", "Ctrl+Space", "Ctrl+Shift+K".
# Super+Space is normally taken by the system (Windows' input-language switcher,
# macOS Spotlight, GNOME's input sources). Sevak takes the key over on Windows
# with a keyboard hook, and on macOS and GNOME after asking you once. To go back
# to a key the system leaves alone, use "Alt+Space" (Option+Space on macOS).
# On Linux Wayland sessions applications cannot grab global keys. Run
# `sevak --setup-hotkey` to bind this key to `sevak --toggle` in GNOME instead.
hotkey = "Super+Space"

# Shortcut for Universal Actions: it copies what you have selected in the app
# you are using (text, a URL, files) and offers actions for it. "" turns it off.
# On Wayland run `sevak --setup-hotkey` to bind it to `sevak --actions`. See [actions].
actions_hotkey = "Ctrl+Alt+Space"

# Hide the window when it loses focus.
hide_on_blur = true

# Start Sevak in the background when you log in.
launch_at_login = false

# Check GitHub for a new version at startup and once a day. Updates are only
# installed after you agree. Apart from the optional currency rates (see
# [calculator]), this is the only request Sevak makes on its own.
check_for_updates = true

[window]
# Width of the search window in logical pixels (400-1600).
width = 720

[linux]
# On Wayland sessions, draw Sevak's window through XWayland. Native Wayland
# windows cannot position themselves and may be refused focus, so this keeps the
# launcher centered and typeable. Set to false to use the native Wayland backend.
wayland_use_xwayland = true

[search]
# Number of results shown (1-20).
max_results = 8
# Keyword of the web search engine offered when nothing else matches ("" to
# disable). A list offers several, in order: ["g", "yt", "gh"].
fallback_web_search = "g"
# Up/Down on an empty search box recalls the last 50 searches you ran. They are
# kept in usage.json in the data folder; false stops recording and forgets them.
query_history = true

[appearance]
# "system", "light" or "dark".
theme = "system"
# Accent color as "#rrggbb", "#rgb" or "rgb(r, g, b)". "" keeps the theme's own.
accent = ""
# Size of the result titles in pixels (12-22); the rest of the bar scales with it.
```

```

font_size = 15
# Font for the search bar, e.g. "Fira Sans, sans-serif". "" uses the system font.
font_family = ""
# How opaque the search bar's background is, in percent (30-100).
opacity = 100
# Corner radius of the search bar in pixels (0-32).
radius = 14
# A theme file inside this config folder, for example "themes/Nord.toml". Settings,
# Appearance, Theme editor creates and applies them. "" uses no theme file.
theme_file = ""
# A stylesheet inside this config folder that overrides the theme's CSS variables
# (see docs/themes.md), for example "theme.css". "" loads none.
custom_css = ""

```

[plugins]

```

# Ids of built-in plugins to turn off: "apps", "calculator", "files",
# "bookmarks", "system", "tasks", "media", "shell", "clipboard", "snippets",
# "emoji", "selection" (Universal Actions), "contacts", "1password", "dict",
# "web:<keyword>".
disabled = []

```

[calculator]

```

# Convert currencies ("100 usd in eur"). Off by default because it needs the
# network: when on, Sevak downloads the European Central Bank's daily reference
# rates (a small XML file, no account or key) in the background at most once a
# day and keeps them on disk. Unit conversion ("10 km in mi") never needs it.
currency = false

```

[files]

```

# Folders whose files and subfolders are searchable. "~" is your home folder.
directories = ["~/Desktop", "~/Documents", "~/Downloads"]
# How many folder levels below each directory are indexed.
max_depth = 4
# Index dot-files and dot-folders.
include_hidden = false
# Type "<keyword> <name>" to search only files.
keyword = "f"
# Also show (lower-ranked) file results for plain queries.
global = true
# Whole-disk search through the operating system's own index (Windows Search,
# Spotlight, locate / Tracker / Baloo). Nothing leaves your computer. Type
# "<index_keyword> <name>" for file names and "<content_keyword> <words>" for
# what is inside files. false turns both off.
use_os_index = true
index_keyword = "ff"
content_keyword = "in"

```

[bookmarks]

```

# Browsers whose bookmarks are searchable; [] means every browser found.
# Names: "chrome", "edge", "brave", "vivaldi", "chromium", "opera", "opera-gx",
# "firefox", "librewolf", "zen". All profiles of each browser are read.
browsers = []
# Type "<keyword> <text>" to search only bookmarks.
keyword = "b"
# Also show (lower-ranked) bookmark results for plain queries.
global = true

```

[system]

```

# System commands: lock, sleep, hibernate, restart, shut down, log out, empty

```

```

# the trash, and shortcuts to OS settings pages. Ask before restart, shut down,
# log out and emptying the trash.
confirm = true
# Commands or pages to hide: "lock", "sleep", "hibernate", "restart",
# "shutdown", "logout", "empty_trash", "settings" (every settings page) or
# "settings:<page>" such as "settings:bluetooth". To turn the whole plugin off,
# add "system" to [plugins] disabled instead.
disabled = []

[tasks]
# Automation tasks: toggle dark mode, show the desktop, mute and set the volume
# ("vol 30"), take a screenshot, quit an app ("quit"), kill a process by name
# ("kill chrome"), eject a drive ("eject"), keep the computer awake ("awake 30")
# and more. What is offered depends on your system.
# Ask before force quitting an app, ending a process and restarting Explorer or
# Finder.
confirm = true
# Tasks to hide: "dark_mode", "show_desktop", "hide_others", "minimize_all",
# "screenshot", "downloads", "recent_files", "flush_dns", "restart_shell",
# "empty_clipboard", "mute", "unmute", "volume_up", "volume_down", "volume",
# "wifi", "bluetooth", "keep_away", "stop_keep_away", "quit_app",
# "force_quit_app", "kill", "eject". To turn the whole plugin off, add "tasks"
# to [plugins] disabled instead.
disabled = []
# Type "<keyword> <task>" to search only tasks.
keyword = "t"
# Also show tasks for plain queries ("dark mode", "kill chrome").
global = true

[media]
# Media controls: play/pause, next, previous, stop, and what is playing now.
# Type "<keyword> <button>" to search only the controls.
keyword = "play"
# Also show the controls for plain queries ("pause", "next track").
global = true
# Show the track that is playing as a row (Enter plays or pauses it). It is read
# from the system's media player on request; nothing is stored or sent anywhere.
now_playing = true

[shell]
# Type "> <command>" (or ">command") to run a command in a terminal window. It
# only runs when you press Enter; recent commands are offered again.
# Terminal to use. "" detects one: Windows Terminal (else a console window) on
# Windows, Terminal.app on macOS, $TERMINAL then common terminals on Linux.
# Examples: "wt", "iterm", "kitty", "gnome-terminal", "alacritty --class sevak".
terminal = ""
# Shell that runs the command. "" picks pwsh, powershell, then cmd on Windows
# and $SHELL (or sh) on Linux. Not used on macOS: your login shell runs it.
shell = ""
# Leave the terminal open, at a shell prompt, after the command exits.
keep_open = true

[paste]
# Clipboard history and snippets paste into the app you were using before Sevak
# opened. With this on, the clipboard's previous text is put back afterwards.
restore_clipboard = false

[actions]
# Universal Actions (see actions_hotkey under [general]). Sevak presses Ctrl+C

```

```

# (Cmd+C) in the app you were using, reads the result and puts your clipboard
# back. The selection is never stored or logged. Terminal windows are skipped
# on Windows and Linux because Ctrl+C would interrupt the running program.
# Linux (X11): read the PRIMARY selection (text you just highlighted) first,
# without pressing a key.
use_primary_selection = true
# When the selection cannot be captured (Wayland, a terminal, missing macOS
# Accessibility permission), act on the current clipboard contents instead.
use_clipboard_fallback = false

[file_buffer]
# The file buffer (Alt+Up / Alt+Down on a file result collects it). By default it
# is emptied whenever the launcher hides; true keeps what you collected.
keep_between_shows = false

[clipboard]
# Clipboard history ("cb <text>"). Off by default: turning it on makes Sevak
# watch the clipboard and keep what you copy in clipboard-history.json in its
# data folder: text, images (as PNG files in a "clipboard" folder next to it)
# and the paths of copied files. All of it is stored unencrypted. Content that
# apps mark as secret (password managers) is never recorded.
enabled = false
# Items kept, of all kinds together (the oldest are dropped).
max_items = 200
# Longer text is not recorded.
max_item_bytes = 65536
# Record copied images, and copied files and folders (paths only).
images = true
files = true
# An image whose PNG is larger than this is not recorded.
max_image_bytes = 10485760
# Never record text copied from these apps, e.g. ["KeePassXC", "1Password"].
# Matched case-insensitively against the program or app name.
ignore_apps = []

[contacts]
# Search your contacts ("c <name>" or "@name"): copy an email or phone number,
# write an email, call (tel: link), or open the card. Off by default. Contacts
# are read into memory only; nothing is written to disk or sent anywhere.
enabled = false
# Keyword (the "@" keyword always works too). "" keeps the default.
keyword = "c"
# Also read the system address book: macOS Contacts (asks for permission the
# first time you use it), the Windows People store and Evolution's local
# address books on Linux.
use_system = true
# vCard files (.vcf) or folders of them, e.g. ["~/contacts.vcf", "~/Contacts"].
# This works everywhere and needs no permission.
vcard_files = []

[onpassword]
# Search your 1Password logins ("1p github"): Enter opens the item's website,
# the action panel opens it in the 1Password app or copies the username. Needs
# the official `op` command-line tool, signed in (1Password > Settings >
# Developer > "Integrate with 1Password CLI"). Only titles, websites and
# usernames are read, never passwords or one-time codes. Off by default.
enabled = false
keyword = "1p"
# Path to the `op` program. "" looks on PATH and in the usual install folders.

```

```

op_path = ""
# Which account to use when several are signed in: its address (my.1password.com),
# short name or ID. "" uses op's default.
account = ""
# How long the list of logins is kept in memory before `1p` refreshes it.
cache_minutes = 10

[dictionary]
# "define <word>" shows definitions and "spell <word>" suggests corrections, all
# offline. macOS uses its Dictionary and Windows its spell checker; elsewhere a
# bundled English dictionary (WordNet) is used. Turn it off with "dict" in
# [plugins] disabled.
define_keyword = "define"
spell_keyword = "spell"
# false always uses the bundled dictionary and word list.
use_system = true

# Snippets ("s <name>"): text you paste often. Placeholders: {date}, {time},
# {datetime}, {date:%d %B %Y}, {clipboard}, {uuid}; write {{ and }} for literal
# braces. "keyword" is optional and also matches the search.
# [[snippet]]
# name = "Email signature"
# keyword = "sig"
# text = "Best regards,\nNinad"

[snippets]
# Expand snippets as you type in any app (a snippet needs a "keyword"). OFF by
# default: while on, Svak watches your keystrokes (in memory only, last 64
# characters, never stored or logged) to notice a keyword. See "Privacy" in the
# README. Not available on Wayland.
auto_expand = false
# Typed before every keyword, e.g. ";" so that ";sig" expands and "sig" does not.
prefix = ""
# "immediate" expands the moment the keyword is typed; "delimiter" waits for a
# space or punctuation mark, which is kept after the text.
expand_on = "immediate"
# false: "SIG" and "sig" both expand.
case_sensitive = true
# Never expand in these apps (program or app name, case-insensitive).
ignore_apps = []
# Terminal windows are skipped unless this is on.
expand_in_terminals = false

# Web search engines: type "<keyword> <terms>". "{query}" is replaced by the
# URL-encoded terms. Defining any [[web_search]] entry replaces this list.
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"

```

```
# Extra global hotkeys. Each [[hotkey]] has a "key" and exactly one of:
#   query = "... " open Sevak with this text already typed
#   run = "... "   run a result directly, without showing Sevak; the value is a
#                   result id such as "apps:firefox.desktop" or "files:<full path>"
# On Linux Wayland, `sevak --setup-hotkey` binds these in GNOME as well.
#
# [[hotkey]]
# key = "Ctrl+Alt+T"
# query = "> "
#
# [[hotkey]]
# key = "Ctrl+Alt+F"
# run = "apps:firefox.desktop"
```

Command-line reference

Sevak can be controlled from the command line or automated with scripts. When Sevak is already running, a new command forwards its request to the running instance (single-instance architecture).

Quick reference

Flag	Effect
(none)	Start Sevak; show the search bar if not already visible
<code>--toggle</code>	Show the search bar, or hide it if visible
<code>--query TEXT</code>	Show the search bar with TEXT already typed
<code>--run ID</code>	Run a result by its id without showing the search bar
<code>--actions</code>	Universal Actions: capture the selection in the foreground app
<code>--trigger WORKFLOW/ID [TEXT]</code>	Start a workflow's external trigger, with TEXT as its argument
<code>--background</code>	Start Sevak without showing the window
<code>--settings</code>	Open the Settings window
<code>--quit</code>	Quit the running instance
<code>--setup-hotkey [KEY]</code>	Bind hotkeys to GNOME (Wayland desktop on Linux); offers to free Super+Space
<code>--restore-hotkey</code>	Put back system shortcuts Sevak changed with your permission (GNOME input sources, macOS Spotlight)
<code>--config PATH</code>	Use a custom config folder or file; overrides <code>SEVAK_CONFIG_DIR</code>
<code>-h, --help</code>	Print help

Flag	Effect
<code>-V, --version</code>	Print version

Environment variables

Variable	Purpose
<code>SEVAK_CONFIG_DIR</code>	Path to the config folder (overridden by <code>--config</code>). If this is set and the folder exists, the config is read from there.
<code>SEVAK_DATA_DIR</code>	Path to the data folder (usage statistics, logs). If set, overrides the default platform-specific location.
<code>SEVAK_LOG</code>	Log level filter. Examples: <code>SEVAK_LOG=debug</code> for detailed output, <code>SEVAK_LOG=warn</code> for warnings and errors only. Defaults to <code>info</code> .

Flags

Start or show

(no arguments)

`sevak`

Start Sevak in the background if it is not running. If it is already running and the search bar is hidden, show it. Does nothing if the bar is already visible.

`--toggle`

`sevak --toggle`

Toggle the search bar: show it if hidden, hide it if visible. This is the primary way to bind Sevak to a global hotkey. On Wayland, where applications cannot grab global keys, bind this to a key in your desktop settings (or use `sevak --setup-hotkey` on GNOME).

`--query TEXT`

```
sevak --query "firefox"
sevak --query "> pip install requests"
sevak --query ""
```

Show the search bar with TEXT already typed. Used to:

- Open Sevak with a specific search, e.g. "firefox" to search for Firefox
- Pre-fill a shell prompt with "> " or ">" (see [Shell commands](#))
- Open an empty launcher with an empty string ""

--background

```
sevak --background
```

Start Sevak in the background. The search bar does not appear. Useful for the `launch_at_login` setting or autostart scripts.

--run ID

```
sevak --run "apps:firefox.desktop"
sevak --run "files:/home/user/Documents"
sevak --run "system:lock"
```

Run a result by its id directly without showing the search bar. The launcher stays hidden. Useful for hotkey bindings, e.g. bind `Ctrl+Alt+L` to `sevak --run "system:lock"`.

See [custom hotkeys](#) for result id formats. Automation tasks have ids too: `tasks:dark_mode`, `tasks:volume:30`, `tasks:keep_awake:45`, `tasks:kill:chrome.exe` (see [Automation tasks](#)).

--actions

```
sevak --actions
```

Universal Actions: capture what you have selected in the currently focused app (text, a URL, files) and offer actions on it. On Wayland, bind this to a key in your desktop settings or use `sevak --setup-hotkey` on GNOME.

--trigger WORKFLOW/ID [TEXT]

```
sevak --trigger my-flow/go                # no text
sevak --trigger my-flow/go some text      # several words are one text
sevak --trigger my-flow/go -- --starts-with-dashes
```

Start the external trigger `ID` of the [workflow](#) in the folder `WORKFLOW` (under `workflows/` next to `config.toml`), with the rest of the command line as its argument. The words after it are its text, up to the next option; `--` makes everything after it text. Use it from a desktop shortcut on Wayland, a script, a file manager action or a scheduled task.

The workflow must be enabled, valid and allowed to run; otherwise the launcher opens and says why. A workflow's hotkey trigger can also be run with `sevak --run workflow:<folder>:run:<node id>`.

Interaction

`--settings`

```
sevak --settings
```

Open the Settings window. If Sevak is not running, start it with the Settings window shown.

`--quit`

```
sevak --quit
```

Quit the running instance. If Sevak is not running, this does nothing and exits cleanly.

Configuration

`--config PATH`

```
sevak --config ~/.local/share/sevak
sevak --config /etc/sevak/config.toml
sevak --config .\Config
```

Use a custom config folder or config file instead of the default location. Takes precedence over the `SEVAK_CONFIG_DIR` environment variable.

- If `PATH` ends in `.toml` and is not an existing directory, it is treated as the config file itself; the folder containing it is the config folder.
- Otherwise, `PATH` is treated as the config folder; the file is `config.toml` inside it.
- `~` is expanded to your home directory.
- Relative paths are resolved relative to the current working directory.

This flag only affects a newly started instance. If Sevak is already running, it keeps its own config folder; the flag is noted in the log for reference.

`--setup-hotkey [KEY]`

(Linux Wayland with GNOME only)

```
sevak --setup-hotkey          # use the hotkey from config.toml
sevak --setup-hotkey "Ctrl+Space"
sevak --setup-hotkey "Super+F"
```

Create GNOME custom keyboard shortcuts that bind your hotkeys to Sevak commands. On Wayland, where applications cannot register global keys, this command asks GNOME to handle key events and forward them to Sevak.

It creates shortcuts for:

- The main hotkey (`[general] hotkey`) → `sevak --toggle`
- Universal Actions hotkey (`[general] actions_hotkey`) → `sevak --actions`
- Each `[[hotkey]]` entry → `sevak --query '<text>'` or `sevak --run <id>`

If you pass a KEY, it overrides the hotkey from `config.toml` just for this setup. The config file is not modified.

On other desktops (KDE, Sway, etc.), bind hotkeys through your desktop's keyboard settings. On X11, hotkeys are registered directly by Sevak and this command is not needed.

After editing `config.toml`, run `--setup-hotkey` again to update GNOME shortcuts. Delete old shortcuts in GNOME's keyboard settings if you remove entries from the config.

GNOME uses `Super + Space` to switch input sources, so it would keep that key from Sevak. When your hotkey is `Super+Space` (the default) and GNOME's switcher uses it, `--setup-hotkey` explains what it would change, shows the old value and asks you to confirm (type `y`). It then moves the input-source shortcuts (`switch-input-source` and `-backward` under `org.gnome.desktop.wm.keybindings`) to the same keys with Ctrl added, saves the previous values in `hotkey-takeover.json` in the data folder and prints what changed. Without a terminal it asks nothing and changes nothing. Undo it with `sevak --restore-hotkey`.

`--restore-hotkey`

```
sevak --restore-hotkey
```

Puts back the system shortcuts Sevak changed with your permission: GNOME's input-source shortcuts (Linux) or Spotlight's "Show Spotlight search" shortcut (macOS). It prints what it restored, or says there was nothing to restore. On Windows Sevak never changes a system setting, so there is nothing to restore. After restoring, Sevak's own shortcut may clash with the system's again; choose another key in Settings (for example `Alt + Space`) or run `sevak --setup-hotkey KEY`.

Information

`-h, --help`

```
sevak --help
sevak -h
```

Print the help message and exit.

`-V, --version`

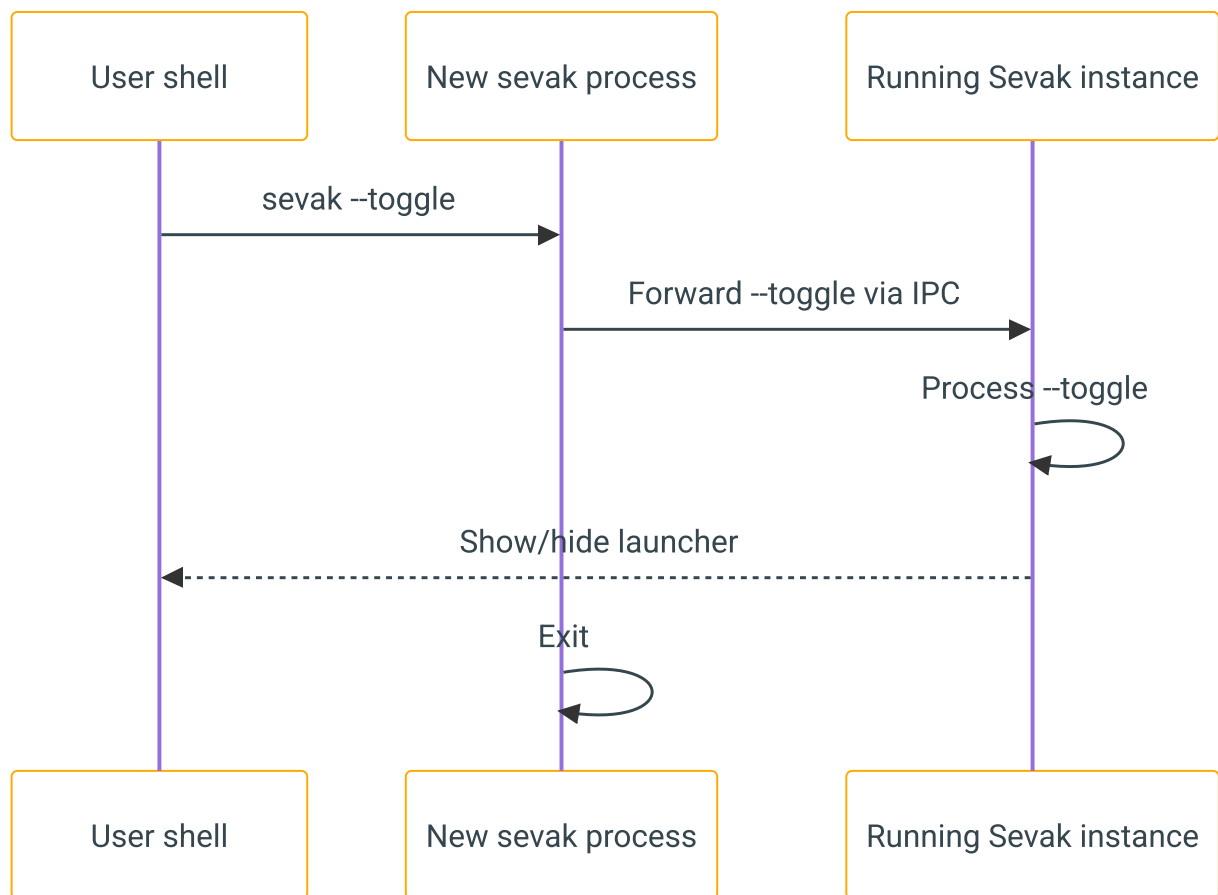
```
sevak --version
sevak -V
```



Print the version and exit.

Single-instance forwarding

When you run `sevak` and the app is already running, your command is forwarded to the running instance. Here is how it works:



This means:

- Only one instance of Sevak runs at a time.
- Commands from new invocations are processed by the running instance.
- If Sevak is not running, the new invocation starts it.
- The `--config` flag is ignored by the running instance (it keeps its own config path).

Exit codes:

Code	Meaning
0	Success (including <code>--help</code> and <code>--version</code>)
1	Sevak could not start or run (the error is printed and logged)
2	Invalid command line: unknown flag, missing value or conflicting flags (the usage text is printed)

Examples

Bind a key to show/hide Sevak with the macOS system shortcuts:

Add to your keyboard shortcuts under System Settings:

```
/Applications/Sevak.app/Contents/MacOS/sevak --toggle
```



Bind Ctrl+Alt+T to open a shell prompt on Windows:

Use Task Scheduler or a tool like AutoHotkey to run:

```
sevak --query "> "
```



Lock the desktop from anywhere:

```
sevak --run "system:lock"
```



Search for a specific file from a script:

```
sevak --query "f my-project"
```



Start Sevak on login (Linux):

Add to your `.bash_profile` or autostart script:

```
sevak --background
```



Check Sevak version in a script:

```
sevak --version
```



Development and debugging

Enable debug logging:

Bash/Linux/macOS

```
SEVAK_LOG=debug sevak
```



PowerShell/Windows

```
$env:SEVAK_LOG = "debug"  
sevak
```



Windows CMD

```
set SEVAK_LOG=debug  
sevak
```



Logs are written to:

Windows

```
%APPDATA%\sevak\logs\
```

macOS

```
~/Library/Application Support/sevak/logs/
```

Linux

```
~/local/share/sevak/logs/
```

Test a config file before deployment:

```
sevak --config /path/to/test/config.toml --version
```



This loads the config without starting the UI. Parsing errors are printed to the terminal.

Files and data locations

Sevak stores configuration, themes, script plugins, workflows, logs, usage history and clipboard history in well-known locations on each platform. You can override these with environment variables or command-line flags.

Default locations

Item	Windows	macOS	Linux
Config folder	%APPDATA%\sevak\	~/Library/Application Support/sevak/	~/.config/sevak/
Config file	%APPDATA%\sevak\c onfig.toml	~/Library/Application Support/sevak/config.to ml	~/.config/sevak/con fig.toml
Data folder	%APPDATA%\sevak\	~/Library/Application Support/sevak/	~/.local/share/seva k/
Logs	%APPDATA%\sevak\l ogs\	~/Library/Application Support/sevak/logs/	~/.local/share/seva k/logs/
Usage history	%APPDATA%\sevak\u sage.json	~/Library/Application Support/sevak/usage.jso n	~/.local/share/seva k/usage.json
Clipboard history	%APPDATA%\sevak\c lipboard- history.json	~/Library/Application Support/sevak/clipboard -history.json	~/.local/share/seva k/clipboard- history.json
Clipboard images	%APPDATA%\sevak\c lipboard\	~/Library/Application Support/sevak/clipboard /	~/.local/share/seva k/clipboard/
Theme files	%APPDATA%\sevak\t hemes\	~/Library/Application Support/sevak/themes/	~/.config/sevak/the mes/

Item	Windows	macOS	Linux
Script plugins	%APPDATA%\sevak\plugins\	~/Library/Application Support/sevak/plugins/	~/.config/sevak/plugins/
Workflows	%APPDATA%\sevak\workflows\	~/Library/Application Support/sevak/workflows/ /	~/.config/sevak/workflows/
Shortcut takeover record	%APPDATA%\sevak\hotkey-takeover.json	~/Library/Application Support/sevak/hotkey-takeover.json	~/.local/share/sevak/hotkey-takeover.json

Theme files, script plugins and workflows sit next to `config.toml` in the config folder; clipboard images, approvals and the data folders of plugins and workflows are in the data folder. On Windows and macOS the two folders are the same.

Expand `~` to your home directory and `%APPDATA%` to your roaming app data folder.

Override locations

Config folder

Command line

```
sevak --config ~/my-sevak
sevak --config /etc/sevak/config.toml
```

Windows

```
sevak --config %env:USERPROFILE%\Sync\sevak
```

Environment variable

```
export SEVAK_CONFIG_DIR=~/my-sevak
sevak
```

Windows

```
%env:SEVAK_CONFIG_DIR = "%env:USERPROFILE%\Sync\sevak"
sevak
```

The `--config` flag takes precedence over the environment variable.

Data folder (logs, usage, clipboard history, approvals)

Environment variable

```
export SEVAK_DATA_DIR=~/.local/share/my-sevak  
sevak
```

Windows

```
$env:SEVAK_DATA_DIR = "D:\Data\sevak"  
sevak
```

Note: the data folder cannot be overridden via command line; only the `SEVAK_DATA_DIR` environment variable works.

What each file contains

config.toml

Your configuration: shortcuts, appearance, enabled plugins, file search roots, etc. See [Configuration file reference](#).

Edited by: You (manually or through the Settings window).

When to back up: Before making large manual edits or testing unusual configurations.

logs/

Day-to-day logs from Sevak (one file per day, dated). Useful for troubleshooting. For more detail, set `SEVAK_LOG=debug` before starting Sevak.

Automatic cleanup: Logs older than 7 days are deleted automatically.

Size: Typically a few kilobytes per day under normal use.

usage.json

Records searches and actions you perform: what you typed, which results you ran, timestamps. Used for:

- Query history: press Up/Down on an empty search bar to recall your last ~50 searches.
- Usage statistics and analytics (local only; nothing is sent to Sevak developers).

Edited by: Sevak (you can manually edit it, but Sevak will overwrite it).

Privacy: Text you searched is recorded. Select what to delete (see "Clear history" below).

clipboard-history.json and clipboard/

Optional clipboard history (only if `[clipboard] enabled = true`). `clipboard-history.json` records the text you copy and the paths of files you copy, with timestamps. The `clipboard/` folder next to it holds each copied image as a PNG file plus a small thumbnail (only if `[clipboard] images = true`, the default once the history is on). Both are unencrypted.

Edited by: Sevak (when you copy, and when you type `cb clear` and run **Clear clipboard history**, which deletes the entries and the image files).

Privacy: Disabled by default because it records what you copy. Apps that mark their content as secret (password managers, etc.) are never recorded. Configure `[clipboard] ignore_apps` to never record copies from specific apps, and `[clipboard] images = false` / `files = false` to record text only.

currency-rates.json

Cached European Central Bank exchange rates, only if currency conversion (`[calculator] currency`) is on.

script-plugin-approvals.json

The script plugins and workflows you allowed to run. Delete it to be asked again for each of them.

plugins/ and workflows/ (data folder)

`plugins/<name>/` and `workflows/<name>/` in the data folder are where script plugins and workflow scripts may keep their own files (`SEVAK_PLUGIN_DATA`, `SEVAK_WORKFLOW_DATA`). Sevak creates the folder when the script first runs; what goes in it is up to the script.

themes/, plugins/ and workflows/ (config folder)

Your [theme files](#), [script plugins](#) and [workflows](#) (each a folder with a `workflow.toml` and its scripts). They travel with the config folder, so a synced config folder brings them along.

What is never written to disk

Contacts and the list of 1Password logins are kept in memory only. The keystrokes watched for [snippet expansion](#) (the last 64 characters) are kept in memory only, never logged. The Universal Actions selection is never stored.

Back up your data

Sevak's configuration and history are local, single files that are easy to back up:

```
cp -r ~/.local/share/sevak ~/backups/sevak-backup
```



or on Windows:

```
Copy-Item $env:APPDATA\sevak $env:USERPROFILE\backups\sevak-backup -Recurse
```



Restore from a backup by copying the folder back.

Reset to defaults

To reset Sevak to a fresh install while keeping your data:

1. Delete `config.toml` (or move it aside first). Sevak will write a new default config on next start.
2. Start Sevak.

To also clear history:

1. Delete `config.toml`.
2. Delete `usage.json` (or just the queries and actions you want to forget).
3. If clipboard history is enabled, delete `clipboard-history.json` and the `clipboard` folder.
4. Start Sevak.

Clear search history

Search history is stored in `usage.json`. To clear it entirely:

```
rm ~/.local/share/sevak/usage.json
```



On Windows:

```
Remove-Item "$env:APPDATA\sevak\usage.json"
```



Sevak will create a fresh file on next start.

To selectively delete searches, edit `usage.json` in a text editor (it is JSON format) and remove entries from the `searches` array.

Clear clipboard history

To delete all saved clipboard items, type `cb clear` in Sevak and press `Enter` on **Clear clipboard history**. It deletes the entries and the image files.

Or, with Sevak closed, delete the file and the image folder yourself:

```
rm -r ~/.local/share/sevak/clipboard-history.json ~/.local/share/sevak/clipboard
```



On Windows:

```
Remove-Item "$env:APPDATA\sevak\clipboard-history.json", "$env:APPDATA\sevak\clipboard" -Recurse
```

Sevak will create a fresh file on next use.

Uninstall cleanly

Windows

1. Open Settings → Apps → Installed apps, find Sevak, and click Uninstall.
2. The installer removes Sevak itself but keeps config and data.
3. To also remove config and data:

```
Remove-Item "$env:APPDATA\sevak" -Recurse
```

macOS

1. Delete the Sevak app from Applications.
2. This does not remove config and data.
3. To also remove config and data:

```
rm -rf ~/Library/Application\ Support/sevak
```

Linux

Uninstall using your package manager:

Ubuntu/Debian

```
sudo apt remove sevak
```

Fedora

```
sudo dnf remove Sevak
```

Other distributions

```
# Delete the AppImage  
rm ~/Sevak_*.AppImage
```

To also remove config and data:

```
rm -rf ~/.config/sevak ~/.local/share/sevak
```

Portable setup

You can place Sevak's config and data anywhere and point to it with environment variables:

```
export SEVAK_CONFIG_DIR=/mnt/portable/sevak/config
export SEVAK_DATA_DIR=/mnt/portable/sevak/data
sevak
```

or

```
sevak --config /mnt/portable/sevak/config
```

This is useful for running Sevak from a USB drive or shared network folder. The `--config` flag overrides the default but not the environment variable if both are set; the flag takes precedence.

Troubleshooting file access

"Cannot access config file" error:

- Check file permissions. Your user must be able to read and write the config folder.
- Check disk space. If the drive is full, writing a new config may fail.
- Check paths with special characters (non-ASCII, spaces). Ensure the path is valid on your system.

Logs are not being written:

- Check that `~/.local/share/sevak/logs/` (or equivalent) exists and is writable.
- Set `SEVAK_LOG=debug` to enable debug logging, but this is stored in memory unless persisted to a file.
- Restart Sevak after changing the data folder.

Old clipboard history or usage data still showing:

- Sevak loads these files at startup. Changes made externally are not live-reloaded.
- Restart Sevak after editing or deleting `usage.json` or `clipboard-history.json`.

Help

Troubleshooting

Start here

Before diving into specific issues, try these first:

1. **Confirm Sevak is running:** Look for the tray icon (Windows/Linux) or menu bar icon (macOS). If it is missing, open Sevak from your application menu or run `sevak` in Terminal.
2. **Check your hotkey:** Open Settings and look at the configured shortcut under General. Try a different key if the current one is already in use by another app or your desktop.
3. **Reload the index:** After installing a new app or changing file search settings, open the tray menu and choose **Reload index**.
4. **Check the logs:** If the problem persists, open Settings → "Reveal logs folder" and look at the most recent log file. Enable debug logging with `SEVAK_LOG=debug sevak` for more detail.

The hotkey does not work

Diagnosis: Open the tray/menu bar icon and click "Show". If the launcher appears, Sevak is running and the hotkey is not registered correctly.

Try these steps:

1. **Choose a different key** in Settings → General. Your current key may be in use by another app or your desktop.
2. **macOS:** The default is `⌘ Cmd + Space`, which is also Spotlight's shortcut. Sevak asks once whether it may turn Spotlight's shortcut off; if you said No it uses `⌘ Option + Space` (Alt and Option are the same key on Mac). See [Win+Space, Cmd+Space and Super+Space](#).
3. **Linux Wayland (GNOME):** Sevak cannot register global keys on Wayland. Run this command:

```
sevak --setup-hotkey
```

This creates a GNOME custom keyboard shortcut. If GNOME's input-source switcher uses the same key (`⌘ Super + Space`), the command offers to move it; see [below](#). For other conflict warnings, follow the instructions to free the key or choose another one.

1. **Linux Wayland (other desktops):** Bind a key such as `⌘ Alt + Space` to `sevak --toggle` in your desktop's keyboard settings (KDE Settings, Sway config, etc.).
2. **Linux X11 on GNOME:** `⌘ Super + Space` switches input sources: Sevak asks once whether it may move that shortcut (see [below](#)). `⌘ Alt + Space` is reserved for the window menu. Try `⌘ Ctrl + Space`, or free `⌘ Alt + Space`:

```
gsettings set org.gnome.desktop.wm.keybindings activate-window-menu "[ ]"
```



Win+Space, Cmd+Space and Super+Space

Sevak's default shortcut is **Super+Space**: Win + Space on Windows, Cmd + Space on macOS and Super + Space on Linux. Every system already uses that key (Windows switches the input language, macOS opens Spotlight, GNOME switches input sources), so Sevak takes it over. **Settings** → **General** → **Shortcut** always says how the key reaches Sevak:

Status line	Meaning
(nothing)	Registered with the system in the normal way
Taken over with the Windows keyboard hook	Windows, no setting changed
Spotlight's shortcut is turned off, with your permission	macOS
GNOME's input-source shortcut was moved ...	GNOME
Cmd+Space stays with Spotlight, so Sevak is using Option+Space	macOS, you said No

Existing installations keep the shortcut already in their `config.toml`; only new config files default to Super+Space. Sevak never edits another app's settings.

Windows: the input-language switch

Windows uses Win + Space to switch the input language. Sevak takes the key with a keyboard hook: it sees the key before Windows, recognises exactly your shortcut and swallows it, so Windows does not also switch language and the Start menu does not open when you release Win. Nothing in Windows' settings changes, and the key is Windows' again when Sevak quits.

- Only the exact combination is taken. Win + ↑ Shift + Space (previous input language) keeps working, as does Alt + ↑ Shift if it is your switcher. If you rely on Win + Space to change layout, switch with those or pick another Sevak shortcut.
- The hook only compares key presses with your shortcuts. It does not read, keep, log or send what you type.
- Windows does not hand keys of an administrator window to a program that is not running as administrator, so while such a window has focus the shortcut does not reach Sevak. Run Sevak as administrator if you need it there.

- If the status says the keyboard hook could not be installed (some security software blocks hooks), choose `⌘ Alt + Space` or another key Windows lets Sevak register.
- The hook also takes any other shortcut that another app has already registered, and says so in the status.

macOS: Spotlight

Spotlight owns `⌘ Cmd + Space`, so registering it fails while Spotlight's shortcut is on. The first time that happens Sevak shows a dialog: "Cmd+Space is used by Spotlight. Let Sevak use it?"

- **Yes:** Sevak turns off Spotlight's "Show Spotlight search" keyboard shortcut (it writes that one entry of `com.apple.symbolichotkeys` and asks macOS to reload its settings) and registers `⌘ Cmd + Space`. Nothing else about Spotlight changes. If the shortcut still opens Spotlight, log out and in again, or switch it off in System Settings → Keyboard → Keyboard Shortcuts → Spotlight.
- **No:** Sevak uses `⌥ Option + Space` instead and does not ask again.

Your answer is kept in `hotkey-takeover.json` in the data folder. To restore Spotlight's shortcut, click **Restore Spotlight's shortcut** in Settings → General, or run `sevak --restore-hotkey`. To let Sevak have the key later, click **Let Sevak use Cmd+Space** in Settings (it asks again). If a macOS update turns Spotlight's shortcut back on, Sevak turns it off again, because you allowed it.

GNOME: input sources

GNOME uses `⌘ Super + Space` to switch input sources (keyboard layouts). `sevak --setup-hotkey`, the **Set up GNOME shortcut** button (Wayland) and, on X11, the first failed registration of `⌘ Super + Space` offer to move that shortcut: `switch-input-source` and `switch-input-source-backward` in `org.gnome.desktop.wm.keybindings` get `^ Ctrl` added (`⌘ Super + Space` becomes `^ Ctrl + ⌘ Super + Space`, `⇧ Shift + ⌘ Super + Space` becomes `^ Ctrl + ⇧ Shift + ⌘ Super + Space`). Sevak shows the old and new values, changes nothing without your yes, logs what it changed and keeps the old values in `hotkey-takeover.json`.

To put GNOME's shortcuts back, click **Restore GNOME's input-source shortcut** in Settings or run:

```
sevak --restore-hotkey
```

Sevak's own shortcut may then clash with GNOME's again: choose another key, for example `⌘ Alt + Space` or `^ Ctrl + Space`.

Going back to Alt+Space

Open **Settings** → **General** → **Shortcut**, choose **Presets** → **Alt+Space** (Option+Space on macOS) and press **Save**, or set it in `config.toml`:

```
[general]
hotkey = "Alt+Space"
```

Reload with the tray menu. Then use the **Restore** button or `sevak --restore-hotkey` if you want the system shortcut back.

There is no tray icon on Linux

Sevak shows a tray icon through AppIndicator. Some GNOME sessions do not have the AppIndicator extension enabled.

Ubuntu: The extension is enabled by default. If there is no tray icon, check that GNOME Extensions are working.

Fedora: Install and enable the AppIndicator extension:

```
sudo dnf install gnome-shell-extension-appindicator
killall -HUP gnome-shell # or restart GNOME
```

Without a tray: Sevak still works. Use your hotkey to show it, `sevak --quit` to exit, and `sevak --settings` to open Settings.

An installed app is missing

- Choose **Reload index**, wait for indexing, and search again.
- Confirm the **Applications** plugin is enabled.
- Try the application's full name.
- Confirm the app has a platform launcher entry: a Start Menu shortcut or packaged app on Windows, an application bundle on macOS, or a desktop entry on Linux. Merely copying an executable into a folder may not create one.

A file or folder is missing

Check these in order:

Check	What to do
Query length	Use at least two characters after <code>f</code>
Indexed root	Add the containing folder in Settings → Files
Search depth	Increase depth if the file is nested too deeply
Dot-file visibility	Enable hidden-file indexing if needed

Check	What to do
Excluded directory	Generated/cache directories are deliberately pruned
New or moved file	Reload the index
Permissions	Ensure your user account can read that directory
Index size	Reduce broad roots if you hit the 100,000-entry cap

`f` matches names in your indexed folders, not document contents. A file outside them, or a word that is only inside a document, needs `ff` or `in` ([whole-disk search](#)). If those say "File index unavailable", start the Windows Search service (`WSearch`), install `plocate` and run `updatedb`, or install Tracker or Baloo, as the row explains; the OS only finds what it has indexed, so a file created seconds ago may take a moment to appear. [Full file-search behavior](#).

I get a web suggestion instead of a local result

The fallback web engine is offered when an ordinary query has no matches. It does not mean your local query has already been sent to that provider. A browser request happens when you execute the web result.

Check local plugins and indexing first, or disable the fallback under **Settings** → **Search**.

`c`, `1p`, `define` or `spell` does nothing

- **Contacts and 1Password are off by default.** The row says "Contacts are off" or "1Password is off"; add `[contacts] enabled = true` or `[onpassword] enabled = true` to `config.toml` and choose **Reload index**.
- **No contacts found.** The plugin reads vCard files from `vcard_files` and, with `use_system = true`, the system address book. On macOS the first `c` shows "Allow Sevak to read your Contacts": press Enter and answer the system question; if you said no, enable Sevak under *System Settings* → *Privacy & Security* → *Contacts*. On Windows the People store may be empty when your accounts' contacts are not synced there; export a `.vcf` file instead. On Linux only Evolution's local address books are read.
- **1Password says `op` was not found or is not signed in.** Install the [1Password CLI](#), turn on *Settings* → *Developer* → *Integrate with 1Password CLI* in the 1Password app, and run `op account list` in a terminal to check. If `op` is somewhere unusual, set `[onpassword] op_path`. After you dismiss the unlock prompt, press Enter on "Try again". Sevak never reads or shows passwords.
- **`define` finds nothing.** The bundled dictionary has single words and their common inflections, not phrases or names of people. Check the spelling with `spell`.
- **`spell` calls a word wrong that is right.** The bundled word list is general English. Specialist words are missing; on Linux the hunspell lists in `/usr/share/hunspell` are added when installed.

A web keyword does not work

Use `g rust traits`, with a space after `g`. Then check that:

- The engine exists and is enabled.
- Its keyword is unique and has no spaces.
- Its URL starts with HTTP(S) and contains `{query}`.
- Your custom TOML engine list includes all the default engines you still want.

Defining `[[web_search]]` entries replaces the default list. See [Web search configuration](#) for examples and details.

A calculation looks unexpected

Check parentheses and operators. `%` means remainder; use `200*15/100` for 15% of 200. Powers have precedence over a leading minus, so `-2^2` is `-4`; use `(-2)^2` for `4`. Press Enter to **copy** the answer, then paste it where you need it.

Sevak uses finite-precision arithmetic. Invalid expressions and operations such as division by zero do not produce a valid copyable answer.

An automation task or media control is missing

Sevak lists only what can work on your machine. Type `t` to see the tasks that are available. On Linux, tasks that need a helper program are hidden until it is installed: `wmctrl` (show desktop), `gnome-screenshot`, `spectacle` or `flameshot` (screenshot), `wpctl`, `pactl` or `amixer` (volume), `nmcli` or `rkill` (Wi-Fi), `rkill` or `bluetoothctl` (Bluetooth), `udisksctl` (eject), `systemd-inhibit` (keep awake) and `playerctl` (the media buttons and the playing track). On macOS, Bluetooth needs `blueutil`, and dark mode, volume, quit and hide-others ask once for permission to control System Events (hide-others and show-desktop also need the Accessibility permission). On Windows, Wi-Fi and Bluetooth are listed only when the PC has the radio, and switching them needs *Settings* → *Privacy & security* → *Radios* to allow desktop apps. Flush DNS is not offered on Windows or macOS because it needs administrator rights. You can also hide tasks in [\[tasks\]](#).

The list after `quit`, `kill` or `eject` appears a moment after you type the command, because Sevak asks the system for it in the background.

A snippet does not expand as I type

Expansion is off until you turn it on (**Settings** → **Plugins**, or `[snippets] auto_expand = true`), and the snippet needs a `keyword`. Then:

- Type the keyword in one go, with your `prefix` if you set one. Clicking, arrow keys, Enter, Escape, shortcuts and switching windows make Sevak forget what you typed. Without a prefix a keyword only works at the start of a word.

- With `expand_on = "delimiter"` finish with a space or punctuation mark.
- Sevak does nothing in its own windows, in terminals (set `expand_in_terminals = true`), in apps listed in `ignore_apps`, in a password box, or in windows running as administrator when Sevak is not.
- **macOS:** allow Sevak under *Privacy & Security* → *Input Monitoring* and *Accessibility*, then quit and start Sevak again. Settings shows what is missing.
- **Linux Wayland:** not possible; Wayland does not allow it. Use X11 or paste snippets with `s <name>`.
- Security software may block or warn about the keyboard hook; allow Sevak. Turning the setting off removes the hook.

[How expansion works and what it watches.](#)

The launcher disappears

This is expected with **Hide on blur** enabled: the launcher hides when another window takes focus. Turn it off under Settings → General if you prefer. Pressing Esc also hides the launcher; **Quit** in the tray stops the app.

Settings will not save

The Settings window shows validation errors if there are problems. Fix them in order:

Common issues:

- Duplicate web search keywords (each must be unique)
- URL without `{query}` placeholder
- Invalid or empty hotkey
- Fallback web search engine that no longer exists
- Blank fields where text is required

Hand-edited TOML files:

Check for syntax errors (unmatched quotes, missing brackets, typos in section names). If the file can't be read, Sevak starts with default settings (or, on **Reload index**, keeps the settings already in use) and writes the reason to the log. See [Files and data](#) for the config path and how to back it up.

An update check fails

Confirm internet access and that GitHub Releases is reachable. The release must include a valid update manifest and a compatible package. Automatic checks run in release builds; development/debug builds skip them.

Use **Check for updates** in the tray to see the immediate outcome. You can also check the [release page](#) and follow the [installation guide](#). Turning off automatic checks does not disable the manual menu action.

Why did the installer show an operating-system warning?

Updater signatures and OS code signing are different mechanisms. An update can be signed for Sevak's updater while an installer is not yet signed for Windows or notarized by Apple. Read the [platform installation notes](#) for the current status and make sure the package came from the project's release page.

Does Sevak work offline?

App search, indexed file search, calculations, contacts, the dictionary and spelling, and local ranking work without network access. Web searches, checking for a release, and downloading updates need an internet connection, as do the optional workflow and theme galleries and currency conversion. Indexes depend on locally accessible applications and folders.

Where is my data? Does Sevak collect analytics?

No telemetry. Sevak does not collect analytics. All config, usage history, and logs stay on your machine.

Network activity:

- **Optional update checks** (default on): Check GitHub for a new version at startup and daily. Disable with `[general] check_for_updates = false`. An update is downloaded only after you agree.
- **Web searches and links:** When you run a web search or open a bookmark, your browser contacts that site (Google, YouTube, GitHub, etc.). Sevak does not proxy or log these.
- **Currency conversion** (off by default): When enabled, Sevak downloads the European Central Bank's daily rates once per day for currency conversion.
- **Theme and workflow galleries:** Only when you click **Browse online themes** or **Load gallery**, and a package only when you click **Install**.

The full list is in [Privacy](#). See [Files and data](#) for where everything is stored, and [Configuration](#) for the settings.

Reporting a problem

Open an issue through the [repository's issue form](#) and include:

- Sevak version and operating system.
- Linux desktop and X11/Wayland session type, if relevant.
- The query or shortcut that reproduces the issue.
- What you expected and what happened.
- Relevant recent log lines, with private paths or unrelated data removed.

Send security issues through [SECURITY.md](#) instead of a public issue.

Frequently asked questions

Getting started

How do I change the hotkey?

Open **Settings** → **General**, then change **Shortcut** to a key combination not already in use by your desktop or another app. Click **Save**.

On **Linux Wayland**, click **Set up GNOME shortcut** to create the binding in your desktop.

More: [Custom hotkeys](#).

How do I make Sevak start at login?

Open **Settings** → **General** and turn on **Launch at login**. Click **Save**.

Why doesn't my search show any results?

1. Check that the plugin is **enabled** in **Settings** → **Plugins** (e.g., Applications, Files)
2. If searching files with **f**, make sure the folder is added in **Settings** → **Files** → **Folders to search**
3. Reload the index: click the tray icon and select **Reload index**

Why is a recent app or file missing?

After installing an app or adding files, choose **Reload index** from the tray menu to refresh Sevak's index. New results appear on the next search.

How do I search inside a folder?

Type the path: `~/Documents/` (note the trailing slash). Sevak lists the contents. Use **↑/↓** to select and **Tab** to drill down. **Shift+Tab** goes back up.

For ordinary searches to also show file results, turn on **Settings** → **Search** → **Show in global results**.

Universal Actions

What is Universal Actions?

Press **Ctrl + Alt + Space** (or your configured hotkey) to show what you can do with what you have selected: text, a link, or files in a file manager.

Sevak copies the selection, shows the available actions and puts your clipboard back. Read more: [Universal Actions](#).

Why doesn't Universal Actions work on Wayland?

Applications on Wayland cannot read each other's selections or send keystrokes. Workaround: enable **Use the clipboard if the selection can't be read** in **Settings** → **General**, copy the text yourself, then press  +  + .

Why doesn't Universal Actions work in terminal windows?

On Windows and Linux, Sevak skips terminal windows (because  +  would interrupt the running program). Select the text with your mouse and use the clipboard fallback instead.

Appearance and display

How do I use a dark theme?

Open **Settings** → **Appearance** and set **Theme** to **Dark**. Click **Save**.

Or set `[appearance] theme = "dark"` in `config.toml`.

Can I customize the colors?

Yes. Open **Settings** → **Appearance**:

- **Accent color**: set the highlight color (#rrggbb or rgb(...))
- **Custom stylesheet**: name a CSS file in your config folder to override any theme variable

More: [Themes and appearance](#).

The launcher is too small or too big. Can I resize it?

Open **Settings** → **Appearance** and adjust **Window width**. The height scales with the font size.

To change the font size, adjust **Font size** under Appearance (affects the whole launcher).

Files and searching

How do I search only files?

Type the keyword followed by a space and filename. Default keyword is `f: f project`.

Change or disable the keyword in **Settings** → **Files** → **Keyword**.

More: [File search](#).

Can Sevak search inside document text?

Yes, through your operating system's own file index: type `in` and the words, for example `in invoice 2026 . ff report` searches file names anywhere on the disk. `f` still searches names in the folders you chose. The queries go only to the local index (Windows Search, Spotlight, Tracker or Baloo), never over the network. See [Whole-disk and content search](#).

The file index is incomplete or seems stuck.

1. Check in **Settings** → **Files** that the folder is listed
2. Check that **Folder depth** is enough for your nested folders
3. Choose **Reload index** from the tray menu
4. Wait for indexing to finish (check the launcher: it shows a status message if indexing is running)

If a folder is slow or unreachable, indexing continues without blocking.

Calculations and conversions

Why doesn't currency conversion work?

Currency conversion is **off by default** because it needs a network connection. Turn it on in **Settings** → **Plugins** under **Currency conversion**. Click **Save**.

On the first search, Sevak downloads exchange rates from the European Central Bank and caches them for 24 hours.

Can I use other currencies?

Sevak supports 30+ ISO currency codes and symbols. Use `100 usd in eur`, `50 € to $`, or `$100 in eur`. See [Currency conversion](#).

How do I calculate percentages?

The `%` operator is remainder, not percentage. For 15% of 200, type `200*15/100` or `200*0.15`.

More: [Calculator](#).

Customization and advanced

How do I add a keyboard shortcut that runs a command?

In **Settings** → **Hotkeys**, click **Add hotkey**:

- Set the key (e.g., `^ Ctrl` + `⌘ Alt` + `T`)
- Choose **Query** and type `>` to open the launcher with a shell command ready
- Or choose **Run** and type a result ID (e.g., `system:lock`)

More: [Custom hotkeys](#).

Can I add my own search engine?

Yes. Open **Settings** → **Web search**, click **Add engine** and fill in:

- **Keyword:** what you type (e.g., `so` for Stack Overflow)
- **Name:** display name
- **URL:** search URL with `{query}` placeholder (e.g., `https://stackoverflow.com/search?q={query}`)

Click **Save**.

More: [Web search keywords](#).

Can I preview a file without opening it?

Yes. Tap `↑ Shift` or press `^ Ctrl` + `Y` to open the preview pane under the results: text files, images, folders, links, snippets and clipboard entries. `^ Ctrl` + `T` opens a long text in a scrollable view. See [Preview, Text View and Grid View](#).

Can Sevak expand text as I type, like a text expander?

Yes, opt-in. Give a snippet a `keyword` and turn on **Settings** → **Plugins** → **Expand snippets as you type**. While it is on, Sevak watches your keystrokes (the last 64 characters, in memory only). See [Expand snippets as you type](#).

What is the difference between a script plugin and a workflow?

A script plugin adds a keyword whose results come from your script. A [workflow](#) chains a trigger (a keyword, a hotkey, Universal Actions, `sevak --trigger`) to several steps, such as running a script, transforming text, opening a link, pasting and showing a notification, built visually in **Settings** → **Workflows**. Both ask for permission before they run code.

How do I write a script plugin?

Script plugins let you add custom keywords without rebuilding Sevak. Put a folder with a `plugin.toml` and a script in the `plugins` folder next to `config.toml`.

See [Script plugins](#) and the [Plugins guide](#). Three examples are in `examples/plugins/`.

Privacy and data

Does Sevak send my data anywhere?

No. Your searches, files, clipboard and settings stay on your machine. The network requests Sevak itself makes are:

1. **Links you open**: web searches and bookmarks open in your browser
2. **Update checks** (on by default, `[general] check_for_updates`): checks GitHub at startup and once a day; an update is downloaded only after you agree
3. **Currency rates** (off by default): downloads ECB daily rates once per day if you enable it
4. **Theme gallery** and **workflow gallery**: only when you click **Browse online themes** or **Load gallery**, and a package only when you click **Install**

Nothing is uploaded. [Read the full privacy policy.](#)

Where are my settings and data stored?

See [Files and data locations](#).

Can I sync my config to another machine?

Yes. Copy your config folder to another computer (or store it in a synced folder like Dropbox). Sevak reads it on startup.

See [Files and data locations](#) for where your config is stored.

Troubleshooting

The launcher won't open. What do I do?

1. Check that Sevak is running: look for it in your system processes
2. Try the CLI: open a terminal and run `sevak` or `sevak --toggle`
3. Restart Sevak: run `sevak --quit`, then open it again
4. Check the logs: **Settings** → **Open config file** folder, look for logs

More: [Troubleshooting guide](#).

I see an error message in Settings. What does it mean?

Validation errors appear under the field that has a problem (shown in red). Fix the value and try again. Common issues:

- **Hotkey already in use**: the key is registered by another app. Choose a different one.
- **Invalid path**: the file or folder does not exist or is not accessible. Check the path.
- **Invalid format**: the value does not match the required format (e.g., color must be `#rgb` or `#rrggbb`)

More: [Troubleshooting](#).

Sevak takes a long time to start. Can I speed it up?

1. Reduce the number of indexed folders in **Settings** → **Files** → **Folders to search**
2. Lower **Folder depth** to skip deep nested directories
3. Disable plugins you don't use in **Settings** → **Plugins**
4. If indexing is slow, exclude large generated directories (e.g., `node_modules`, `.git`) — they are usually pruned automatically, but check your index

You can run Sevak in the background with `sevak --background` and press the hotkey to show it when you need it.

How do I reset Sevak?

Sevak reads `config.toml` at startup. To reset to defaults:

1. Close Sevak: `sevak --quit`
2. Open the config folder (**Settings** → **Open config file** folder)
3. Delete `config.toml`
4. Restart Sevak; a fresh config is created with defaults

Settings, usage history and clipboard history are in the same folder on Windows and macOS (see [Files and data](#) for Linux).

Wayland (Linux)

Why doesn't the shortcut work on GNOME Wayland?

Wayland prevents apps from registering global hotkeys directly. Sevak asks you to run `sevak --setup-hotkey` to create a GNOME desktop shortcut instead.

In Settings, click **Set up GNOME shortcut** to do this automatically, or run the command from a terminal.

More: [Setting up the hotkey on Linux](#).

Can I use Sevak on other Wayland desktops (KDE, Weston, etc.)?

Partial support. On KDE and other desktops, you can:

- Use `sevak --toggle` from a terminal or custom keybind
- Leave Sevak running and use the tray icon

Global hotkeys are not possible without desktop-specific setup (if available). Universal Actions and pasting do not work on Wayland.

See [Installing on Linux](#).

macOS

How do I allow pasting and Universal Actions?

System Settings → **Privacy & Security** → **Accessibility**: add Sevak to the list. This permission lets Sevak paste results and read your selection for Universal Actions.

Why doesn't Cmd+Space work?

Cmd+Space is Spotlight's key and Sevak's default. The first time it cannot register it, Sevak asks whether it may turn off Spotlight's shortcut; if you said No, it uses Option+Space instead. You can change your mind in **Settings** → **General** (**Let Sevak use Cmd+Space**, or **Restore Spotlight's shortcut**). See [Win+Space, Cmd+Space and Super+Space](#).

Performance and limits

How many files can Sevak index?

The index is capped at **100,000 entries**. If you hit the limit, reduce the number of folders or lower the depth.

Is there a limit to how many snippets I can create?

No hard limit, but very large configs (100+ entries) may slow down loading slightly.

Why is search slow when I have thousands of files?

File search uses fuzzy matching on filenames, which is fast. If search is slow, check:

1. Are you searching a very large folder tree? Reduce **Folder depth** or add specific folders only
2. Is indexing still running? Look for a status message in the launcher
3. Are you on a slow or network drive? Local drives are faster

Questions?

Still stuck? Check the [Troubleshooting guide](#) or open an issue on [GitHub](#).

Privacy

Sevak is local-first. Your data stays on your machine except when you explicitly choose a network feature.

What stays local

- Application index, files and bookmarks (bookmarks are read from your browsers' files, read-only)
- Whole-disk and content file search (`ff` , `in`): queries go only to your computer's own file index (Windows Search, Spotlight, `locate` , Tracker or Baloo)
- The preview pane: it reads the selected file or folder from your disk, only while it is open; links are shown as addresses and never fetched
- Search history (if enabled)
- Clipboard history (if enabled), including copied images (PNG files) and the paths of copied files, unencrypted
- Snippet library, and snippet expansion as you type (if enabled): the last 64 typed characters are kept in memory only, never stored, logged or sent
- Contacts (if enabled) and the 1Password list of logins (if enabled): in memory only, and kept out of the search history and usage statistics
- The dictionary and spelling checker (bundled WordNet data or the system's own), the emoji picker, automation tasks and media controls
- Themes: built-in, edited, imported or exported
- Settings and configuration
- Usage statistics (how often you run each result)
- Logs (for debugging)
- Script plugins, workflows and their data

All data is stored in a single folder on your machine; none is uploaded anywhere.

What touches the network

Only when you opt in or use network features. Every network request Sevak itself makes is listed in this section: links you open, update checks, update downloads, currency rates, the theme gallery and the workflow gallery.

Web search and links

When you search with a web keyword or open a web search result, a bookmark or any other link (including a Universal Actions web search), the address opens in your browser. For a web search, your query is sent to the selected search engine (Google, YouTube, GitHub, etc.) in your browser. Your search engine provider receives the query. This is **always your choice** — you type a web keyword and press Enter, or click a web result.

Example: typing `g rust traits` constructs `https://www.google.com/search?q=rust+traits` and opens it in your browser. Google sees `rust traits` as your query.

Currency conversion

Opt-in feature (off by default). When enabled via `[calculator] currency = true`:

- Sevak downloads the **European Central Bank's daily reference rates** once per day
- URL: `https://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml` (small XML file, ~1.5 KB)
- No account or API key required
- Downloaded in the background (not during typing)
- Cached locally and reused for 24 hours
- Kept in `currency-rates.json` in your data folder

The ECB does not require authentication. Your IP address and request timestamp are visible to the ECB's servers, as with any web request. Sevak does not send additional context.

Disable with `[calculator] currency = false` to turn off the network request.

Update checks

Opt-in feature (on by default). When enabled via `[general] check_for_updates = true`:

- Sevak checks **GitHub Releases** at startup and once per day for a new version
- URL: `https://github.com/ninad-k/Sevak/releases/latest/download/latest.json` (small JSON metadata)
- No account required
- Done in the background (not during typing)
- Checking only; updates are never installed without your permission

Disable with `[general] check_for_updates = false` to turn off the network request.

Installing an update

When an update is available and you agree to install it, its package is downloaded from GitHub Releases and its update signature is verified before it is installed. Windows installation may also download Microsoft's WebView2 runtime if it is missing.

Theme gallery

Only when you click **Browse online themes** (**Settings** → **Appearance** → **Theme editor**):

- One request for `https://raw.githubusercontent.com/ninad-k/Sevak/main/gallery/themes.json`, the list of community themes

- **Install** on a theme downloads that one theme file, saved only if its SHA-256 matches the one in the list
- Nothing is requested in the background or on startup

See [Theme gallery](#).

Workflow gallery

Only when you press **Load gallery** (**Settings** → **Gallery**):

- One request for `gallery/index.json` from `raw.githubusercontent.com`, the list of ready-made workflows and script plugins
- **Install** on an entry downloads that one package, checked against the checksum in the index before anything is written; the installed folder still has to be allowed before it runs
- Nothing is requested in the background or on startup

See [The gallery](#).

Both galleries send nothing but the request itself (no cookies or identifiers beyond a `Sevak/<version> (gallery)` user agent). Sevak never downloads plugins or workflows on its own.

What is NOT collected

- No telemetry (no tracking, analytics, or usage data sent anywhere)
- No crash reports
- No advertising
- No profiling or behavioral analysis
- No timestamps of your searches or results
- No identifiers or user profiles

Script plugins and workflows run with your permissions and can make network requests themselves; review what they do before installing them. Workflows send nothing themselves and keep what you type or select out of the logs. 1Password's `op` tool, which the optional 1Password plugin runs when you type `1p`, talks to 1Password as it normally does; Sevak asks it only for the list of logins, never passwords.

Local data files

Inside your config folder (see [Files and data locations](#)):

File	Purpose	Sent elsewhere?
<code>config.toml</code>	Your settings	No

File	Purpose	Sent elsewhere?
<code>usage.json</code>	Frequency/recency of results; search history (if enabled)	No
<code>clipboard-history.json</code> , <code>clipboard/</code>	Clipboard history: text, paths of copied files, images (if enabled)	No
<code>currency-rates.json</code>	Cached ECB rates (if currency enabled)	No
<code>script-plugin-approvals.json</code>	Script plugins and workflows you allowed	No
<code>hotkey-takeover.json</code>	Whether you allowed Sevak to take Spotlight's or GNOME's shortcut, and the old GNOME values (so they can be restored)	No
<code>plugins/</code> folder	Script plugins and their data	No, unless the plugin makes network requests
<code>workflows/</code> folder	Workflows and their data	No, unless a workflow's script makes network requests
<code>themes/</code> folder	Theme files	No
Logs	Diagnostic output for troubleshooting	No (you can share them manually)

Clipboard behavior

- **Pasting:** when you paste a result, Sevak hides, brings the previous window back, and presses `⌘ + V` (`⌘ Cmd + V` on macOS). The app itself receives the text.

- **Copying:** `⌘+C` in the launcher copies the selected result's value or path to your clipboard. Your OS clipboard history (Windows Win+V, macOS, or a clipboard manager) may record it.
- **Accessibility:** on macOS, pasting and Universal Actions need the **Accessibility** permission. You grant this once in System Settings.

The launcher shortcut

The default shortcut, Super+Space, is also used by the system. How Sevak takes it over:

- **Windows:** a keyboard hook, only while a shortcut needs it (Win+Space, or a key another app has registered). It compares each key press with your configured shortcuts and swallows those; it does not read, keep, log or send what you type, and no Windows setting changes. The hook is removed when you quit Sevak.
- **macOS and GNOME:** Sevak changes Spotlight's shortcut or GNOME's input-source shortcut only after you say yes in a dialog, and not at all if you say no. What you answered and the previous GNOME values are kept in `hotkey-takeover.json` in the data folder, so Sevak never asks twice and can restore the old settings. Nothing is sent anywhere.
- Settings in other apps are never changed.

See [Win+Space, Cmd+Space and Super+Space](#).

Snippet expansion as you type

Off by default (`[snippets] auto_expand`). While it is on, Sevak watches your keystrokes to notice a snippet keyword:

- Only the last 64 characters you typed are kept, in memory, and they are wiped whenever the text could have changed and after every expansion. They are never written to disk, logged or sent anywhere.
- Nothing is observed while the setting is off: expansion does not listen to the keyboard. (The Windows hook for the launcher shortcut, above, only compares key presses with your shortcuts.)
- Sevak's own windows, terminals, apps listed in `[snippets] ignore_apps` and password boxes the system can detect are skipped.

See [Expand snippets as you type](#).

Selection reading (Universal Actions)

When you press the Universal Actions hotkey:

1. Sevak reads your system clipboard (and on X11 Linux, your PRIMARY selection)
2. Presses `⌘+C` (`⌘+C` on macOS) in the app you were using
3. Waits up to 0.3 seconds for the copy to complete

4. Reads the new clipboard content (text or files)
5. Restores your previous clipboard

The original app makes the copy, not Sevak. Your OS clipboard history may see it. Sevak's own clipboard history does not record this copy. The selection is never stored.

On **Windows and macOS**, this works everywhere except admin windows (Windows only). On **Linux Wayland**, app-to-app selection reading is not possible; use the fallback: copy the text yourself and set `[actions]`

```
use_clipboard_fallback = true.
```

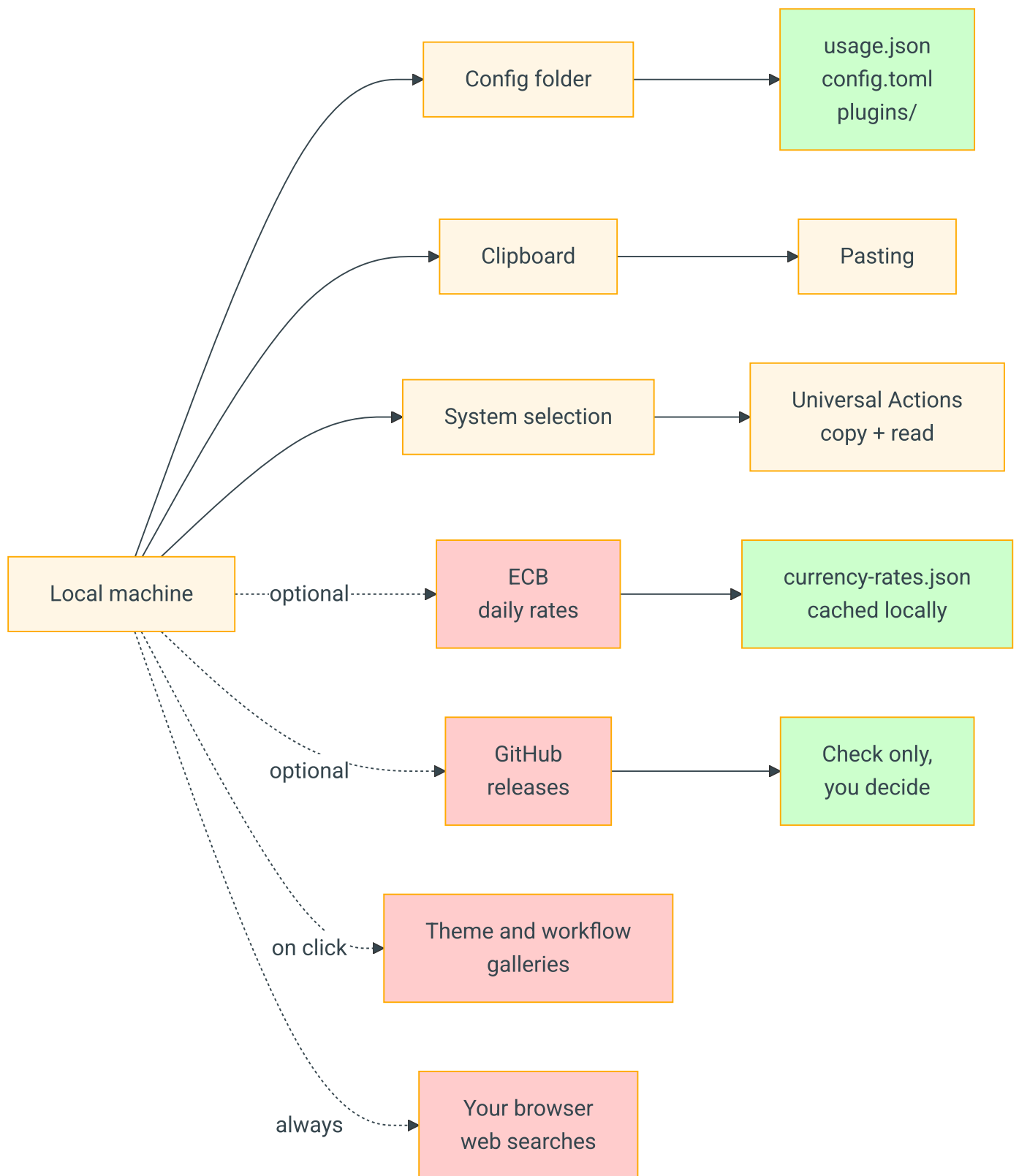
Data locations

See [Files and data locations](#) for where everything is stored.

Philosophy

Sevak is designed to be helpful without demanding trust. No cloud, no accounts, no surprises. You can verify what it does by reading the source code on GitHub.

Architecture diagram



Red borders (top-right) = network. Green borders (bottom) = stays local.

Developers